目次

Module 0. 機器學習的常見概念 5
資料基本用語 5
機器學習的種類 5
機器學習的流程 6
垃圾進，垃圾出（Garbage In, Garbage Out） 7
Module 1. 資料前處理 8
數值型資料 8
標準化 8
StandardScaler 9
MinMaxScaler 11
RobustScaler 12
Normalizer 13
類別型資料 15
名目特徵 15
有序特徵 15
獨熱編碼 16
LabelBinarizer 16
LabelEncoder 17
遺漏值 18
SimpleImputer 18
KNNImputer 18
切割資料集 19
train_test_split 19
K-fold Cross Validation(CV) 20
異常值 21
偵測離群值 21
處理離群值 26
偵測新奇值（novelty detection） 27
選取重要特徵 29
依統計性質過濾 29
嘗試錯誤法挑選特徵 37
Module 2. 線性迴歸 40
簡單線性迴歸（Simple Linear Regression） 40
多元線性迴歸（Multiple Linear Regression） 40
共線性（Collinearity） 41

變異膨脹因子（Variance Inflation Factor，VIF） 42
線性迴歸模型評估指標 43
平均平方誤差（Mean Squared Error, MSE） 43
平均平方根誤差（Root Mean Squared Error, RMSE） 44
平均絕對誤差（Mean Absolute Error, MAE） 44
決定係數（R-squared, $R2$） 44
殘差分析 46
常態性 46
獨立性 47
變異數同質性 50
正規化的線性迴歸 51
Ridge Regression 51
Lasso Regression 53
Elastic Net 54
多項式迴歸（Polynomial Regression） 55
隨機森林迴歸（Random Forest Regression） 56
Module 3. 監督式學習：分類 58
迴歸 vs. 分類 58
評估分類器的效能 59
混淆矩陣與其它評估指標 60
多元分類器 61
分類結果的可靠度 62
邏輯斯迴歸 64
加入正規化 65
支援向量機 66
線性支援向量機 68
加入核函數處理非線性分類 68
樸素貝氏分類 71
決策樹 73
資訊增益（Information Gain） 74
決策樹分類與迴歸 76
整合多棵決策樹：隨機森林 79
K 最近鄰居 81
Module 4. 模型擬合、評估與超參數調校 84
工作流程 84
管道化 84
擬合 86

模型擬合（Fitting）..... 86
欠擬合（Underfitting）..... 86
過擬合（Overfitting）..... 87
評估模型效能 87
學習曲線與驗證曲線 87
P-R 曲線與 ROC 曲線 90
調校超參數 93
網格搜尋與隨機化搜尋 94
參數搜尋小技巧 96
處理類別不平衡..... 98
Module 5. 非監督式學習：降維與分群 102
主成分分析 102
提取主成分..... 105
解釋變異數..... 106
核主成分分析處理非線性對應 107
K-means 109
基本概念 109
建立 k-means 分群..... 109
評估分群結果..... 111
階層式分群 112
DBSCAN 分群..... 114
Module 6. 集成學習..... 116
袋裝法..... 118
Bootstrap 抽樣 119
Aggregating 整合 119
分類問題：使用多數決 120
迴歸問題：使用平均數 120
隨機森林 121
提升法..... 121
弱學習器 121
AdaBoost..... 122
堆疊法..... 124
基礎模型 124
元學習器 124
基本架構 125
訓練流程 125

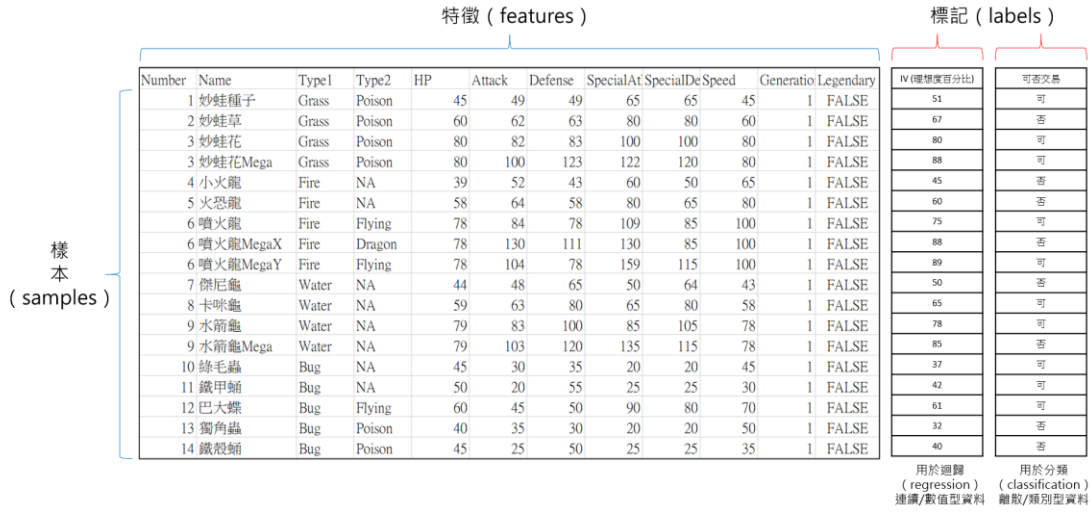
- GitHub 專案連結
  https://github.com/telunyang/python_machine_learning

Module 0. 機器學習的常見概念

能夠從現有的資料（data）當中，不斷地從中學習經驗（experience），找出共通的模式（pattern），以便引導未來的決策。在學習這門知識之前，強烈建議先行研讀統計學（含機率）、微積分以及線性代數，可以較容易地理解模型運作過程，以及它們背後的原理。

資料基本用語

資料（有時稱之為「數據」）的基本用語如下：



Number	Name	Type1	Type2	HP	Attack	Defense	SpecialAt	SpecialDe	Speed	Generatio	Legendary	IV (理想度百分比)	可否交易
1	妙蛙種子	Grass	Poison	45	49	49	65	65	45	1	FALSE	51	可
2	妙蛙草	Grass	Poison	60	62	63	80	80	60	1	FALSE	67	否
3	妙蛙花	Grass	Poison	80	82	83	100	100	80	1	FALSE	80	可
3	妙蛙花Mega	Grass	Poison	80	100	123	122	120	80	1	FALSE	88	可
4	小火龍	Fire	NA	39	52	43	60	50	65	1	FALSE	45	否
5	火恐龍	Fire	NA	58	64	58	80	65	80	1	FALSE	60	否
6	噴火龍	Fire	Flying	78	84	78	109	85	100	1	FALSE	75	可
6	噴火龍MegaX	Fire	Dragon	78	130	111	130	85	100	1	FALSE	88	否
6	噴火龍MegaY	Fire	Flying	78	104	78	159	115	100	1	FALSE	89	可
7	傑尼龜	Water	NA	44	48	65	50	64	43	1	FALSE	50	否
8	卡咪龜	Water	NA	59	63	80	65	80	58	1	FALSE	65	可
9	水箭龜	Water	NA	79	83	100	85	105	78	1	FALSE	78	可
9	水箭龜Mega	Water	NA	79	103	120	135	115	78	1	FALSE	85	否
10	綠毛蟲	Bug	NA	45	30	35	20	20	45	1	FALSE	37	可
11	鐵甲蛹	Bug	NA	50	20	55	25	25	30	1	FALSE	42	可
12	巴大蝶	Bug	Flying	60	45	50	90	80	70	1	FALSE	61	可
13	獨角蟲	Bug	Poison	40	35	30	20	20	50	1	FALSE	32	否
14	鐵殼蛹	Bug	Poison	45	25	50	25	25	35	1	FALSE	40	否

圖：資料的基本用語

註：樣本有時會被稱為「觀察值」（observation），標記有時會被稱為「目標」（target）或「標籤」。

機器學習的種類

根據解決的任務類型不同，所採用的機器學習方法，可粗略地分為三種：

- 監督式學習（supervised learning）：

  透過已知的標記（label）來建立模型，可以對未見的樣本資訊，來預測出它可能的答案（預測未見樣本的標記）。比較常見的監督式學習任務是分類（classification）和迴歸（regression）。在分類任務中，程式必須學習從一個或多個特徵來預測一個或多個離散值，例如預測一支股票的價格會「上漲」或「下跌」，或是決定一篇新聞屬於「政治版」還是「娛樂版」等；在迴歸任務中，程式必須學習從一個或多個特徵來預測一

個或多個連續型數值，例如預測一個新產品的銷售收入，或是根據一個職務的描述來預測其薪水。

- 非監督式學習（unsupervised learning）：

  在樣本沒有標記或不考慮標記（label）的情況下，透過既有的特徵（feature）來探索資料的結構，進而發現有用的資訊。比較常見的非監督式學習任務，就是在資料集當中，發現互相關聯的樣本，基於一些相似性衡量標準，把樣本放在和其它群組（group）相比，更加類似的群組當中，這樣的過程稱之為分群（cluster），例如在市場中為某項產品發現客戶群體，透過了解特定客戶群體的共同屬性，銷售人員可以決定採用何種銷售策略，用於該客戶群體。

- 強化學習（reinforcement learning）：

  經由與環境互動產生的獎勵（reward）來改善模型的效能，驅使模型朝既定的目標前進。例如下象棋，每吃掉對方一個棋子作為正面獎勵，被吃掉為負面獎勵，根據棋盤狀態以及先前落子的順序，來決定下一步的程動行為（action），透過不斷地反饋與獎勵進行學習。

機器學習的流程

1. 收集資料：

   明確地定義問題，同時收集相關的資料，是機器學習的第一步。資料的完整性對模型的預測效能有著關鍵性的影響，除了資料量要足夠大以外，代表性也很重要。資料的收集夠完整，才能順利進行後續的分析，這部分往往都是最耗時間。

2. 資料前處理：

   真實世界的資料是非常雜亂的，經常會有缺漏、雜訊，或是資料不平衡的情況發生，於是需要進行探索性資料分析（exploratory data analysis，EDA），大致上瀏覽資料的樣式與分佈程度，依需求進行資料清理（data cleaning），將資料整理成特定的格式，也可以透過特徵工程（feature engineering）來抽取、轉換或產生重要的特徵，方便之後的建模工作。

3. 訓練模型與調校：

   在訓練模型之前，會將整個資料集分成三個部分，一個是訓練資料集（training dataset），一個是驗證資料集（validation dataset），一個是測試資料集（testing dataset）。一個完整的資料集，通常訓練資料集佔 50% ~ 75%，測試資料集佔 10% ~ 25%，剩餘的部分，就是驗證資料集。資料切分完成後，依照問題的類型，選擇一個或數個機器學習模型來擬合（fit）訓練資料，藉由擬合過程來學習模型的參數，以建立

最終版本的模型。大部分機器學習的模型，都需要使用者預先提供的超參數（hyperparameters），這可以透過調校來取得，有效地發揮模型潛力。

4. 測試模型：

   模型建立完成後，會使用先前切分出來的測試資料集，依照效能量測指標進行測試，來評估（evaluation）模型擬合結果的好壞，再從幾個模型當中挑選相對合適的模型。

5. 改善效能：

   不斷地重複觀察與調整 3、4 步驟的過程來進行改善；必要時，可以重新進行 1、2 步驟，來取得更多有利於資料規律的特徵。

垃圾進，垃圾出（Garbage In, Garbage Out）

許多模型預測能力的提升，有賴於訓練資料數量的增加。然而，機器學習演算法（algorithm）遵循著格言「Garbage In, Garbage out」，假如一個學生透過閱讀一本錯誤百出、令人困惑的教材來準備考試，他的成績，並不會比閱讀教材篇幅短少，但內容品質較高的學生的成績來得好。在現實世界裡，如果在一個包括噪音（noise）、不相關或是帶有錯誤標記的資料集上進行訓練，其表現不會比一個在更能代表問題的小型資料集上所訓練的模型來得好。

在機器學習中，資料前處理是建模之前最重要的步驟之一。即使模型本身非常強大，如果輸入的資料品質不佳，例如資料型態混亂、存在大量遺漏值、特徵尺度差異過大、資料中包含異常值，或是不重要的特徵過多，都可能導致模型學習到錯誤的規律。如果輸入的是品質不佳的資料，模型輸出的結果也很難可靠。因此，在正式訓練模型之前，我們通常會先進行以下工作：理解資料型態、處理遺漏值、切割訓練與測試資料、偵測異常值，以及挑選重要特徵。

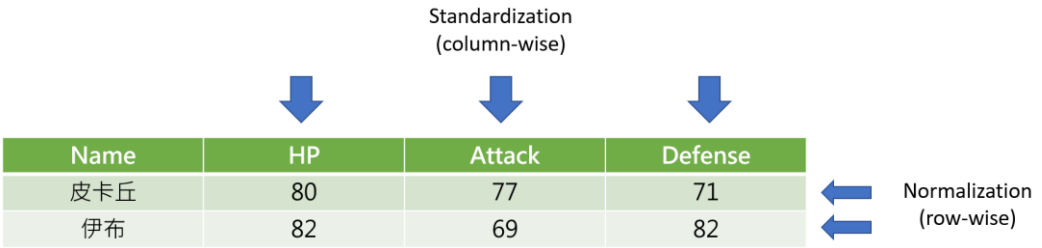
Module 1. 資料前處理

數值型資料

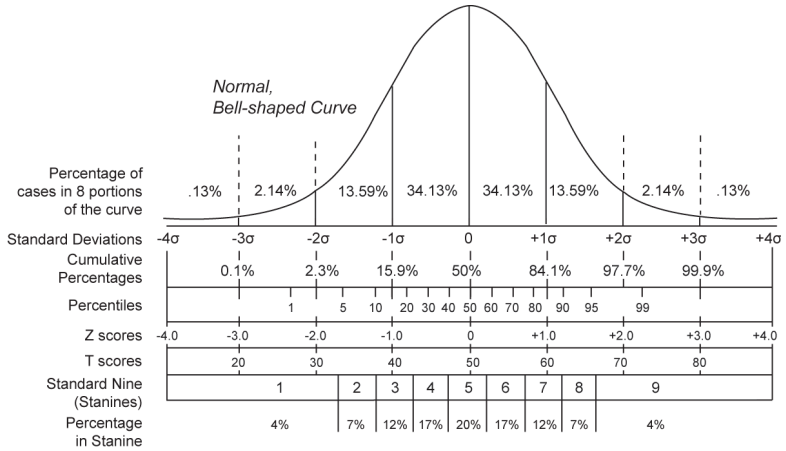
在機器學習中，數值型資料是指可以用數字表示，並且通常具有大小、距離或比例意義的資料。這類資料可以直接或經過適當處理後作為模型的輸入，例如年齡、身高、體重、收入、溫度、點擊次數、考試分數等。數值型資料常見於迴歸、分類與分群等任務中，模型可以根據這些數值之間的差異與變化，學習資料中的規律與關係。

標準化

將數值型特徵轉換為「平均數為 0、標準差為 1」的分佈，使不同尺度的特徵具有可比性，可提升訓練穩定性與收斂速度，目的是讓「不同特徵」變公平。



圖：對欄位進行標準化；對樣本進行正規化



圖：取自 https://zh.wikipedia.org/zh-tw/標準分數

補充
fit - 作用：計算並儲存模型參數（如平均值、標準差）。 - 適用對象：估計器（用於模型訓練）、轉換器（用於資料轉換）。 - 主要使用場景：參數計算。 transform - 作用：使用 fit 方法計算所得的參數，對資料進行轉換。 - 適用物件：轉換器。 - 主要使用場景：資料轉換，進行縮放、標準化或正規化。 fit_transform - 作用：先 fit（計算並儲存參數），再 transform（轉換資料）。 - 適用物件：轉換器。 - 主要使用場景：資料預處理時一步完成參數計算和資料轉換。 參考資料: 深入理解 Scikit-Learn 中的 fit, transform 和 fit_transform https://blog.csdn.net/jjqhj/article/details/140906700

StandardScaler

一種常見的標準化方法，用來將資料進行縮放和中心化處理，使資料的平均值（mean）變為 0，標準差（standard deviation）變為 1，大部分的資料會集中在 [-1, 1] 的範圍內，通常大約 68% 的資料點會落在這個範圍。這種方法通常應用於需要使不同特徵保持一致資料範圍的情境，例如在機器學習模型中，確保不同特徵不因為範圍不同而對模型產生不均衡的影響。

平均值為 0、標準差為 1 的資料通常指的是「**標準化**」後的資料，也稱為「**Z-score 標準化**」或「**Z 標準化**」。這是一種常用的資料預處理方法，尤其在機器學習模型訓練中很常見。讓我們深入了解這兩個概念的具體意義。

「平均值為 0」的意思是，資料經過轉換後，其所有資料點的**平均值**被調整到 0。計算平均值的公式是：

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

經過標準化後，資料的整體中心被平移到 0，表示資料的中心點與原點對

齊。換句話說，經過標準化後，大部分資料會集中在零附近，資料點大約一半位於零的左邊，另一半位於右邊。

「標準差為 1」的意思是，資料經過標準化後，其**離散程度**（分散程度）調整到 1。標準差衡量資料點相對於平均值的偏離程度，公式為：

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

當標準差為 1 時，資料的離散程度被縮放，使得資料點的變化範圍在 1 個標準差內。也就是說，絕大多數資料點都會落在 0 的附近，並在 -1 到 1 之間波動。

標準化資料的公式如下：

$$Z = \frac{X - \mu}{\sigma}$$

- Z：標準化後的數值。
- X：原始數值。
- μ：資料的原始平均值。
- σ：資料的原始標準差。

為什麼要標準化？

- **使模型對不同特徵同等重視**：

  如果資料中的不同特徵有著不同的範圍（例如收入用元表示，年齡用年表示），那麼在模型訓練過程中，某些數值較大的特徵可能會過分影響模型。標準化可以解決這個問題，讓所有特徵處於相同的範圍。

- **加速模型收斂**：

  某些機器學習演算法（如梯度下降法）在資料標準化後，收斂速度會更快，從而提高訓練效率。

- **有利於距離度量**：

  在需要計算距離的模型（如 KNN 或 SVM）中，標準化確保距離計算不會被數值較大的特徵主導。

注意事項

- **異常值的影響**：

  StandardScaler 使用均值和標準差進行標準化，因此對於包含極端異常值的資料集，這些異常值可能會對均值和標準差產生顯著影響，從而

影響資料的標準化效果。如果資料集中存在異常值，應該考慮使用對異常值更為穩健的標準化方法，如 RobustScaler。

- **資料分佈的假設**：

  StandardScaler 假設資料接近常態分佈。在資料分佈非常偏態（skewed）的情況下，標準化後的資料仍然可能存在一些問題，這時可能需要進行額外的資料處理。

MinMaxScaler

一種常見的資料標準化方法，它將資料縮放到指定的區間（通常是 [0, 1] 之間）。MinMaxScaler 的核心概念是**線性縮放**，即將資料中的最小值轉換為 0，最大值轉換為 1，其餘資料根據其相對位置映射到這個範圍內。

MinMaxScaler 的標準化公式如下：

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- X：原始資料值。
- X_{min}：資料集中該特徵的最小值。
- X_{max}：資料集中該特徵的最大值。
- X_{scaled}：經過標準化後的資料值。

注意事項

- 異常值的影響：

  MinMaxScaler 對異常值較為敏感，因為它的縮放是基於最大值和最小值。如果資料中有極端值，這些異常值會對縮放範圍產生很大的影響。例如，如果資料中存在非常大的異常值，則其他正常資料可能會被壓縮得非常小。因此，如果資料集中存在異常值，應考慮先去除異常值或使用對異常值更為穩健的標準化方法，如 RobustScaler。

- 資料分佈的假設：

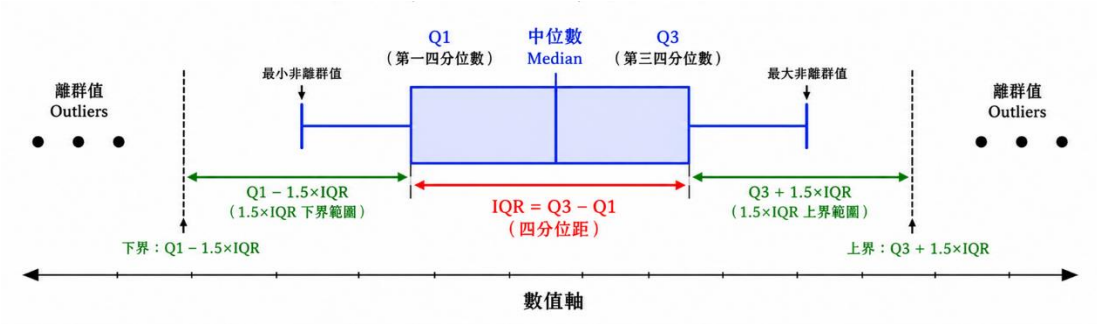
  MinMaxScaler 假設資料的分佈是均勻的。當資料的分佈非常不均勻時（例如，大部分資料集中在某個範圍，而少數資料遠離中心），可能會導致縮放後資料分佈過於壓縮。因此，對於這類資料分佈，應考慮其他標準化技術。

RobustScaler

一種資料標準化的方法，它的目的是對資料進行縮放和中心化處理，但與傳統的標準化（如 StandardScaler）不同，RobustScaler 對異常值（outliers）的影響較小。這使得它特別適合處理具有極端數值的資料集。

核心思想是使用中位數（median）和四分位距（IQR，Interquartile Range）來進行縮放，這與 StandardScaler 使用的均值和標準差不同。

- 中位數：是資料集中間的數字，對極端資料點（異常值）不敏感。
- 四分位距（IQR）：是上四分位數（75% 分位數）與下四分位數（25% 分位數）之間的距離。它是資料分布範圍的一個穩健度量，同樣對異常值的影響較小。



圖：IQR 圖解說明

RobustScaler 的標準化公式是：

$$X_{scaled} = \frac{X - Q2}{Q3 - Q1}$$

- X：原始資料。
- Q_2：中位數。
- Q_3：第 75 個百分位數（上四分位數）。
- Q_1：第 25 個百分位數（下四分位數）。
- $Q_3 - Q_1$：四分位距。

為什麼使用 RobustScaler？

- 對異常值（outliers）的抗性：

  傳統的標準化方法，如 StandardScaler，是基於均值和標準差，這

意味著資料集中存在的異常值會極大地影響均值和標準差，從而影響資料的標準化效果。而 RobustScaler 基於中位數和四分位距進行縮放，這兩個指標不受極端值的影響，因此能夠更加穩健地處理資料中的異常值。

- 適合具有異常值的資料集：

  如果資料集中存在極端值，使用 StandardScaler 會導致縮放後的資料不均衡，影響模型的表現。而 RobustScaler 可以有效減少這些極端值對資料標準化的影響，保證資料縮放後更加穩定。

RobustScaler 的應用場景

- 帶有極端資料點的資料集：

  當你的資料集包含一些異常值，而這些異常值可能會影響模型性能時，RobustScaler 是一個很好的選擇。

- 資料範圍變動較大：

  如果你的資料中存在大量分散且範圍寬廣的資料，RobustScaler 能夠在保持穩定性的同時，進行合理的縮放。

Normalizer

主要目的是將每個資料點（通常是向量）進行向量範數正規化，使得每個資料點的向量長度（即範數 norm）為 1。與其他標準化方法（如 StandardScaler 或 MinMaxScaler）不同，Normalizer 是針對每一個資料點（樣本）進行處理，而不是針對特徵（行），目的是讓「不同樣本」變可比。

Normalizer 通過將每個資料點縮放，使得該資料點的範數（norm）變為 1，這樣每個樣本都被縮放到單位範圍內。常用的範數有：

- L2 範數（歐幾里得範數）：也稱為平方和範數，通常用於標準的向量正規化，表示向量的平方和的平方根為 1。
- L1 範數（絕對值範數）：表示向量的元素絕對值之和為 1。

對於一個資料點（向量）$X = [x_1, x_2, \ldots, x_n]$，L2 範數的正規化公式為：

$$X_{normalized} = \frac{X}{\|X\|_2} = \frac{X}{\sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}}$$

這意味著，資料點中的每個元素都會除以該資料點的 L2 範數，使得結果向量的總長度為 1。

Normalizer 的工作原理

- 作用對象是資料點（樣本）：

  Normalizer 是針對每個資料點進行處理的，不像其他標準化方法那樣針對整個特徵列。每個資料點都被縮放，使其向量範數為 1。

- 使用範數進行縮放：

  Normalizer 根據指定的範數（L1 或 L2）將每個樣本進行縮放，使得資料的整體方向保持不變，但範數為 1。

- 保持資料方向：

  Normalizer 只改變資料的範數，保持資料的方向。這意味著原資料中兩個資料點之間的方向關係不變，只是資料點的長度被縮放到單位範圍內。

什麼時候使用 Normalizer？

- 稀疏資料：

  Normalizer 常用於稀疏資料集，如文本資料或詞袋模型中的詞頻資料。當你有一組向量，並且更關心資料的方向或模式而不是其大小時，Normalizer 是理想選擇。

- 保持資料的幾何形狀：

  如果你的模型（例如基於距離的模型，如 KNN 或餘弦相似度）更關心向量之間的相對幾何形狀（方向）而不是數值的絕對大小，Normalizer 能幫助統一向量的大小。

- 特徵向量的處理：

  在處理特徵向量時，尤其是高維度資料如圖像特徵、文本向量表示時，使用 Normalizer 來使每個資料點的向量保持一致的長度，可以讓模型更關注於特徵之間的相對關係，而不是數值本身的大小。

注意事項

- 範數的選擇：
  - 默認的範數是 L2，但在某些應用中，如當資料具有稀疏性時，L1 範數有時會更合適，因為它強調非零特徵的重要性。
- 與其他標準化技術的區別：
  - Normalizer 是針對每個樣本進行正規化，而其他標準化技術（如 StandardScaler 和 MinMaxScaler）則是針對每個特徵列進行處理。
  - Normalizer 保證了每個樣本向量的範數相同，但它不會改變資料點之間的相對大小（與標準化特徵的技術不同）。

面向	標準化（Standardization）	正規化（Normalization）
處理單位	每個特徵（feature-wise）	每筆樣本（sample-wise）
目的	消除不同特徵的尺度差異	統一每筆資料的向量長度
操作方式	減平均、除標準差	除以向量範數
分佈影響	轉為平均 0、標準差 1	不改變方向，只改變長度
常見用途	梯度式模型、距離模型	相似度計算（cosine）、文本向量

表：標準化是對「特徵」做縮放（平均 0、變異 1），正規化是對「樣本向量」做縮放（長度 = 1），兩者目的與使用情境不同。

類別型資料

類別型資料是用來表示類別或群組的資料，例如性別、顏色、地區、血型等。

血型	獨熱編碼	類別 ID	二進位編碼
A	1 0 0 0	0	0 0
B	0 1 0 0	1	0 1
AB	0 0 1 0	2	1 0
O	0 0 0 1	3	1 1

圖：樣本資訊透過 1 和 0 組合而成的稀疏向量作為表示

名目特徵

名目特徵（Nominal Features）是沒有內在順序的類別型資料。例如，血型就是名目特徵，因為「A 型」、「B 型」、「AB 型」、「O 型」之間沒有大小、次序的關係。這些值只是表示不同的類別而已。

有序特徵

有序特徵（Ordinal Features）則是帶有內在順序的類別型資料。例如，如果我們在某個場景中有評分等級（如「差」、「中」、「好」，或是「一年級」、「二年級」、「三年級」，…），這些數值是有順序的。類別型資料圖片中的資料似乎不涉及有序特徵，因為血型沒有順序關係。

獨熱編碼

獨熱編碼（One-Hot Encoding）是一種常用來將類別型資料轉換為數值的方法。對於每一個類別，獨熱編碼會建立一個二進制向量，在對應類別的位置上設置為 1，其他位置設置為 0。例如，血型「A」的獨熱編碼是 1 0 0 0，而「B」的獨熱編碼是 0 1 0 0。

獨熱編碼的缺點：

- 高維度問題：

  如果類別數非常多，獨熱編碼會導致資料的維度急劇增加，引發「維度災難」(curse of dimensionality)，這會帶來內存和計算資源的消耗問題。

- 稀疏矩陣：

  獨熱編碼產生的矩陣通常是稀疏的（大多數元素為 0），在機器學習模型中處理稀疏矩陣的效能可能不好。

LabelBinarizer

專門用來將類別型資料轉換成二進制表示。它常用於對類別型標籤進行處理，尤其是在多分類問題中。這個方法與獨熱編碼（One-Hot Encoding）類似，但也可以進行多重標籤（multi-label）的二進制化處理。

LabelBinarizer 的功能

- 將多類別標籤轉換為二進制數組。
- 在二分類情況下，則只返回一個二進制列。

主要特點：

- 二分類模式：

  如果只有兩個類別，LabelBinarizer 會將資料轉換為單列的二進制數組。例如，對於標籤 ['A', 'B']，它會輸出 [0, 1] 或 [1, 0]。

- 多類別模式：

  對於多個類別（如三個或更多），它會產生與獨熱編碼類似的結果，但不同於獨熱編碼，LabelBinarizer 更加靈活，特別是在處理多標籤問題時。

- 適用於多標籤資料：

LabelBinarizer 也支持將多標籤資料轉換為二進制表示。這與多標籤分類有關（比如一個樣本可以屬於多個類別），在這種情況下，它可以將多標籤資料表示成一個二維二進制矩陣。

LabelBinarizer 的優點：

- 簡單易用：它能夠簡化標籤轉換過程，尤其在進行分類任務時。
- 靈活性高：能夠處理多標籤問題，並根據不同的需求調整輸出格式。

LabelEncoder

專門用來將類別型資料轉換成數字標籤，它將每個類別標籤映射到一個唯一的整數值。

LabelEncoder 的功能

- 將類別型標籤（如文字標籤）轉換成對應的數字編碼。
- 適合於那些具有名目特徵的類別型資料。

LabelEncoder 的主要用途是處理類別標籤，使得它們能夠被大多數機器學習模型直接接受，因為大部分模型只接受數值型資料，而不是文字標籤。

工作原理

例如，假設你有一組類別標籤 ['A', 'B', 'C']，LabelEncoder 會將這些類別標籤映射到對應的整數，如：

A -> 0
B -> 1
C -> 2

主要特點

- 單列資料：

  LabelEncoder 是針對單一特徵進行轉換的，並且它返回的是整數標籤（而不是二進制編碼）。
- 適合名目特徵：

  LabelEncoder 適用於那些沒有內在順序的類別（如顏色、血型、性別等），但是其編碼結果的數字並沒有順序關係。
- 反轉編碼：

  使用 LabelEncoder.inverse_transform() 方法，你可以將編碼過

的數字重新轉換回原本的類別標籤。

方法	適合用在特徵 X 嗎？	適合用在目標 y 嗎？	說明
pd.get_dummies()	適合	較少用	適合處理類別型**特徵 x** 欄位
LabelEncoder	不建議	適合	適合把分類**標籤 y** 轉成整數
LabelBinarizer	通常不建議	適合	適合把分類**標籤 y** 轉成獨熱

表：在「類別型資料與數值型資料整合在一起訓練」時，若三者只能選一個，通常最合適的是 pd.get_dumies()

遺漏值

又稱缺失值（missing value），處理真實資料時常會遇到資料不完整或資料遺漏的情況，遺漏值除了容易造成分析結果的偏誤外，也難以直接交由機器學習模型來擬合。

SimpleImputer

用來處理資料中的缺失值。當資料集包含有遺漏值（如 NaN、空白或其他缺失標記）時，SimpleImputer 可以自動填補這些缺失值。它提供了多種填補策略，以確保資料完整性，從而能夠進行後續的分析或建模。常見的填補策略包括：

- 平均值填補（mean）：
  使用每列的平均值來填補缺失值。
- 中位數填補（median）：
  使用每列的中位數來填補缺失值，適合處理偏態較大的資料。
- 最常出現值填補（most_frequent）：
  用每列中最常見的值來填補缺失值。
- 常數填補（constant）：
  可以指定一個固定的值來填補缺失值。

KNNImputer

一個用於填補缺失值的類別，它基於 K 近鄰（K-Nearest Neighbors, KNN）演算法來估算和填補缺失的資料。與簡單的填補策略（如使用平均值或

中位數）不同，KNNImputer 是通過考慮資料集中與缺失值最接近的觀測點來進行更智能的填補。具體來說，它會基於其他相似樣本的值來推測缺失值。

KNNImputer 的運作原理：

- 對於每一個缺失值，KNNImputer 會找到資料集中與該行最相似的 K 個樣本（即最接近的鄰居）。
- 它會計算這些鄰居的非缺失值，並使用這些值的平均值來填補缺失的資料點。

這種方法在某些情況下比簡單的填補方法更準確，因為它根據樣本的整體結構來推斷缺失值，而不僅僅依賴單個特徵的統計值，且資料之間具有相似性時表現較好。例如，對於地理資料或時間序列資料，KNNImputer 可能會比簡單的平均值或中位數填補方法更有效。

切割資料集

train_test_split

主要目的是將原始資料集劃分為兩部分，一部分用來訓練模型（訓練集），另一部分用來評估模型的表現（測試集）。這樣的分割可以幫助避免模型過度擬合，也能更準確地評估模型在未見資料上的泛化能力。

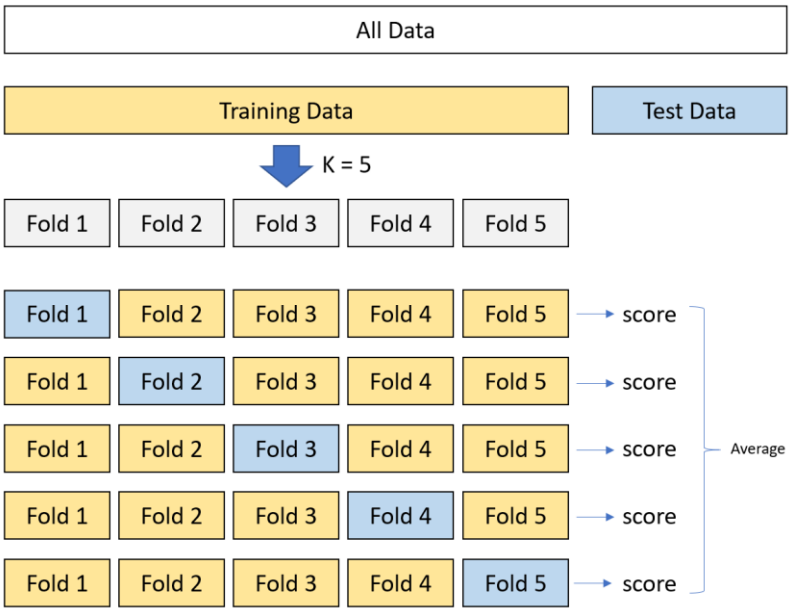
概念上，train_test_split 做了以下幾件事：

- 訓練集（Training Set）：

  用來訓練模型，讓模型學習資料中的模式或規律。這部分資料是模型在訓練過程中能夠「看到」的。
- 測試集（Test Set）：

  用來評估模型的準確性和泛化能力。測試集不會在模型訓練過程中使用，而是留到訓練完成後進行模型的最終評估。這能幫助檢查模型在未見資料上的表現。

K-fold Cross Validation(CV)



圖：交叉驗證，K = 5 的情形

是一種用來評估機器學習模型性能的技術，尤其適合於資料量有限的情況。它將資料集劃分為 K 個子集（folds），並進行多次訓練和測試。具體過程如下：

- 資料分割：

  將整個資料集隨機分為 K 個等份。
- 迭代訓練與測試：
  - 在第 i 次迭代中，選取第 i 個子集作為測試集，其餘的 K-1 個子集作為訓練集。
  - 訓練模型並在該測試集上評估性能（如準確率 Accuracy、F1-score 等）。
- 重複 K 次：

  每次都使用不同的子集作為測試集，並重複這個過程 K 次。
- 計算平均結果：

  最後，將 K 次測試的評估指標平均，作為模型的最終性能評估。

這樣的過程有助於模型的評估更加穩健，因為模型在不同的資料劃分下進行了多次測試，有效減少了因資料集劃分不同而造成的隨機誤差，同時也可以當作參數挑選器（從交叉驗證中找到合適的超參數）。

異常值

實務上，要判斷一個新的觀察值是否屬於已知樣本的分析，例如判斷信用卡交易是否異常，一般稱為異常偵測（anomaly detection），有時候也會異常值為雜訊（noise），機器學習對這一類的訓練樣本特別敏感，容易造成模型擬合後的效能低落。主要偵測方式分成兩種：

- 離群值（outlier）偵測：

  找出在訓練資料集當中，與其它樣本有相當大差異的觀察值，通常是非監督式（unsupervised）的作法。
- 新奇值（novelty）偵測：

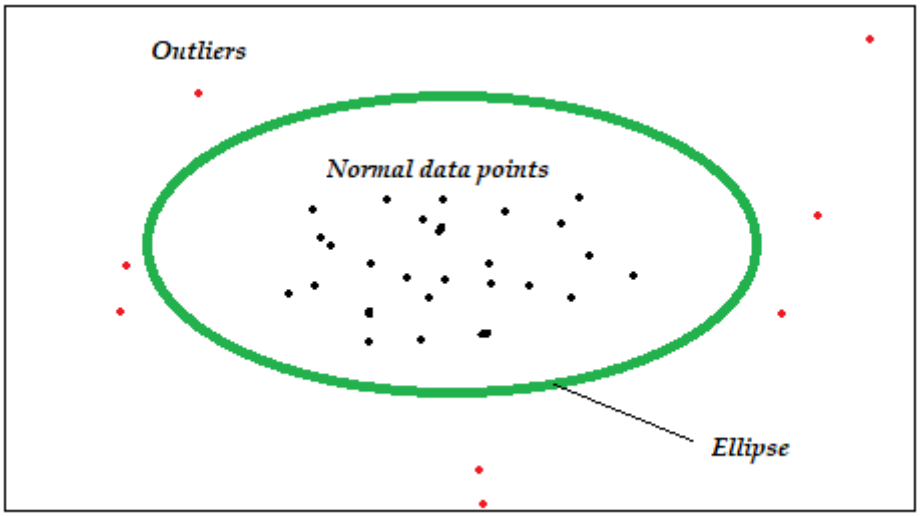
  決定一筆新的觀察值是否為離群值，一般為半監督式（semi-supervised）作法（同時使有、無標籤的資料）。

偵測離群值

以下是一些常見的方法，包括橢圓包絡法（Elliptic Envelope）、四分位數區間（IQR method）、孤立森林（Isolation Forest） 和 局部離群因子（Local Outlier Factor）。這些方法各有特點和適用情境，以下為詳細的說明：

1. 橢圓包絡法（Elliptic Envelope）

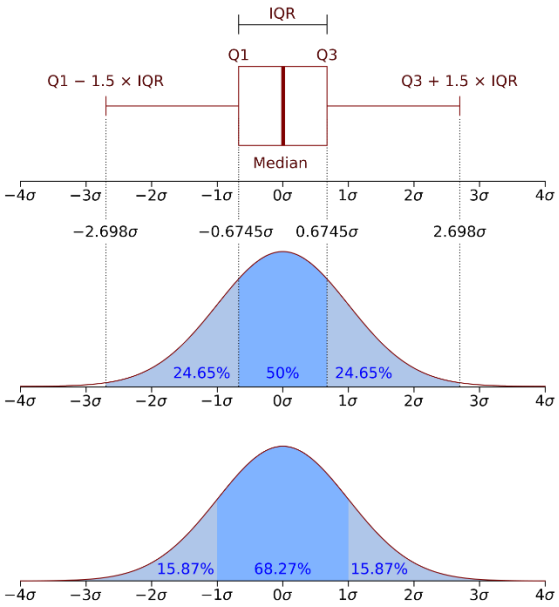
- 橢圓包絡法基於高斯分布來偵測離群值。會先假設「正常資料」大多集中在某個中心附近，而且分布形狀大致像橢圓；離這個橢圓範圍太遠的資料，就可能是離群值。
- 使用時機：適合資料接近高斯分布的情況，例如金融、物理領域的資料，但不適合高度偏態或非高斯分布的資料。
- 優點：
  - 對高斯分布資料有較高的準確性。
  - 可以偵測出在多變量空間中明顯的離群值。
- 缺點：
  - 假設資料必須為高斯分布，對非高斯分布效果較差。
  - 在高維度空間的表現不佳，且需要大量資料來擬合高維橢圓。



圖：在橢圓範圍外，就有可能是離群值。取自 https://medium.com/data-science/machine-learning-for-anomaly-detection-elliptic-envelope-2c90528df0a6

2. 四分位數區間法（Interquartile Range, IQR）

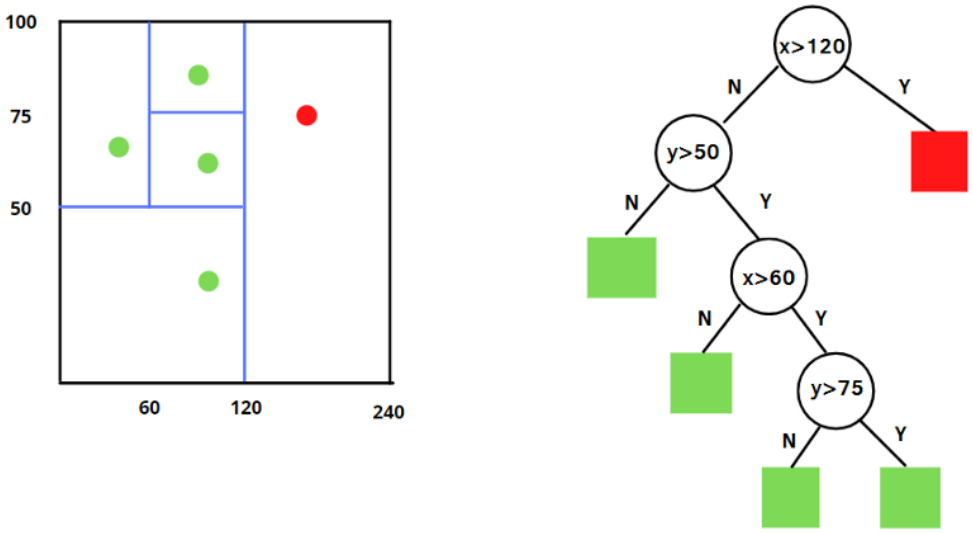
- IQR 方法使用資料的四分位數來定義離群值。通常定義小於 第一四分位數 - 1.5 倍 IQR 或大於 第三四分位數 + 1.5 倍 IQR 的資料點為離群值。
- 使用時機：適合簡單資料集且沒有多變量需求的情境，特別適合一維的資料分布。
- 優點：
  - 簡單易用且無需對資料分布做出假設。
  - 對於非高斯分布或長尾分布的資料效果良好。
- 缺點：
  - 不適合多維資料。
  - 對於擁有極端長尾的分布效果不佳，可能將正確的資料判為離群值。



圖：在 IQR 法中，通常會將 Q1 - 1.5 × IQR 設為下界，將 Q3 + 1.5 × IQR 設為上界。若某筆資料小於下界或大於上界，就會被視為可能的離群值。取自 https://zh.wikipedia.org/zh-tw/%E5%9B%9B%E5%88%86%E4%BD%8D%E8%B7%9D

3. 孤立森林（Isolation Forest）

- 孤立森林是一種基於隨機森林的無監督學習方法。它通過隨機選擇資料分割條件，將資料劃分為不同區域，並根據某些資料點容易被「隔離」的程度來判定其是否為離群值。越容易隔離的點（即只需少量切割次數），越可能是離群值。
- 使用時機：適用於大規模資料集，且資料分布不明的情境，如金融欺詐檢測或網路入侵檢測。
- 優點：
  - 計算效率高，適合大型資料集。
  - 無需假設資料分布，且在高維度空間下也表現良好。
- 缺點：
  - 對於資料分布非常稀疏的資料效果有限。
  - 對於小型資料集可能不準確，因為隨機性可能導致不穩定性。



圖：孤立森林採用二元樹結構對資料進行區分，樣本選取、特徵選取、分支點選取都採用隨機化的方式。如果某個樣本是異常值，可能需要很少次數就可以區分出來。取自 https://zhuanlan.zhihu.com/p/492469453

4. 局部離群因子（Local Outlier Factor, LOF）

- 局部離群因子是一種基於密度的方法，它的核心問題不是問「這個點離所有資料遠不遠？」，而是找出局部空間中的離群值。
- LOF 通過計算每個點的密度及其鄰近點的密度，來評估每個樣本點是否與其鄰近點顯著不同。如果某個樣本的密度明顯低於周圍鄰居，就會被視為離群值（這個點附近的密度，是否明顯比它的鄰居還要低？相對於它附近的鄰居，是否顯得異常？）。
- 你要找的是「局部異常」，不是「全域異常」。
- 使用時機：適合需要發現群內異常或多群異常的情境，如客戶消費行為分群、學生學習表現分群、不同類型商品的銷售特徵、不同族群的健康檢查資料等。
- 演算法核心：
  - 找出每筆資料的 k 個最近鄰居
  - 根據鄰居距離估計每筆資料的局部密度
  - 比較該資料點與鄰居的局部密度
  - 如果該資料點比鄰居稀疏很多，就判定為離群值
- 假設：
  - 有一個點 A，它附近的鄰居都很密集
    - A 的密度 = 10，鄰居平均密度 = 11，那麼 LOF(A) = 11 / 10 = 1.1，這代表 A 和鄰居差不多，通常不是離群值。
  - 另一個點 B

- ◆ B 的密度 = 2，鄰居平均密度 = 10，LOF(B) = 10 / 2 = 5，這代表 B 比它的鄰居稀疏很多，所以 B 很可能是離群值。

- LOF 是用「自己附近有多稀疏」和「鄰居附近有多密集」之間的差距，來判斷一筆資料是否為離群值。
- 優點：
  - ■ 能偵測出局部異常點，對於局部密度變化的資料效果好。
  - ■ 不依賴整體資料分布，適合發現群體內的異常點。
- 缺點：
  - ■ 計算過於密集，尤其是在高維度資料下。
  - ■ 需要選定鄰居數參數，這對於不同資料集的效果影響很大，且需要適當調整。

LOF 分數	**解讀**
約等於 1	和鄰居密度差不多，通常是正常值
大於 1	自己比鄰居更稀疏，可能是離群值
遠大於 1	明顯比鄰居稀疏，很可能是離群值
小於 1	自己比鄰居更密集，通常不是離群值

表：局部離群因子（LOF）分數與離群程度判讀

但是在 scikit-learn 裡面，要特別注意：它提供的屬性是：
`clf.negative_outlier_factor_`

這是負的 LOF 分數，也就是 opposite LOF score。官方文件說，negative_outlier_factor_ 越低，代表越異常；正常值通常接近 -1，離群值會有更低的數值。所以在 scikit-learn 裡：

- negative_outlier_factor_ 接近 -1：正常
- negative_outlier_factor_ 更小，例如 -2、-3：比較異常

總結

- Elliptic Envelope：
  適合高斯分布的多變量資料，但在高維度下效率低。
- IQR Method：
  適合一維資料，無需分布假設，但無法處理多變量資料。
- Isolation Forest：
  適合大規模、高維度且無分布假設的資料集，對小資料效果不佳。
- Local Outlier Factor：

適合多變量且密度變化的資料，但在高維度下運算成本高，且需要選定合適的參數。

處理離群值

1. 直接刪除檢測到的離群值

- 此方法直接移除資料集中被識別為離群值的資料點。這種方法適合當資料集中離群值非常少，且這些離群值可能對模型性能造成負面影響的情況。
- 使用時機：
  - 離群值數量極少，刪除後對資料樣本總體結構影響較小。
  - 離群值顯著偏離正常資料範圍，且屬於顯然的異常點（例如儀器測量錯誤）。
  - 適用於不要求保留每筆資料樣本的場景，例如探索性資料分析或前期清理過程。
  - 可以使用 Isolation Forest 或 Local Outlier Factor 來檢測離群值，檢測後再刪除。

2. 新增一個特徵用以標示離群值

- 此方法在資料中增加一個新特徵，該特徵標示資料是否為離群值。例如，可使用二元變數標示：1 表示離群值，0 表示非離群值。這種方法保留了所有資料，同時向模型提供離群值的資訊，有助於某些模型利用離群值進行更好的預測。
- 使用時機：
  - 離群值具有潛在資訊，不應被刪除。例如，客戶信用風險中，高消費可能是潛在風險指標。
  - 模型可以有效利用該特徵，以增強預測能力。
  - 離群值標示能夠提升模型解釋性或提供額外的分析維度。
- 使用離群值檢測算法（如 Isolation Forest），並在檢測結果基礎上新增標示特徵。

3. 轉換特徵值來降低離群值影響

- 當離群值集中於某個特定的特徵上時，可以使用資料轉換技術降低其影響，例如對數變換、平方根變換或分位數截斷。這種方法能減少極端值對整體模型的影響，而不必刪除離群值。
- 使用時機：
  - 離群值主要存在於某一特徵上，且屬於極端偏態的情況。
  - 離群值可能包含有效資訊，不希望簡單刪除或標示。
  - 常用於處理資料分佈不均勻或極端偏態資料，如收入、交易額等。

- 可使用 PowerTransformer 或 QuantileTransformer 來進行資料轉換，使資料分佈更趨於常態。

偵測新奇值（novelty detection）

用來偵測資料中不符合訓練資料分佈的異常樣本的方法。主要有兩種方法：單類別支持向量機(One-Class SVM) 和 局部離群因子 (Local Outlier Factor, LOF)。

1. 單類別支持向量機(One-Class SVM)

- 它通常只使用「正常資料」進行訓練，模型會學習正常資料在特徵空間中的分佈邊界。當新的資料進來時，如果落在正常邊界之外，就會被判斷為異常或新奇值。One-Class SVM 使用支持向量機的原理，將大部分樣本隔離在邊界內，並用核函數（如高斯核）來建立非線性的邊界。當進行新奇值偵測時，新樣本若落在邊界外，就會被標記為異常值。
- 使用時機：
  - 想判斷未來新進資料是否異常：
    當我們希望偵測未出現在訓練集中的新奇樣本時。
  - 訓練樣本都是正常樣本：
    假設訓練資料是純正常樣本，這在某些情況下是理想的，比如製造業中的產品檢測，只有符合標準的產品才能進入生產線。
- 優點與局限：
  - 優點：能夠有效偵測新奇樣本，尤其在訓練資料單純為正常樣本時效果良好。
  - 局限：One-Class SVM 計算成本較高，尤其在高維資料集上，且對於大規模資料表現不佳。

2. 局部離群因子 (Local Outlier Factor, LOF)

- 基於局部密度的方法，用於判斷樣本是否偏離周圍的樣本分佈。該方法基於 k 近鄰演算法，計算每個樣本的「離群因子」，該因子表示該樣本的局部密度與其鄰近點的密度差異。若一個樣本的密度顯著低於周圍鄰居的密度，則該樣本被標記為離群值。LOF 的評估方式是相對性的，即根據周圍的樣本來決定樣本的異常程度。
- 使用時機：
  - 訓練資料大多是「正常資料」：
    新奇值偵測的基本假設是訓練資料沒有被異常值污染，或至少大部分都是正常資料。scikit-learn 官方文件也把 novelty

detection 定義為：訓練資料未被 outliers 污染，目標是判斷新的 observation 是否異常。

- 關心的是「局部異常」，不是全域異常：

  LOF 適合用於 novelty detection 的時機，取決於「新資料是否可以透過局部密度差異來判斷異常」，也就是說，LOF 不是單純問「這筆新資料離正常資料遠不遠？」而是問「這筆新資料所在的位置，和它附近的正常鄰居相比，密度是否異常地低？」

  - 當你設定 `novelty=True` 的時候：

    此時模型會先根據 X_train 學習正常資料的局部密度結構，之後再對新的資料做判斷。

- 優點與局限：
  - 優點：適合正常資料有不同族群的情境。如果正常資料本身有多種合理型態，例如一般使用者、重度使用者、企業使用者、學生使用者等，這些族群的行為密度可能不同。LOF 可以讓新資料和它附近相似的正常樣本比較，因此較適合處理「多群集、非均勻密度」的正常資料。
  - 局限：
    - 必須設定 novelty=True
    - 只能對新的 unseen data 使用 predict()
    - 對 n_neighbors 很敏感
    - 不適合非常高維的資料

比較面向	**LOF（Local Outlier Factor）**	**One-Class SVM**
中文名稱	局部離群因子	單類別支持向量機
主要概念	比較樣本與鄰近樣本之間的**局部密度差異**	學習正常資料的**決策邊界**
判斷依據	如果某筆資料的局部密度明顯低於鄰居，則可能是異常	如果新資料落在正常邊界之外，則可能是異常
偵測角度	偏向「局部異常」	偏向「整體邊界異常」
是否可用於 novelty detection	可以，但需設定 novelty=True	可以，常用於 novelty detection
是否需要異常樣本	不需要	不需要
適合的資料型態	正常資料有多個群集、不同區域密度不一致	正常資料有較明確的分布邊界
對局部結構的掌握	較強	較弱
對全域邊界的掌握	較弱	較強

主要參數	n_neighbors、contamination	kernel、nu、gamma
對特徵尺度是否敏感	敏感，通常需標準化	敏感，通常需標準化
高維資料表現	可能受距離失效影響	也可能受影響，但 kernel 可提供非線性邊界
訓練資料若混有異常值	容易影響局部密度估計	也會影響決策邊界
可解釋性	較直觀，可解釋為局部密度異常	較抽象，尤其使用 RBF kernel 時較難解釋
適合情境	異常點在局部區域中密度不合理	異常點落在正常資料整體邊界之外

表：LOF 關心「局部是否不合理」，One-Class SVM 關心「整體是否超出正常邊界」。

選取重要特徵

在機器學習中，「選取重要特徵」(Feature Selection) 和「特徵縮減」(Feature Reduction) 是非常重要的步驟。這些技術的主要目的是減少模型的複雜度、提升模型的效能，以及防止過擬合（Overfitting）的發生。原因如下：

- 減少模型複雜度：

  過多的特徵可能會增加模型的複雜度，導致計算成本過高，模型的訓練和預測速度變慢。
- 避免過擬合：

  當特徵數量遠超過訓練樣本數時，模型可能會過度擬合訓練資料，降低對新資料的泛化能力。
- 提升效能：

  移除冗餘或無關的特徵可以使模型專注於與目標變量最相關的特徵，提升預測精度。
- 可解釋性：

  去除無意義的特徵可以讓模型更簡單、更容易解釋，有助於了解哪些因素真正影響目標變量。

依統計性質過濾

- 透過變異性來篩選特徵：
  - 是一種根據特徵本身統計性質進行篩選的方法。它的核心想法是：如

果某個特徵在大多數資料中幾乎都一樣，代表它能提供的區分資訊很少（變異數過低），因此可以考慮移除。

- 變異數越大，代表該特徵的數值分散程度越高；變異數越小，代表該特徵的值越集中。如果某個特徵的變異數為 0，表示所有資料在這個特徵上的值都完全相同，這種特徵通常無法幫助模型區分不同樣本。

- 透過相關性來篩選特徵：
  - 是一種根據特徵之間，或特徵與目標變數之間的相關程度，來進行特徵篩選的方法。其核心概念是：如果某個特徵與目標變數高度相關，代表它可能對預測有幫助；相反地，如果兩個特徵彼此高度相關，代表它們可能提供了重複資訊，可以考慮移除其中一個，以降低特徵冗餘。皮爾森、斯皮爾曼和肯德爾相關係數，這三種相關係數常用來衡量兩個變量之間的線性或單調關係。
  - 皮爾森相關係數 (Pearson Correlation Coefficient)
    - 用途：用來衡量兩個連續型變數之間的線性相關性。
    - 原理：觀察兩個變數是否會一起變大、一起變小，或呈現相反方向變化。
    - $$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$
    - 其中
      - x_i：第 i 筆資料在特徵 X 上的值
      - y_i：第 i 筆資料在特徵 Y 上的值
      - $\hat{x}$：特徵 X 的平均值
      - $\hat{y}$：特徵 Y 的平均值
      - n：資料筆數
    - Pearson 相關係數的取值範圍為 [-1,1]
      - $r = 1$　完全正線性相關
      - $r = -1$ 完全負線性相關
      - $r = 0$　 沒有明顯線性相關
    - Pearson 相關係數主要衡量線性關係。如果兩個變數存在非線性關係，即使它們有明顯關聯，Pearson 相關係數也可能接近 0。
    - 例如
      - 攝氏溫度與華氏溫度之間是完全線性關係，因為華氏溫度可以由攝氏溫度透過固定公式轉換而來，兩者的 Pearson 相關係數會非常接近 1
      - 在房價預測中，房屋坪數與房價可能具有正向線性關係。當

坪數增加時，房價也傾向增加，且兩者關係若大致接近直線，則適合使用 Pearson 相關係數來衡量其線性相關程度。

- 斯皮爾曼相關係數 (Spearman's Rank Correlation Coefficient)
  - 用途：衡量兩個變數之間的單調關係（monotonic relationship），無論是否為線性。
  - 所謂單調關係是指：
    - X 越大，Y 整體上也越大
    - X 越大，Y 整體上越小
    - 不要求兩者一定是直線關係
    - 原理：不是直接比較原始數值，而是先將資料轉換成排名（rank），再計算排名之間的相關性，從而捕捉到變數之間的單調相關。
    - 例如原始資料為分數「90, 70, 80」，轉換成排名「1, 3, 2」，Spearman 關心的是「兩個變數的排序是否一致」。
  - $$\rho = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)}$$
  - 其中
    - p：Spearman 等級相關係數
    - d_i：第 i 筆資料在兩個變數排名上的差距
    - n：資料筆數
  - 若資料中有相同排名，通常會先計算平均排名，再對排名後的資料計算 Pearson 相關係數。
  - Spearman 相關係數的取值範圍為 [-1,1]
    - $p = 1$　兩個變數的排名完全一致
    - $p = -1$　兩個變數的排名完全相反
    - $p = 0$　沒有明顯單調關係
  - 例如：收入與消費能力可能不是完全線性關係，但整體上收入越高，消費能力越高，這時 Spearman 可能比 Pearson 更合適。

學生	讀書時間	讀書時間排名	考試成績	考試成績排名
A	1 小時	1	55 分	1
B	2 小時	2	60 分	2
C	3 小時	3	70 分	3

D	4 小時	4	85 分	4
E	5 小時	5	95 分	5

表：兩個排名完全一致，所以 Spearman 相關係數會非常高，甚至等於 1

學生	讀書時間	讀書時間排名	考試成績	考試成績排名
A	1 小時	1	55 分	1
B	2 小時	2	80 分	3
C	3 小時	3	65 分	2
D	4 小時	4	90 分	5
E	5 小時	5	85 分	4

表：這時候排名大致同方向，但不是完全一致，Spearman 仍然可能是正相關，但不會等於 1

- 肯德爾相關係數 (Kendall's Tau Correlation Coefficient)
  - 用途：用來衡量兩個變數的排序一致程度，適用於序列資料。
  - 原理：基於觀測對（pair）之間的一致性計算，考慮了每一對資料之間是否一致，即若其中一個變數的值增加，另一個變數的值也相應增加或減少。
  - 假設有兩筆資料 i 和 j
    - 如果在特徵 X 中，$x_i > x_j$，且在特徵 Y 中，$y_i > y_j$，代表這一組資料的排序是一致的，稱為 concordant pair（一致配對）
    - 如果在特徵 X 中，$x_i > x_j$，但在特徵 Y 中，$y_i < y_j$，代表這一組資料的排序是不一致的，稱為 discordant pair（不一致配對）
  - $$\tau = \frac{C - D}{C + D}$$
  - 其中
    - C：一致配對數量
    - D：不一致配對數量
    - $\mathcal{T}$：Kendall 等級相關係數
  - 若資料中沒有相同排名，也可以寫成：
  - $$\tau = \frac{C - D}{\frac{1}{2}n(n-1)}$$

- 其中 $\mathcal{T} = \frac{C-D}{\frac{1}{2}n(n-1)}$ 是所有可能的資料配對數量。
- Kendall 相關係數的取值範圍為 [-1,1]
  - $\mathcal{T} = 1$　所有配對排序都一致
  - $\mathcal{T} = -1$　所有配對排序都相反
  - $\mathcal{T} = 0$　排序沒有明顯一致性
- Kendall 與 Spearman 都是根據排名計算，但 Kendall 更強調「兩兩配對的排序一致性」，因此在樣本數較小或需要解釋排序一致程度時，Kendall 會比較直觀。
- 例如
  - 兩位老師對同一批作文進行排名。如果老師甲認為作文 A 比作文 B 好，老師乙也認為作文 A 比作文 B 好，這就是一組排序一致的配對；如果兩位老師的判斷相反，則是不一致配對。Kendall 相關係數可以用來衡量兩位老師的排名結果是否接近。
  - 在搜尋系統中，兩個排序模型都針對同一個查詢回傳文件排名。如果兩個模型都認為文件 A 比文件 B 更相關，文件 C 比文件 D 更相關，代表它們在這些文件配對上的排序是一致的。
    Kendall 相關係數就是透過比較這些「兩兩文件配對」的排序是否一致，來衡量兩個排序結果的相似程度。

學生	讀書時間 X	考試成績 Y
A	1 小時	60 分
B	2 小時	80 分
C	3 小時	70 分
D	4 小時	90 分

表：Kendall's Tau 會把所有學生「兩兩成對」比較

Pair	讀書時間比較	成績比較	判斷
A-B	B 比 A 多	B 比 A 高	一致
A-C	C 比 A 多	C 比 A 高	一致
A-D	D 比 A 多	D 比 A 高	一致
B-C	C 比 B 多	C 比 B 低	不一致
B-D	D 比 B 多	D 比 B 高	一致
C-D	D 比 C 多	D 比 C 高	一致

表：讀書時間和考試成績有明顯的正向排序關係，但不是完全一致

在特徵選取中，不同的特徵和目標項型態適合不同的相關性檢驗方法。以下介紹卡方檢驗、ANOVA 的 F-value 及 Mutual Information 的使用時機。

變數類型	衡量相關性的做法
數值型 vs 數值型	Pearson 相關係數
類別型 vs 類別型（名目尺度）	卡方檢定（Chi-square test）或費雪精確檢定（Fisher's exact test）
類別型 vs 類別型（順序尺度）	Spearman 等級相關係數（Spearman's rank correlation coefficient）或 Kendall 等級相關係數（Kendall's tau correlation coefficient）
數值型 vs 類別型	獨立樣本 t 檢定（t-test）或 ANOVA（變異數分析）F-value

表：特徵與目標項（target，也可稱為 label）之間的相關性

1. 卡方檢驗 (Chi-Square Test)

使用時機：當特徵與目標項都是類別型資料時，適合使用卡方檢驗來衡量兩者的相關性。此檢驗的目的是評估觀察值與期望值之間的差異，判斷特徵和目標是否有顯著關聯。

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

其中

- χ^2：卡方檢驗統計量
- O：觀察次數（Observed frequency）
- E：期望次數（Expected frequency）
  - $E = \frac{該列總和 \times 該欄總和}{總樣本數}$
  - 例如，若要檢驗「性別」與「是否購買」是否有關，會先建立列聯表，然後比較每一格的觀察次數與期望次數。
- 若列聯表有 r 列、c 欄，則自由度為 df：
  - $df = (r-1)(c-2)$

解釋：卡方檢驗的結果值越大，代表特徵與目標變量之間的關聯越強（如果某個特徵和目標變數有顯著關聯，通常代表它是比較值得保留的候選特徵）。根據卡方分佈的臨界值可以判斷是否具有顯著性，通常搭配 p 值來確定。

2. ANOVA 的 F-value

使用時機：當數值型特徵與類別型目標項之間，需要衡量特徵對目標的影響程度時，適合使用單因子變異數分析（ANOVA）。ANOVA 的 F-value 用於判斷數值型特徵是否能有效區分不同的類別型目標。

$$F = \frac{MS_{between}}{MS_{within}} = \frac{SS_{between}/(k-1)}{SS_{within}/(N-k)}$$

其中

- F：ANOVA 的 F-value
- $MS_{between}$：組間均方（Mean Square Between）
- MS_{within}：組內均方（Mean Square Between）
- $SS_{between}$：組間平方和

$$SS_{between} = \sum_{j=1}^{k} n_j(\bar{x}_j - \bar{x})^2$$

  - n_j：第 j 組的樣本數
  - $\bar{x}_i$：第 j 組的平均值
  - $\bar{x}$：全部資料的總平均
- SS_{within}：組內平方和

$$SS_{within} = \sum_{j=1}^{k}\sum_{i=1}^{n_j}(x_{ij} - \bar{x}_j)^2$$

  - x_{ij}：第 j 組中第 i 筆資料
  - $\bar{x}_j$：第 j 組的平均值
- k：群組數量
- N：總樣本數

解釋：F 值越大，代表分組間的差異相對於組內變異越大，表示該特徵對目標項有較大的區分力（這個特徵越可能有助於區分目標類別），F 值高不代表這個特徵一定能讓模型變好，只代表它單獨看起來和目標類別有關。通常會搭配 F 值和 p 值來判斷是否具有顯著性。

3. Mutual Information

使用時機：當特徵與目標項皆為數值型時，可以使用 Mutual Information (相互資訊) 來衡量兩者之間的依賴關係。Mutual Information 測量了變量之間的非線性關聯度，能捕捉線性和非線性關係。

- 離散型變數的 Mutual Information

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \left(\frac{p(x,y)}{p(x)p(y)} \right)$$

- $I(X;Y)$：X 與 Y 的 Mutual Information
- $p(x,y)$ ：X = x 且 Y = y 同時發生的聯合機率
- $p(x)$：X = x 的邊際機率
- $p(y)$：Y = y 的邊際機率

- 連續型變數的 Mutual Information

$$I(X;Y) = \iint p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \, dx \, dy$$

- 若 X 與 Y 是連續型變數，則加總會改成積分
- 其中 $p(x,y)$、$p(x)$、$p(y)$ 是機率密度函數

- 可以用 Entropy（熵）來表示

$$I(X;Y) = H(Y) - H(Y|X)$$

- 代表：原本對 Y 的不確定性 – 知道 X 之後對 Y 剩下的不確定性
- 也可以寫成

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

解釋：某個特徵 X 對目標變數 Y 提供多少資訊。若某個特徵與目標變數的 Mutual Information 較高，代表該特徵可能對預測有幫助；若 Mutual Information 接近 0，代表該特徵與目標變數之間可能關聯很弱。Mutual Information 能夠衡量非線性關聯，是較為靈活的度量方式。

情況	Mutual Information
(X) 與 (Y) 完全獨立	I(X;Y)=0
(X) 對 (Y) 有預測資訊	I(X;Y)>0
(X) 與 (Y) 關聯越強	MI 越大

表：Mutual Information 的直觀解釋

特徵 X	目標 y	常見任務	sklearn 用法	對應概念
離散型	離散型	分類	mutual_info_classif(..., discrete_features=True)	離散型 MI 估計
連續型	離散型	分類	mutual_info_classif(..., discrete_features=False)	連續-離散 MI 估計

離散型	連續型	回歸	mutual_info_regression(..., discrete_features=True)	離散-連續 MI 估計
連續型	連續型	回歸	mutual_info_regression(..., discrete_features=False)	連續型 MI 估計

表：決定哪種情形下使用不同的 Mutual Information 函式

嘗試錯誤法挑選特徵

在特徵選擇過程中，使用遞迴特徵汰除 (Recursive Feature Elimination, RFE) 和排列特徵重要性 (Permutation Feature Importance) 都是常用的「嘗試錯誤法」，它們能幫助識別對模型預測影響較大的特徵並減少不必要的特徵。以下是這兩種方法的概念與適合的使用情境：

遞迴特徵汰除 (Recursive Feature Elimination)

- RFE 是一種遞迴的特徵選擇方法。它從一開始包含所有特徵的模型開始，訓練模型並根據特徵的重要性去除最不重要的特徵（例如權重較小的特徵），重複該過程，直到只剩下預定數量的特徵。RFE 通常和支持向量機 (SVM)、線性回歸等模型一起使用，以確保重要性評估的準確性。
- 使用時機：
  - 當資料量較小：

    RFE 適合於特徵數量適中或較少的資料集，因為該方法會重複訓練模型，對於較大資料集的運算資源需求較高。
  - 目標是選擇少數高相關特徵：

    如果我們希望篩選出對模型影響最大的特徵集，並保留少數幾個關鍵特徵以提升模型的可解釋性，RFE 是較好的選擇。
  - 與特定模型結合：

    RFE 會考慮與模型相關的特徵權重，因此適合需要與特定模型結合使用的特徵選擇場景。

排列特徵重要性 (Permutation Feature Importance)

- 排列特徵重要性方法會隨機打亂單個特徵的數值，觀察模型準確性或損失的變化，來量化該特徵對模型的貢獻。打亂特徵後的模型表現若下降較

多，則說明該特徵對模型預測的貢獻較大。此方法不依賴於特定的模型，因此適用於任何 scikit-learn 的模型。

- 使用時機：
  - 適用於大型資料集：

    此方法不需要重新訓練模型，因此在資料集較大或模型訓練時間較長的情況下，它是較為高效的選擇。
  - 適合任何模型：

    因為它不依賴於模型內部的特徵權重或結構，排列特徵重要性適合需要對多種模型進行特徵篩選的場景。
  - 需求模型的整體解釋性：

    如果我們想要了解每個特徵對於模型的整體影響，並非僅僅是為了提高模型的精度，排列特徵重要性能夠提供一個穩健的特徵貢獻評估。

編號	HP	Attack	Legendary
0	45	49	0
1	60	62	0
2	90	100	1
3	100	120	1

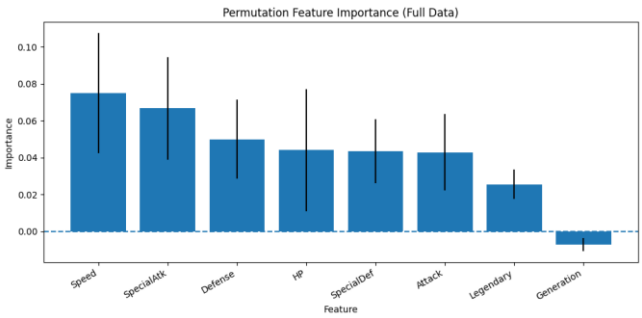
表：原始資料尚未打亂的樣子

編號	HP 被打亂	Attack	Legendary
0	90	49	0
1	45	62	0
2	100	100	1
3	60	120	1

表：打亂 HP 的意思是「只把 HP 這一欄重新排列，其他欄位保持不動」

使用建議

- 小型資料集且對運算資源不敏感：考慮使用 RFE。
- 大型資料集或複雜模型：選擇排列特徵重要性以節省時間和資源。
- 追求解釋性：排列特徵重要性能提供特徵對模型的全局性影響評估。



圖：不切分資料，直接列出特徵重要性

Module 2. 線性迴歸

在 Scikit-learn 中，線性迴歸（Linear Regression）有多種形式，包括簡單線性迴歸、多元線性迴歸、多項式迴歸等。除了基本的線性迴歸模型，還有正則化版本的線性迴歸模型，例如 Ridge 和 Lasso Regression，它們可以幫助控制模型的複雜度。以下是這些模型和它們的評估方式的詳細說明：

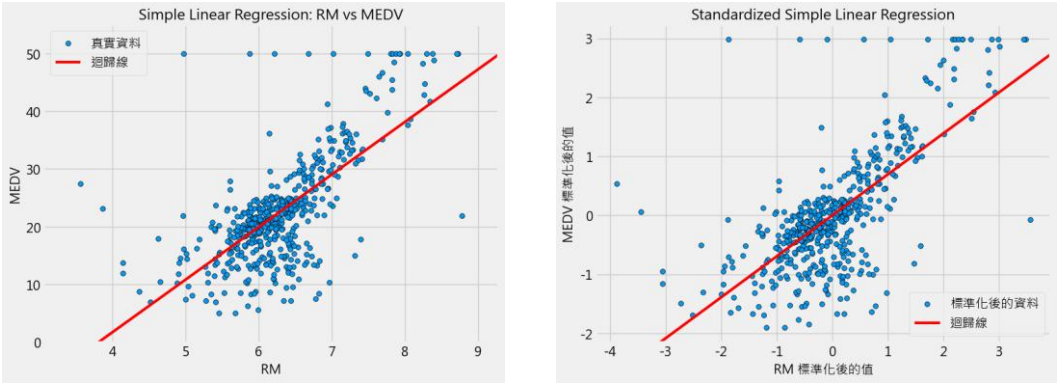
簡單線性迴歸（Simple Linear Regression）

定義：簡單線性迴歸是指單一自變量（只有一個特徵）對應一個目標變量的回歸模型。

$$y = \beta_0 + \beta_1 x + \epsilon$$

- y 是目標變量（應變數）。
- x 是自變量（獨立變數）。
- β_0 是截距，表示當 $x = 0$ 時 y 的預測值。
- β_1 是回歸係數，表示 x 每變動一單位時 y的變動量。
- ϵ 是誤差項，用來表示實際觀測值與預測值之間的差異。

使用場景：適合用於描述兩個變量之間的線性關係。



圖：檢視資料散佈程度與繪製迴歸線

多元線性迴歸（Multiple Linear Regression）

定義：多元線性迴歸擴展了簡單線性迴歸的概念，允許多個自變量對單一目標變量的影響（使用兩個以上的特徵來預測一個連續型目標變數的線性迴歸模型）。

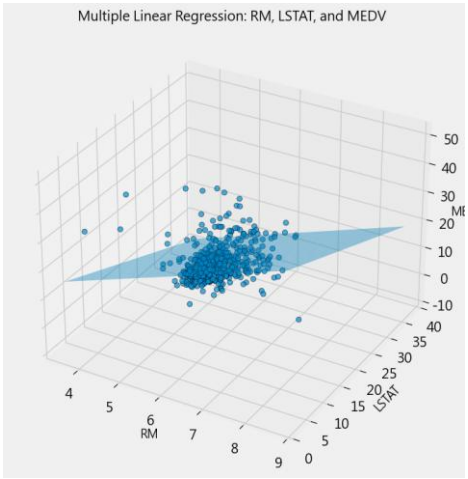
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

- y：目標變量（應變數）。
- $x_1, x_2, \ldots, x_p$：p 個自變量（獨立變數）。
- β_0：截距，表示當 $x_i = 0$ 時 y 的預測值。
- $\beta_1, \beta_2, \ldots, \beta_p$：回歸係數，表示自變量 x_i 每變動一單位時 y 的變動量（在其它變量固定不變的情況下）。
- ϵ：誤差項，用來表示實際觀測值與預測值之間的差異。

使用場景：適合多個變量之間存在線性關係的場景，常用於描述和預測多個因素如何影響目標變量。例如預測房價時，只用 RM 可能不夠，因為房價可能受到：

- 房間數 RM
- 低收入人口比例 LSTAT
- 犯罪率 CRIM
- 師生比例 PTRATIO
- 距離就業中心遠近 DIS
- 等多個因素影響，

這時候就比簡單線性迴歸更適合使用多元線性迴歸。



圖：3 維的多元線性迴歸視覺化

共線性（Collinearity）

共線性是指迴歸模型中的特徵之間存在高度線性關係，例如在房價資料中，有兩個特徵：

- RM：平均房間數
- 住宅面積：房屋坪數

通常房間數越多，房屋面積也越大，如果這兩個特徵高度相關，它們提供的資訊就會部分重複，如果某些特徵之間高度相關，模型會很難判斷，到底是 RM 影響 MEDV，還是另一個和 RM 很像的特徵影響 MEDV？這會造成係數不穩定，例如：

- 係數正負號可能不符合直覺
- 加入或刪除少量資料後，係數變化很大
- 單一特徵的解釋變得不可靠
- 標準誤（Standard Error）變大，進而影響顯著性檢定

如果泛指三個或更多自變數之間存在線性關係，就稱為多重共線性（Multicollinearity），即某個變數可以由其他多個變數的線性組合來近似表示，例如 RM 可以被 AGE、DIS、LSTAT 等其他特徵高度預測出來，使得變數間的獨立性喪失。

變異膨脹因子（Variance Inflation Factor，VIF）

VIF 是用來檢查「某一個特徵是否可以被其他特徵高度解釋」，它的用途是衡量某一個特徵的迴歸係數變異數，因為和其他特徵相關而被放大了多少倍。

$$VIF_j = \frac{1}{1-R_j^2}$$

其中

- VIF_j：第 j 個特徵的 VIF
- R_j^2：把第 j 個特徵當成 y，其它特徵當成 x，做一次迴歸後得到 R^2

VIF 值	解讀
VIF = 1	幾乎沒有共線性
1 < VIF < 5	通常可接受
VIF ≥ 5	可能有共線性，需要注意
VIF ≥ 10	常被視為嚴重共線性

表：VIF 的解讀方式

OLS Regression Results

Dep. Variable:	MEDV	R-squared:	0.639
Model:	OLS	Adj. R-squared:	0.637
Method:	Least Squares	F-statistic:	444.3
Date:	Tue, 05 May 2026	Prob (F-statistic):	7.01e-112
Time:	00:45:38	Log-Likelihood:	-1582.8
No. Observations:	506	AIC:	3172.
Df Residuals:	503	BIC:	3184.
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	-1.3583	3.173	-0.428	0.669	-7.592	4.875
RM	5.0948	0.444	11.463	0.000	4.222	5.968
LSTAT	-0.6424	0.044	-14.689	0.000	-0.728	-0.556

Omnibus:	145.712	Durbin-Watson:	0.834
Prob(Omnibus):	0.000	Jarque-Bera (JB):	457.690
Skew:	1.343	Prob(JB):	4.11e-100
Kurtosis:	6.807	Cond. No.	202.

圖：多元線性迴歸統計摘要表

線性迴歸模型評估指標

在使用迴歸模型後，可以根據以下評估指標來衡量模型的表現：

平均平方誤差（Mean Squared Error, MSE）

MSE 是迴歸模型中常用的評估指標，用來衡量模型的預測值和真實值之間的差距。在迴歸問題中，模型的目標通常是預測一個連續數值，例如房價、溫度、銷售額、薪資等。模型預測出來的值不一定會完全等於真實值，因此我們需要一個指標來衡量模型的誤差大小。

MSE 的核心概念是：先計算每一筆資料的預測誤差，將誤差平方，再取平均。也就是說，MSE 會把每一筆資料的誤差都轉成正數，並且讓較大的誤差受到更明顯的懲罰。MSE 越小，模型的預測準確度越高。

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- n：樣本數。
- y_i：第 i 個樣本的實際值。
- $\hat{y}_i$：第 i 個樣本的預測值。

平均平方根誤差（Root Mean Squared Error, RMSE）

MSE 會把誤差平方，因此單位會變成原始目標變數單位的平方。RMSE 對 MSE 開根號後，單位會回到和目標變數相同，因此比 MSE 更容易解釋。例如，如果目標變數是房價，那麼 MSE 的單位是房價²，RMSE 的單位是房價。

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- n：樣本數。
- y_i：第 i 個樣本的實際值。
- $\hat{y}_i$：第 i 個樣本的預測值。

平均絕對誤差（Mean Absolute Error, MAE）

計算預測值和實際值之間的絕對差異的平均值，用來衡量模型的預測值與真實值之間平均相差多少。它的核心概念是先計算每一筆資料的預測誤差，再取絕對值，最後計算平均，也就是說，MAE 會計算模型每一次預測「平均錯多少」。例如在房價預測中，如果 MAE = 3.5，可以解釋為：模型預測的房價中位數，平均大約和真實值相差 3.5 個單位。

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- n：樣本數。
- y_i：第 i 個樣本的實際值。
- $\hat{y}_i$：第 i 個樣本的預測值。

MAE 的值越小表示預測結果越準確。

決定係數（R-squared, R^2）

它用來衡量模型可以解釋目標變數變異的比例。R² 通常越接近 1 越好，但也可能小於 0。

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- y_i：第 i 個樣本的實際值。
- $\hat{y}_i$：第 i 個樣本的預測值。
- $\bar{y}$：實際值的平均值。
- $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$：模型的總誤差平方和（Residual Sum of Squares, RSS）。
- $\sum_{i=1}^{n}(y_i - \bar{y})^2$：實際值的總變異平方和（Total Sum of Squares, RSS）。

R^2 數值	**解釋**
$R^2 = 1$	預測完全正確
R^2 接近 1	模型解釋力很好
$R^2 = 0$	模型表現大約等於只預測平均值
$R^2 < 0$	模型表現比只預測平均值還差

表：R^2 可能出現的情況

Adjusted R^2

Adjusted R^2 的目的，是在 R^2 的基礎上加入「特徵數量懲罰」。原因是只要加入更多特徵，訓練資料上的 R^2 通常會上升，或至少不太會下降，但這不代表新加入的特徵真的有幫助。所以 Adjusted R^2 會考慮特徵數量，如果加入的新特徵沒有帶來足夠的解釋力，Adjusted R^2 反而可能下降。

$$Adjusted\ R^2 = 1 - \frac{(1-R^2)(n-1)}{n-k-1}$$

其中

- R^2：決定係數
- n：資料筆數
- k：特徵數量（不包含截距項）
- $n-k-1$：自由度

殘差分析

在機器學習和統計學中，殘差分析是一種評估模型擬合（fitting）程度的工具。它可以幫助檢查回歸模型是否滿足基本假設，從而驗證模型的合理性和預測的可靠性。殘差分析通常涉及三個主要方面：常態性、獨立性 和 變異數同質性。

常態性

常態性檢驗的目的是檢查殘差是否服從常態分佈。回歸模型的殘差通常假設應該是常態分佈的，這有助於確保模型的穩定性和預測精度。如果殘差偏離常態分佈，可能會影響模型的解釋力。

- Shapiro-Wilk Test (SW test)：

  SW 檢定是一種檢查樣本資料是否符合常態分佈的統計檢定。其原假設是「樣本資料來自常態分佈」，p 值越小越拒絕原假設，表示資料偏離常態。一般在樣本數小於或等於 2000 時使用。

- Kolmogorov-Smirnov Test (KS test)：
  - KS 檢定也是用於檢驗資料是否符合特定分佈（例如常態分佈）的方法。KS 檢定通過比較樣本的累積分佈函數（CDF）與理論分佈的 CDF 之間的差異，來測量分佈的一致性（比較樣本分布與指定理論分布）。一般在樣本數超過 2000 時使用。
  - 補充：Lilliefors test

    如果你要檢查「資料是否服從常態分布」，但平均值和變異數是從樣本估計的，那麼比起普通 KS test，更常見的做法是使用 Lilliefors test。

- D'Agostino-Pearson Omnibus 檢定：

  這是一種綜合性的檢定方法，用於同時檢查資料的偏度（Skewness）和峰度（Kurtosis），從而確定資料是否符合常態分佈（因此可以檢查資料是否因為不對稱或尾端過厚而偏離常態分布）

- Jarque-Bera Test (JB test)：

  JB 檢定也使用偏度和峰度來測試常態性。JB 檢定的原假設為「資料來自常態分佈」。當偏度和峰度偏離常態分佈時，該檢定會拒絕

原假設。

補充：QQ 圖

QQ 圖（Quantile-Quantile Plot）是一種視覺化工具，用來比較樣本資料的分位數與理論分布的分位數。在殘差分析中，QQ 圖常用來檢查迴歸模型的殘差是否接近常態分布。如果殘差接近常態分布，QQ 圖中的點會大致沿著對角線排列。若點明顯偏離對角線，尤其是在圖的兩端偏離較嚴重，表示殘差可能存在偏態、厚尾或離群值，模型的常態性假設可能不太理想。

QQ 圖屬於視覺化檢查方法，適合搭配 Shapiro-Wilk 檢定、D'Agostino-Pearson omnibus 檢定或 Jarque-Bera 檢定一起使用。

獨立性

獨立性假設是指殘差之間應該是相互獨立的，特別是在時序資料中，殘差的自相關可能會導致模型不穩定。因此，獨立性檢驗可幫助確保模型的殘差不隨時間或其他變數而變化。如果資料只是隨機排列，那麼相鄰兩筆資料沒有實際意義，自相關就不太值得解釋；如果模型的誤差不是隨機的，代表模型可能漏掉某些規律，因此部分統計推論與預測表現可能不可靠。

主題	說明
基本假設	在迴歸模型中，我們通常假設「誤差項」彼此獨立。由於誤差項通常無法直接觀察，因此在實務上，我們會透過「殘差」來檢查這個假設是否合理。
殘差的定義	殘差是模型真實值與預測值之間的差異。也就是：殘差 = 真實值 - 預測值。
殘差獨立的意義	如果殘差之間彼此獨立，代表模型的誤差大致呈現隨機分布，沒有明顯的時間性、順序性、空間性或系統性模式。
自相關的定義	如果殘差之間存在相關性，稱為「自相關」(Autocorrelation)。這通常代表模型仍有某些資訊沒有捕捉到，例如時間趨勢、空間群聚、遺漏變數或模型形式錯誤。 一階自相關就是：第 i 筆的殘差，會不會和第 i-1 筆的殘差有關，也就是看「相鄰兩筆資料的殘差」是不是有連動關係。自相關的「階」其實「往前隔幾筆」。
為什麼要檢查殘	在進行迴歸分析時，除了觀察模型的 R^2、MAE、RMSE 等指標之外，也

差獨立性	應該檢查殘差是否具有獨立性。因為殘差自相關可能使標準誤、p-value、信賴區間與假設檢定結果變得不可靠。

表：殘差是否存在自相關

原因	說明	範例
遺漏關鍵變數（Omitted Variable）	如果模型中缺少重要的解釋變數，而這些變數本身具有時間、空間或順序上的結構，殘差可能會反映這些未被模型捕捉到的因素，進而產生自相關。	在預測汽車銷售量時，如果模型只考慮廣告支出，卻忽略經濟景氣，殘差可能會呈現時間上的相關性。原因是經濟景氣往往具有連續性，例如經濟繁榮或衰退可能會持續一段時間。若模型沒有納入這個因素，殘差就可能反映出這種被遺漏的時間結構。
時間序列資料中的時間依賴性	在時間序列資料中，觀測值往往會和前一期或前幾期的觀測值相關。如果模型沒有處理這種時間依賴性，殘差之間可能會出現自相關。	在預測每日氣溫時，今天的氣溫通常和昨天的氣溫有關。如果模型只使用月份作為解釋變數，而沒有考慮前一天的氣溫，殘差可能會顯示時間上的自相關。這代表模型沒有完全捕捉氣溫的連續變化特性。
模型形式錯誤（Model Misspecification）	如果真實關係是非線性的，但模型使用線性迴歸來描述，殘差可能會出現系統性模式。若資料本身具有時間順序、空間順序或其他排列順序，這種系統性模式也可能使相鄰殘差出現相關。	若實際上收入與工作經驗之間是拋物線關係，但模型使用線性迴歸，殘差圖可能會呈現曲線形狀。因此，殘差自相關有時候也可能是模型形式選錯的訊號。
測量誤差具有系統性	如果測量誤差不是隨機發生，而是受到儀器漂移、校正不足或系統性偏差影響，殘差之間可能會呈現相關。	在連續生產過程中，如果測量設備隨時間產生誤差累積，模型殘差可能會呈現時間上的趨勢。這種情況下，殘差並不是隨機分布，而是受到測量設備狀態的影響。
順序效應或學習效應	在實驗或調查中，受試者或受訪者可能隨著時間改變行為，	在測試新藥的臨床試驗中，患者可能隨著時間產生耐受性或

	使得後面的觀測結果受到前面觀測結果影響。	適應效果。如果模型沒有考慮這種變化，殘差可能會呈現時間上的自相關。又例如，在教育測驗中，學生可能因為練習而逐漸熟悉題型，導致後續測驗表現受到前面測驗經驗影響。
空間自相關 （Spatial Autocorrelation）	在地理資料中，鄰近地區的觀測值可能彼此相似。如果模型沒有納入空間位置或地區因素，殘差可能會在空間上呈現相關。	在分析城市房價時，鄰近地區的房價往往相似。如果模型沒有考慮地理位置、學區、交通便利性、生活機能或區域發展等因素，殘差可能會出現空間自相關。這代表模型在某些地區可能系統性高估或低估房價。

表：殘差可能彼此相關的原因

重點	**說明**
殘差獨立性的重要性	殘差獨立性是迴歸模型的重要假設之一。
殘差自相關代表什麼	如果殘差之間存在自相關，代表模型可能仍有未捕捉到的結構，例如時間趨勢、空間群聚、遺漏變數或模型形式錯誤。
可能造成的問題	殘差自相關可能使標準誤、p-value、信賴區間與假設檢定結果變得不可靠。
殘差分析的檢查重點	在進行殘差分析時，除了檢查殘差是否接近常態分布，也應該檢查殘差是否具有獨立性。
理想情況	殘差應該像隨機雜訊一樣分布，彼此之間沒有明顯相關。
可能有問題的情況	殘差出現時間趨勢、週期性、連續正負偏差、空間群聚或其他系統性模式。

表：殘差分析的重點說明

- Durbin-Watson Test：

  Durbin-Watson 檢定是一種專門用來檢查殘差自相關的檢定，特別是自迴歸模型。它的檢定統計值範圍為 0 到 4，接近 2 表示沒有自相關，接近 0 表示正自相關，接近 4 表示負自相關。當自相關存在時，回歸係數的顯著性可能會被過度估計。

變異數同質性

變異數同質性（Homoscedasticity）假設是指殘差的變異數應該在所有解釋變數的範圍內保持不變（不管解釋變數是大還是小，模型殘差的分散程度應該差不多）。若變異數不等（即異質性，殘差的分散程度會隨著解釋變數改變），可能會導致回歸模型中的參數估計失真，進而影響預測結果的可靠性。異質變異數不一定會讓迴歸係數本身有偏誤，但會讓標準誤、信賴區間、p-value 和假設檢定變得不可靠。

房子	實際房價	預測房價	殘差
A	1000 萬	950 萬	+50 萬
B	2000 萬	2100 萬	-100 萬
C	3000 萬	2800 萬	+200 萬

表：殘差代表模型預測錯多少

房價區間	殘差大約範圍
低價房	-100 萬到 +100 萬
中價房	-100 萬到 +100 萬
高價房	-100 萬到 +100 萬

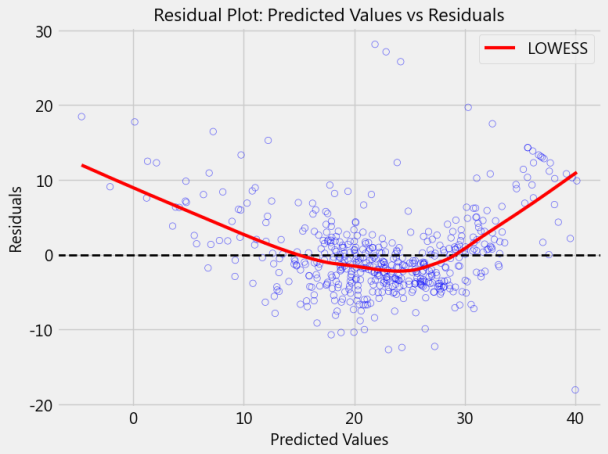
表：代表模型在不同房價區間的錯誤幅度差不多，符合變異數同質性

房價區間	殘差大約範圍
低價房	-50 萬到 +50 萬
中價房	-150 萬到 +150 萬
高價房	-500 萬到 +500 萬

表：房價越高，模型預測誤差越大，這就是變異數異質性。

- 局部加權散點平滑法 (**Lo**cally **Wei**ghted **S**catterplot **S**moothing, LOWESS)：

  LOWESS 是一種常用於檢查變異數同質性的工具。通過對不同區段的殘差進行加權平滑，LOESS 可以有效地繪製出殘差隨解釋變數的變化趨勢。如果平滑曲線呈現一定的模式（系統性模式，代表模型犯錯有規律，模型還有東西沒學到），則可能存在異質性，這樣的模式會顯示出殘差與預測值之間的關係，而不是隨機分佈。



圖：模型可能沒有捕捉到非線性關係。

理想狀況	意義
殘差點隨機分布在 0 附近	模型沒有明顯系統性偏差
LOWESS 紅線接近水平且靠近 0	殘差平均趨勢沒有明顯模式
不同預測值範圍下，殘差上下散布寬度差不多	變異數同質性較合理
沒有明顯曲線、漏斗形或極端離群點	模型假設較合理

表：理想的殘差圖應該長怎樣

正規化的線性迴歸

Ridge Regression

Ridge Regression 中文常稱為嶺迴歸（或脊迴歸），是一種加入 L2 正規化（有時稱為正則化，regularization）項的線性迴歸模型。一般線性迴歸的目標是讓模型預測值與真實值之間的誤差越小越好；Ridge Regression 則是在降低預測誤差的同時，也限制模型係數不要變得太大。

它的主要目的不是改變線性模型的基本形式，而是讓模型的係數變得比較穩定，降低過度擬合（Overfitting）的風險。注意：正規化的作法，會進行特徵之間的相互比較，建議先對所有特徵進行標準化，才能正確比較各個特徵的影響力，取得較佳的迴歸結果。如果模型係數太大，就會受到懲罰。因此，Ridge Regression 不只是希望預測準確，也希望模型係數不要太大。

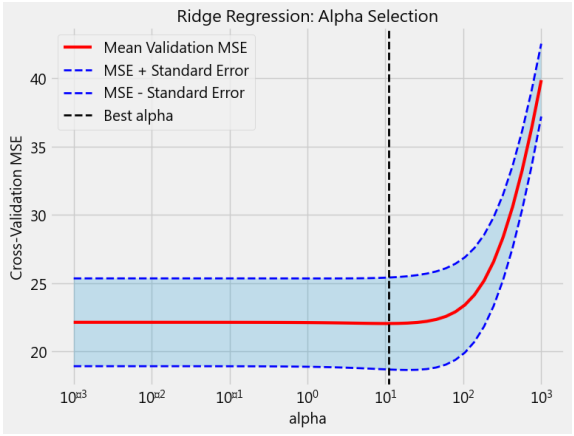
$$\text{Ridge Loss} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} w_j^2$$

其中

- n：資料筆數
- p：特徵數量
- i：第 i 筆資料
- j：第 j 筆特徵
- y_i：第 i 筆資料的真實值
- $\hat{y}_i$：第 i 筆資料的模型預測值
- w_j：第 j 個特徵的係數
- w_j^2：第 j 個特徵係數的平方
- λ：正規化強度，控制徵罰係數大小的程度

λ 的大小	**意義**
$\lambda = 0$	沒有正規化，等同於普通線性迴歸
λ 較小	對係數懲罰較弱
λ 較大	對係數懲罰較強，係數會被壓得更小
λ 過大	模型可能太簡單，導致欠擬合（Underfitting）

表：λ 的意義



圖：用於 Ridge Regression 的 Alpha 選擇

如果有些係數會變大，主要原因是特徵之間可能有共線性，也就是特徵彼此相關，普通線性迴歸的係數分配可能不穩定。Ridge Regression 會透過 L2 正規化限制整體係數大小，使模型係數較穩定，降低過度擬合與共線性造成的不穩定問題。

需要注意的是，Ridge 不保證每一個係數都會比普通線性迴歸更小。因為 Ridge 是同時考慮所有特徵與整體損失函數，當特徵之間存在相關性時，模型可能會重新分配各特徵的權重。

因此，有些係數可能變小，有些係數可能略微變大，甚至有些接近 0 的係數可能改變方向。Ridge 的重點不是讓每一個係數都變小，而是讓整體係數大小受到限制，使模型更穩定。

Lasso Regression

Lasso Regression 中文常稱為套索迴歸，是一種加入 L1 正規化項 的線性迴歸模型。一般線性迴歸只希望模型的預測誤差越小越好；Lasso Regression 則是在降低預測誤差的同時，也會限制模型係數不要太大，最重要的特色是：它可以把某些特徵的係數壓縮成 0。因此，Lasso Regression 不只可以降低過度擬合，也可以用來做特徵選擇。

簡單來說，Lasso Regression 會透過 L1 正規化，讓一些影響較小或不重要的特徵係數變成 0。如果某個特徵係數變成 0，代表模型等於不使用這個特徵。

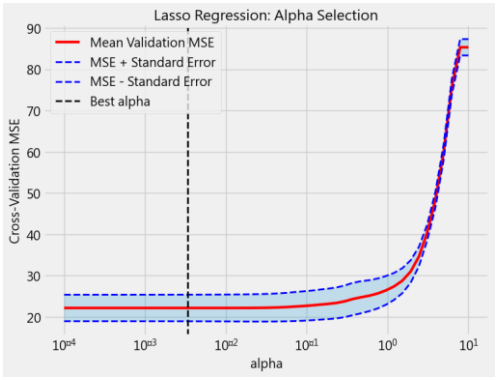
$$\text{Lasso Loss} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |w_j|$$

其中

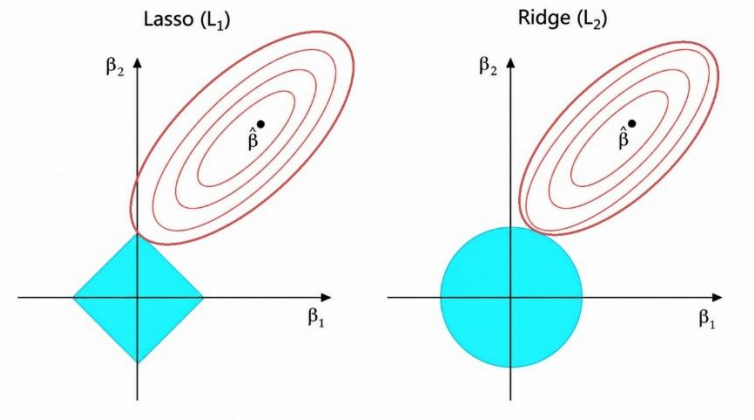
- n：資料筆數
- p：特徵數量
- i：第 i 筆資料
- j：第 j 筆特徵
- y_i：第 i 筆資料的真實值
- $\hat{y}_i$：第 i 筆資料的模型預測值
- w_j：第 j 個特徵的係數
- $|w_j|$：第 j 個特徵的係數取絕對值
- λ：正規化強度，控制徵罰係數大小的程度

λ 的大小	**意義**
$\lambda = 0$	沒有正規化，等同於普通線性迴歸
λ 較小	對係數懲罰較弱，保留較多特徵
λ 較大	對係數懲罰較強，更多係數可能被壓成 0
λ 過大	太多係數變成 0，模型可能欠擬合（Underfitting）

表：λ 的意義



圖：用於 Lasso Regression 的 Alpha 選擇



圖：Lasso 與 Ridge 的正規化效果

Elastic Net

Elastic Net 中文常稱為彈性網路迴歸，是一種同時結合 L1 正規化 與 L2 正規化 的線性迴歸模型，同時保留兩者的優點。

$$\text{Elastic Net Loss} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_{j=1}^{p} |w_j| + (1-\alpha) \sum_{j=1}^{p} w_j^2 \right]$$

α 的值	模型效果
$\alpha = 1$	完全等同於 Lasso Regression
$\alpha = 0$	完全等同於 Ridge Regression
$1 < \alpha < 1$	同時混合 Lasso 與 Ridge

α 越接近 1	越偏向 Lasso，較容易產生 0 係數
α 越接近 0	越偏向 Ridge，係數較平滑穩定

表：這裡的 α 用來控制 Elastic Net 比較像 Lasso 還是 Ridge

數學符號	scikit-learn 參數
λ	alpha
α 或混合比例	l1_ratio

表：在數學公式中，α 有時代表 L1 與 L2 的混合比例；但在 scikit-learn 中，alpha 代表整體正規化強度，l1_ratio 才是 L1 與 L2 的混合比例。

l1_ratio	意思	模型效果
0.0	完全偏向 Ridge	只縮小係數，通常不會讓係數變 0
0.5	Ridge 和 Lasso 各一半	同時縮小係數，也可能讓部分係數變 0
1.0	完全偏向 Lasso	可能把部分係數壓成 0，具有特徵選擇效果

表：l1_ratio 控制的是 Lasso 懲罰項佔多少比例

多項式迴歸（Polynomial Regression）

多項式迴歸是一種將線性模型拓展成非線性模型的迴歸方法。它引入了自變量的多項式形式。它的核心想法是將原本的特徵轉換成多項式特徵，例如平方項、三次方項，再使用線性迴歸模型進行訓練。

原本只有一個特徵 x，普通線性迴歸只能建立 $\hat{y} = w_0 + w_1 x$，這是一條直線；但是多項式迴歸可以加入 x^2、x^3 等特徵：$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3$，這樣模型就可以描述彎曲的關係。

普通線性迴歸假設特徵和目標變數之間是線性關係，但是在真實資料中，很多關係不是直線，而是曲線，如果用普通線性迴歸描述這些資料，模型可能太簡單，殘差圖也可能出現系統性曲線模式。因此，多項式迴歸可以用來讓線性模型捕捉非線性關係。

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + w_3 x_i^3 + \cdots + w_d x_i^d$$

其中

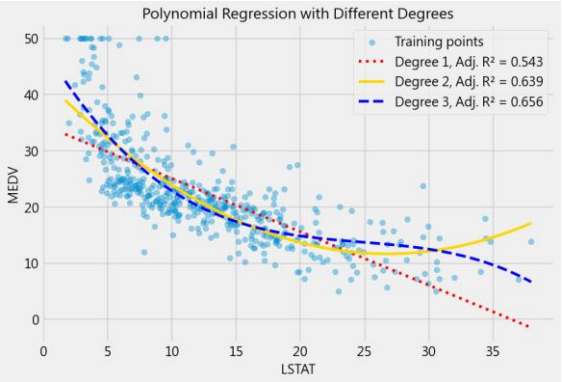
- d：多項式的最高次方，也稱為 degree
- x_i^d：第 i 筆資料的特徵值 x_i 的 d 次方
- w_d：最高次方項 x_i^d 的係數

當 $d = 1$ 時，就是普通線性迴歸。

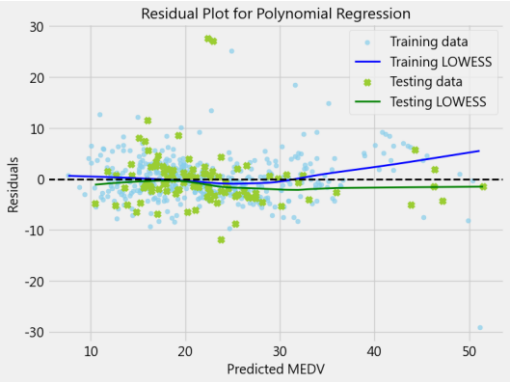
當 $d = 2$ 時，就是二次多項式迴歸。

當 $d = 3$ 時，就是三次多項式迴歸。

多項式迴歸雖然可以畫出曲線，但它仍然屬於線性模型，因為多項式迴歸對係數來說仍然是線性的，這些迴歸係數沒有平方、相乘或放在非線性函數裡，所以多項式迴歸可以理解為：先把特徵轉換成多項式特徵，再使用線性迴歸學習係數。



圖：不同次方用在多項式迴歸模型



圖：多項式轉換後的特徵來進行殘差分析

隨機森林迴歸（Random Forest Regression）

隨機森林迴歸是一種由多棵決策樹組成的集成式（Ensemble）迴歸模型，它的核心想法是建立很多棵決策樹，讓每一棵樹都做出預測，最後再把所有樹的預測結果取平均，如果是分類問題，隨機森林通常使用多數決；如果是迴歸問題，隨機森林通常使用平均值。因此，隨機森林迴歸可以理解為：多棵決策樹迴歸模型的平均結果。

單一決策樹有一個常見問題，它很容易過度擬合訓練資料，也就是說，單一決策樹可能把訓練資料中的細節、雜訊甚至離群值都記住，導致訓練資料表現很好，但測試資料表現變差，隨機森林可以透過 Bootstrap 和隨機選取特徵來減緩問題。因此，每一棵樹都會有些不同，最後把多棵樹的結果平均，可以降低單一模型的不穩定性。

$$\hat{y}_i = \frac{1}{B}\sum_{b=1}^{B}\hat{y}_i^{(b)}$$

其中

- i：第 i 筆資料
- B：隨機森林中的決策樹數量
- b：第 b 棵決策樹

- $\hat{y}_i^{(b)}$：第 b 棵樹對第 i 筆資料的預測值
- $\hat{y}_i$：隨機森林對第 i 筆資料的最終預測值

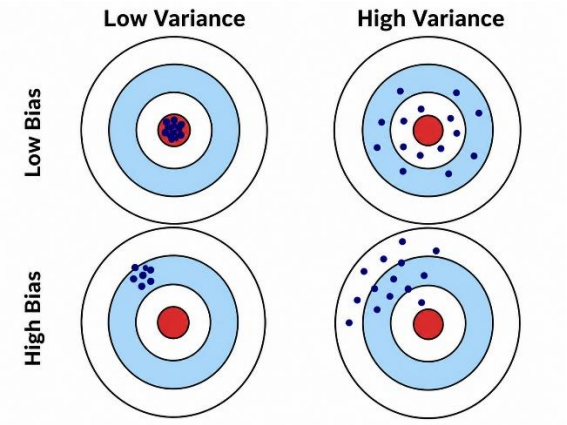
這個公式的意思是：隨機森林會把所有決策樹的預測結果加總後取平均，作為最後的迴歸預測值。

方法	說明
Bootstrap 抽樣	每一棵樹使用不同的抽樣資料訓練
隨機選取特徵	每次分裂節點時，只從部分特徵中選擇最佳分裂

表：隨機森林透過兩種方式降低過度擬合（Overfitting）的問題

隨機森林還會在每次節點分裂時，只從部分特徵中挑選最佳分裂，來達到隨機選取特徵。假設原本有 13 個特徵，單一決策樹在每次分裂時可能會檢查全部 13 個特徵，但隨機森林可能每次只隨機抽出部分特徵，例如 4 個特徵，然後從這 4 個裡面找最佳分裂。這樣做的目的，是讓不同樹之間更有差異，如果每一棵樹都太像，平均的效果就有限；如果每一棵樹有適度差異，最後平均起來通常會更穩定。

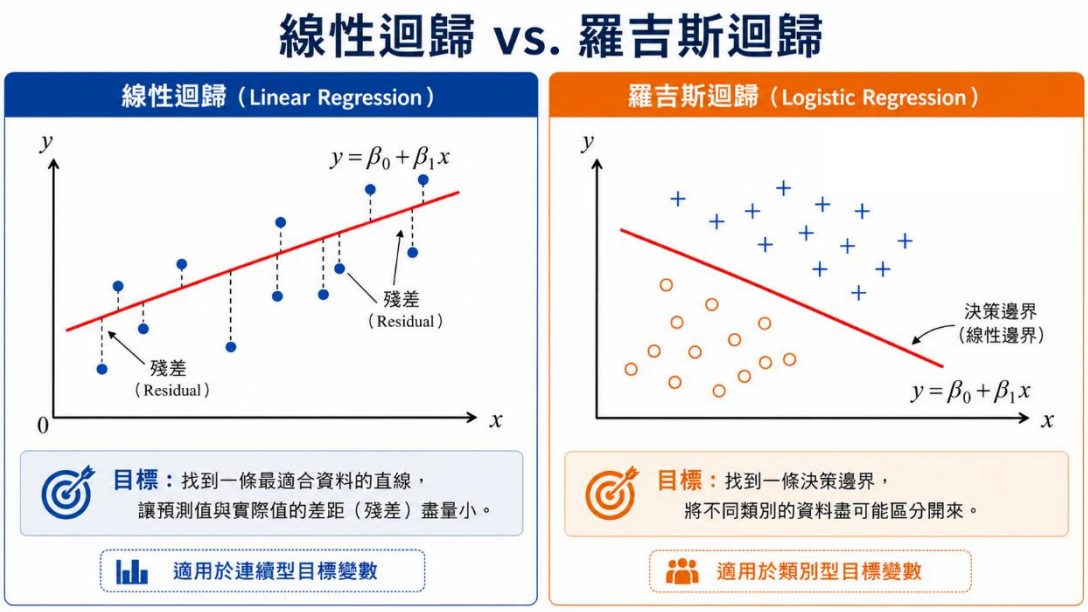
Biass（偏差）和 Variance（變異）是用來理解模型誤差來源的兩個重要概念。Bias 指的是模型的平均預測結果，和真實規律之間的差距，如果一個模型太簡單，它可能無法捕捉資料中的真實關係，這時就會有高 Bias；Variance 指的是如果換一批訓練資料，模型預測結果會不會變很多，如果一個模型太複雜，它可能會把訓練資料中的細節、雜訊或離群值都記住，這時候模型在訓練資料上表現很好，但換到測試資料時表現變差。



圖：Bias-Variance Trade-off

Module 3. 監督式學習：分類

在監督式學習中，模型會從「已知答案」的資料中學習規則，再用這些規則預測新的資料。若預測目標是連續數值，例如房價、溫度、銷售額，這類問題稱為迴歸問題；若預測目標是類別，例如鳶尾花品種、是否為雙屬性寶可夢、是否為傳奇寶可夢，這類問題稱為分類問題。



圖：線性迴歸與羅吉斯迴歸的差異

迴歸 vs. 分類

迴歸（Regression）是監督式學習的一種任務，目標是預測連續型數值。例如：

- 根據房屋坪數、屋齡、地點預測房價。
- 根據廣告支出預測銷售額。
- 根據學生讀書時間預測考試分數。

迴歸問題的輸出通常是數值，例如：$\hat{y} = 35.7$，其中 $\hat{y}$ 代表模型預測出的數值。

分類（Classification）也是監督式學習的一種任務，目標是預測類別標籤。例如：

- 根據花的長度與寬度預測鳶尾花品種。
- 根據寶可夢能力值預測是否有第二屬性。

- 根據電子郵件內容預測是否為垃圾郵件。
- 根據病患檢查資料預測是否罹患某種疾病。

分類問題的輸出通常是類別，例如：$\hat{y} \in \{setosa, versicolor, virginica\}$，其中 $\hat{y}$ 代表模型預測出的類別。

比較項目	迴歸	分類
預測目標	連續數值	類別標籤
輸出範例	房價、溫度、分數	是/否、品種、類別
常見模型	線性迴歸、決策樹迴歸、隨機森林迴歸	邏輯斯迴歸、SVM、樸素貝氏、決策樹分類、隨機森林分類、KNN
常見評估指標	MAE、MSE、RMSE、R^2	Accuracy、Precision、Recall、F1-score、混淆矩陣

表：迴歸與分類的差異

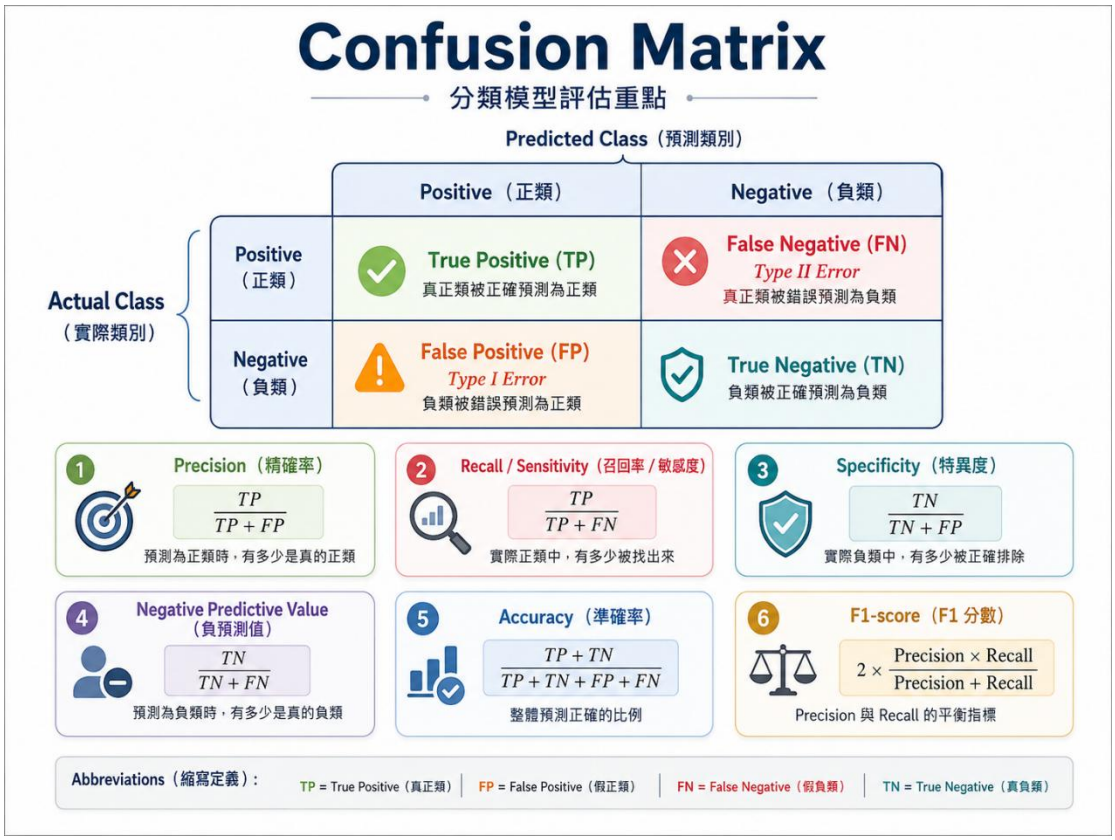
評估分類器的效能

分類模型訓練完成後，不能只看模型是否可以輸出結果，還要評估模型的預測是否正確。分類模型常見的評估方式包括：

- 混淆矩陣
- Accuracy
- Precision
- Recall
- F1-score

不同指標適合不同場景。例如資料類別很平均時，Accuracy 通常可以作為初步參考；但如果資料類別不平衡，例如傳奇寶可夢數量遠少於非傳奇寶可夢，則 Precision、Recall、F1-score 會比 Accuracy 更有參考價值。

混淆矩陣與其它評估指標



圖：用在二元分類的混淆矩陣

名稱	英文	意義
真陽性	True Positive, TP	真實為正類，模型也預測為正類
真陰性	True Negative, TN	真實為負類，模型也預測為負類
假陽性	False Positive, FP	真實為負類，但模型預測為正類
假陰性	False Negative, FN	真實為正類，但模型預測為負類

表：以二元分類為例，混淆矩陣包含四種結果

Accuracy（準確率）表示所有資料中，模型預測正確的比例：

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision（精確率）表示模型預測為正類的資料中，有多少比例真的屬於正

類。例如：

- 垃圾郵件偵測：如果模型把正常郵件誤判為垃圾郵件，可能造成重要信件遺失，因此 Precision 很重要。
- 疾病診斷輔助：如果誤判陽性會造成不必要的壓力與檢查，也需要重視 Precision。

$$Precision = \frac{TP}{TP + FP}$$

Recall（召回率）表示真實為正類的資料中，有多少比例被模型成功找出來。例如：

- 疾病篩檢：若漏掉真正生病的人會造成嚴重後果，Recall 很重要。
- 詐騙偵測：若漏掉真正的詐騙交易，可能造成損失，因此 Recall 也很重要。

$$Recall = \frac{TP}{TP + FN}$$

F1-score 是 Precision 與 Recall 的調和平均數，用來在 Precision 與 Recall 之間取得平衡（適合用在類別不平衡的分類問題）：

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

補充：Support 表示某個類別在測試資料中的真實樣本數：

類別	support
0	95
1	129

表：代表測試資料中，類別 0 有 95 筆，類別 1 有 129 筆，在 classification_report() 會看到。

多元分類器

1. 二元 vs. 多元

二元分類（Binary Classification）是指目標類別只有兩種。例如：

- 是否有第二屬性：有 / 沒有
- 是否為傳奇寶可夢：是 / 否
- 是否為垃圾郵件：是 / 否

多元分類（Multiclass Classification）是指目標類別有三種以上。例如鳶尾花資料集有三個品種（setosa, versicolor, virginic），這種問題不能只判斷「是或否」，而是要從多個類別中選出一個最可能的類別。

2. One-vs-Rest（OvR，一對餘）和 One-vs-One（OvO，一對一）

OvR 策略指的是 「一對餘」(One-vs-Rest) 策略。這種方法將多類別分類問題轉化為多個二元分類問題（例如某一類 vs. 其它所有類），對於每個類別都訓練一個分類器（例如水系寶可夢跟非水系寶可夢）。每個分類器都學習如何區分「該類別」與「其他所有類別」。在預測時，將輸入數據通過所有分類器，選擇得分最高的那個分類器所對應的類別作為最終預測結果。

OvO 策略指的是 「一對一」(One-vs-One) 策略。這種方法將多類別分類問題轉化為多個二元分類問題，為每一對類別（任取兩個類別）組合訓練一個分類器。具體而言，假設有 K 個類別，則需要訓練 $\frac{K(K-1)}{2}$ 個分類器。每個分類器都學習如何在兩個特定的類別之間進行區分。當進行預測時，將輸入數據通過所有分類器，採用「投票」機制，選擇得票數最多的類別作為最終預測結果。

方法	訓練分類器數量	優點	缺點
OvR	K	訓練數量較少，概念直觀	每個分類器都要面對類別不平衡問題
OvO	$\frac{K(K-1)}{2}$	每次只比較兩個類別，分類邊界可能較清楚	類別很多時，分類器數量會快速增加

表：OvR 與 OvO 比較

分類結果的可靠度

分類模型不只會輸出類別，也可以輸出模型對各類別的信心程度。常見方式包括：

- decision function
- predicted probability

Decision function 表示模型對某個類別的判斷分數。分數越高，代表模型越傾向於將樣本分類到該類別。以線性模型為例，decision function 通常可表示為：

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

也可以寫成：

$$z = \mathbf{w}^T\mathbf{x} + b$$

其中：

- z：模型計算出的分數
- x：輸入特徵向量
- w：特徵權重向量
- b：截距項
- n：特徵數量

若 z 越大，通常代表模型越傾向於正類；若 z 越小，代表模型越不傾向於正類。

Predicted probability 表示模型預測某筆資料屬於各類別的機率。以三類鳶尾花分類為例，模型可能輸出：

setosa	versicolor	virginica
0.001	0.237	0.762

表：這代表模型認為該筆資料最有可能是 virginica，因為 virginica 的機率最高。

在鳶尾花分類中，如果模型預測某筆資料為 virginica，並且輸出機率：

$$P(\text{virginica}|\mathbf{x}) = 0.762$$

這表示模型認為該資料屬於 virginica 的機率約為 76.2%。

如果模型輸出：

$$P(\text{setosa}|\mathbf{x}) = 0.34, \quad P(\text{versicolor}|\mathbf{x}) = 0.33, \quad P(\text{virginica}|\mathbf{x}) = 0.33$$

雖然模型仍然會選擇 setosa，但其實三個類別的機率很接近，代表模型對這筆資料的判斷並不穩定。

邏輯斯迴歸

邏輯斯迴歸（Logistic Regression）是一種常用於分類問題的線性模型。雖然名稱中有「迴歸」，但它主要用於分類任務。邏輯斯迴歸的核心想法是：

- 先用線性模型計算分數。
- 再透過 sigmoid 函數把分數轉換成 0 到 1 之間的機率。
- 最後根據機率決定類別。

Odds（勝算）

假設事件發生的機率為 p，事件不發生的機率為 1-p，則勝算為：

$$Odds = \frac{p}{1-p}$$

例如，某寶可夢有第二屬性的機率為 $p = 0.8$，則沒有第二個屬性的機率為 $1 - p = 0.2$，則勝算為：

$$Odds = \frac{0.8}{0.2} = 4$$

表示「有第二屬性」的勝算是「沒有第二屬性」的 4 倍。

Logit 函數

是對勝算取自然對數：

$$logit(p) = \log\left(\frac{p}{1-p}\right)$$

其中

- p：事件發生的機率
- $1 - p$：事件不發生的機率
- log：自然對數

邏輯斯迴歸假設 logit 與特徵之間具有線性關係：

$$logit(p) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

其中

- β_0：截距
- β_i：第 i 個特徵的權重
- x_i：第 i 個特徵值

- n：特徵數量

Sigmoid 函數

因為線性模型的輸出可能是任意實數，因此邏輯斯迴歸會使用 sigmoid 函數把分數轉成 0 到 1 之間的機率：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

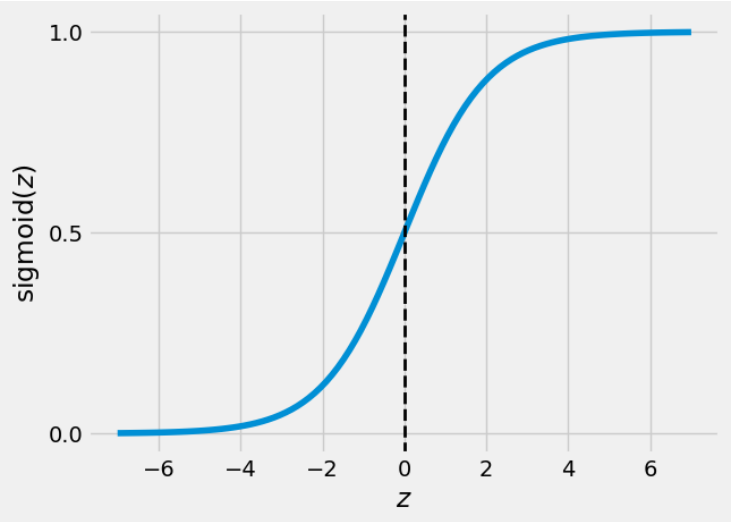
$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

其中

- $\sigma(z)$：sigmoid 函數輸出值
- z：線性模型計算出的分數
- e：自然常數

當 z 很大時，σ(z) 接近 1；當 z 很小時，σ(z) 接近 0；當 z=0 時：σ(z)=0.5，因此在二元分類中，常用 0.5 作為分類門檻：

$$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$$



圖：sigmoid 函數

加入正規化

正規化（Regularization）是一種防止模型過度擬合的方法。它會在模

型訓練時加入懲罰項，使模型不要過度依賴某些特徵，並降低模型複雜度。在邏輯斯迴歸中，模型需要學習每個特徵的權重。如果權重太大，模型可能對訓練資料過度敏感，導致在新資料上的表現變差。

L1 正規化

會懲罰權重絕對值總和：

$$L1 = \lambda \sum_{j=1}^{n} |w_j|$$

其中

- λ：正規化強度
- w_j：第 j 個特徵的權重
- n：特徵數量

L1 正規化的特色是可能讓某些權重變成 0，因此具有特徵選取效果。

L2 正規化

會懲罰權重平方和：

$$L2 = \lambda \sum_{j=1}^{n} w_j^2$$

L2 正規化會讓權重變小，但通常不會直接變成 0。

C 與正規化強度

在 scikit-learn 的 LogisticRegression 中，常使用 C 控制正規化強度。C 與正規化強度成反比：

$$C = \frac{1}{\lambda}$$

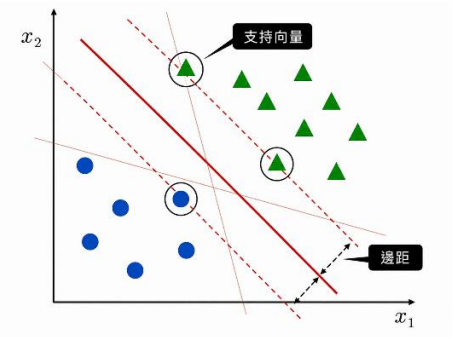
其中

- C 越大，正規化越弱，模型越容易複雜
- C 越小，正規化越強，模型越簡單

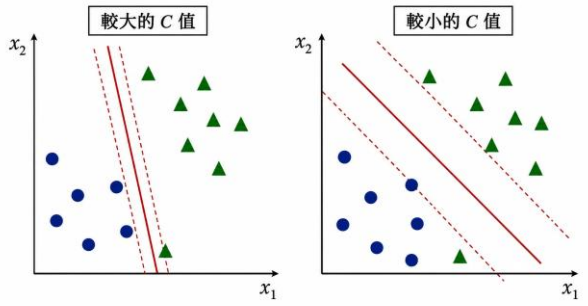
支援向量機

支援向量機（Support Vector Machine, SVM）是一種常用於分類的監督式學習模型。SVM 的核心目標是找到一條最佳的分類邊界，使不同類別之間

的距離最大化。在二維空間中，分類邊界是一條線；在三維空間中，分類邊界是一個平面；在更高維空間中，分類邊界稱為超平面。



圖：支援向量機 SVM



圖：不同超參數 C 值對 SVM 邊距的影響

超平面（Hyperplane）

線性 SVM 的分類邊界可表示為：

$$\mathbf{w}^T\mathbf{x} + b = 0$$

其中

- x：輸入特徵向量
- w：權重向量
- b：截距
- $w^T x + b$：模型的 decision function

邊距（Margin）

SVM 會尋找能最大化兩類資料間隔的分類邊界。邊距可理解為分類邊界到最近樣本點的距離。對線性 SVM 而言，最大化邊距可寫成最小化權重大小：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$$

如果允許部分資料被錯誤分類，則會加入鬆弛變數：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\xi_i$$

其中

- m：訓練樣本數
- ξ_i：第 i 筆資料的錯誤容忍程度

- C：錯誤分類的懲罰強度
  - C 大：模型較不能容忍錯誤分類，可能形成較複雜的分類邊界，容易過度擬合。
  - C 小：模型較能容忍錯誤分類，分類邊界較平滑，可能過度簡化。

線性支援向量機

線性支援向量機（Linear SVM）適合處理資料大致可以用線性邊界分開的分類問題。分類規則可表示為：

$$\hat{y} = \begin{cases} 1, & \mathbf{w}^T\mathbf{x} + b \geq 0 \\ 0, & \mathbf{w}^T\mathbf{x} + b < 0 \end{cases}$$

其中

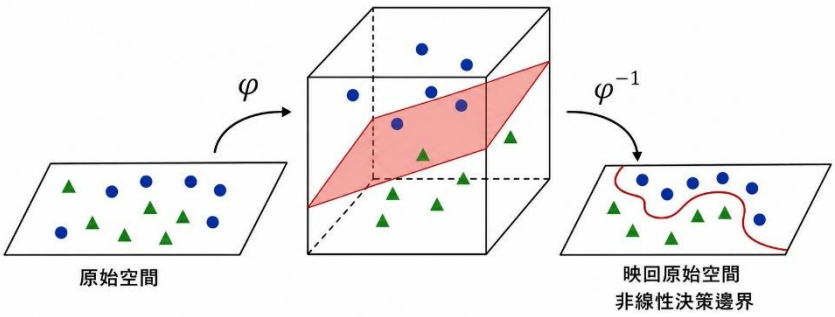
- $\hat{y}$：模型預測類別
- x：輸入特徵
- w：權重
- b：截距

如果寶可夢是否有第二屬性可以大致由能力值的線性組合區分，LinearSVC 就可以建立一個線性分類邊界。例如模型可能學到：

$$z = 0.2 \cdot HP + 0.5 \cdot Defense - 0.1 \cdot Speed + b$$

若 z 大於 0，模型預測為有第二屬性；反之則預測為沒有第二屬性。

加入核函數處理非線性分類



圖：使用 kernel function 讓 SVM 產生非線性決策邊界

當資料無法用直線或線性超平面分開時，就稱為非線性分類問題。SVM 可以透過核函數將資料映射到更高維度空間，使原本非線性可分的資料在高維空間中變得線性可分。常見核函數包括：

- linear kernel
  - LinearSVC 直接學 w 和 b，透過迴歸找出區分資料的「線」或「平面」，再找出分類邊界（margin），support vectors 不見得是重點。
  - SVC(kernel='linear') 先找出關鍵資料，也就是 support vectors，要預測新資料時，再看新資料和這些 support vectors 的關係。
- sigmoid kernel（較少使用）
- polynomial kernel
- RBF kernel（最常用）

RBF 核函數

是常見的核函數，全名為 Radial Basis Function。公式為：

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

其中

- $K(x_i, x_j)$：兩筆資料的相似度
- x_i, x_j：兩筆資料的特徵向量
- γ：控制影響範圍的參數
- $\|x_i - x_j\|^2$：兩筆資料之間的平方距離

假設有兩筆資料：$x_i = (2,3)$，$x_j = (4,3)$

先算平方距離：$\|x_i - x_j\|^2 = (2-4)^2 + (3-3)^2 = 4 + 0 = 4$

假設 $\gamma = 0.5$，代入 RBF kernel：

$$K(x_i, x_j) = \exp(-0.5 \times 4) = \exp(-2) \approx \mathbf{0.135}$$

假設另一筆資料是：$x_k = (2.5, 3)$

和 $x_i = (2,3)$ 的距離是：$\|x_i - x_k\|^2 = (2-2.5)^2 + (3-3)^2 = 0.25 + 0 = 0.25$

γ 同上，代入 RBF kernel：

$$K(x_i, x_k) = \exp(-0.5 \times 0.25) = \exp(-0.125) \approx \mathbf{0.882}$$

兩筆資料	平方距離	RBF kernel 值	解釋
(2,3) 和 (4,3)	4	0.135	距離較遠，相似度低
(2,3) 和 (2.5,3)	0.25	0.882	距離較近，相似度高

表：RBF kernel 的核心 → 距離越近，相似度越高。

Gamma 參數

γ 控制單一樣本對周圍空間的影響範圍

- γ 大：每個資料點的影響範圍小，模型邊界較複雜，容易過度擬合。
- γ 小：每個資料點的影響範圍大，模型邊界較平滑，容易過度簡化。

γ	影響範圍	模型特性	風險
小	大	邊界平滑	可能 underfitting
大	小	邊界複雜	可能 overfitting

表：γ 大小的影響

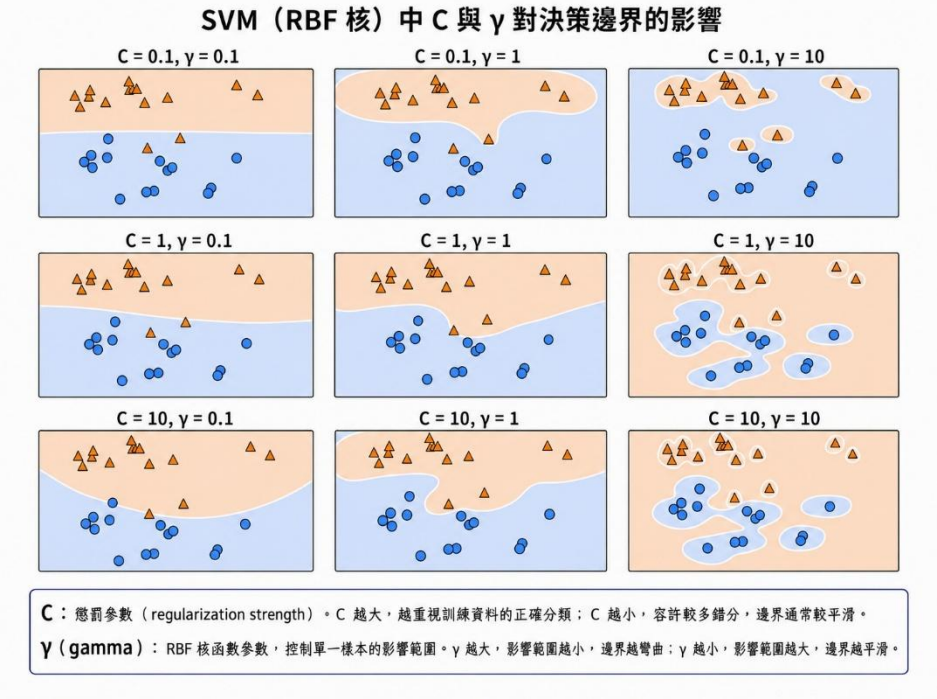
RBF SVM 的分類不是只看一條直線，而是看新資料 x 跟訓練資料中重要的點有多相似，這些重要的點叫做：support vectors。新資料 x 會跟每個 support vector 比較相似度，相似度高的 support vector 對分類結果影響比較大，相似度低的 support vector 影響比較小。

分類函數可以寫成：

$$f(x) = \sum_i a_i y_i K(x_i, x) + b$$

其中

- x：新資料
- x_i：第 i 個 support vector
- y_i：這個 support vector 的類別，通常是 $+1$ 或 -1
- α_i：這個 support vector 的重要程度
- $K(x_i, x)$：新資料與 support vector 的相似度
- b：偏移量



圖：使 kernel function 在不同 C 和 gamma 參數值的分類效果

樸素貝氏分類

樸素貝氏分類器（Naive Bayes Classifier）是一種基於貝氏定理的分類模型。它的核心假設是：在已知類別的條件下，各個特徵彼此獨立。「各個特徵彼此獨立」不是說特徵在現實世界真的完全沒有關係，而是指：在已經知道類別 y 的情況下，某一個特徵的出現，不會影響另一個特徵出現的機率。

雖然這個假設在真實資料中不一定完全成立，但樸素貝氏分類器計算快速、概念清楚，常用於文字分類、垃圾郵件偵測、簡單分類任務。

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

其中

- $P(A)$：事件 A 發生的機率，稱為先驗機率。
- $P(B)$：事件 B 發生的機率，稱為邊際機率。
- $P(B|A)$：在 A 發生的條件下，B 發生的機率，稱為 likelihood。
- $P(A|B)$：在 B 發生的條件下，A 發生的機率，稱為後驗機率。

若一筆資料有多個特徵：

$$\mathbf{x} = (x_1, x_2, \cdots, x_n)$$

則樸素貝氏分類器可表示為：

$$P(y|x_1, x_2, \cdots, x_n) = \frac{P(y)P(x_1|y)P(x_2|y)\cdots P(x_n|y)}{P(x_1)P(x_2)\cdots P(x_n)}$$

其中

- y：類別標籤
- x_i：第 i 個特徵
- $P(y)$：類別 y 的先驗機率
- $P(x_i|y)$：在類別 y 條件下，第 i 個特徵出現的機率
- $P(y|x_1, \ldots x_n)$：給定所有特徵後，資料屬於類別 y 的機率

因為分母對所有類別都相同，所以分類時通常只需要比較分子：

$$\hat{y} = \arg\max_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

高斯樸素貝氏（Gaussian Naive Bayes）

當特徵是連續型資料時，可以使用 Gaussian Naive Bayes。它假設每個特徵在某個類別下服從常態分配。

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

其中

- x_i：第 i 個特徵值
- μ_y：類別 y 下該特徵的平均值
- σ_y^2：類別 y 下該特徵的變異數

多項式樸素貝氏（Multinomial Naive Bayes）

當特徵是離散型計數資料時，可以使用 Multinomial Naive Bayes。常見應用是文字分類，例如每個詞出現的次數（詞頻）。這個範例會在自然語言處理（Natural Language Processing，NLP）的課程裡說明：
https://github.com/telunyang/python_nlp/blob/master/3-4%20%E7%B0%A1%E5%96%AE%E8%B2%9D%E8%91%89%E6%96%AF%E5%88%86%E9%A1%9E%E5%99%A8(Naive%20Bayes%20classifier).ipynb

使用場景或範例

在寶可夢資料集中，HP、Attack、Defense 等能力值是連續型特徵，因此可以使用 GaussianNB 預測是否為傳奇寶可夢。如果要根據某些離散型或計數型特徵預測類別，例如文字中詞彙出現次數，則可以考慮

MultinomialNB。

決策樹

在機器學習中，決策樹（Decision Tree）是一種監督式學習算法，廣泛應用於分類和迴歸問題。它通過樹狀結構來決策，逐步分割特徵空間來達到預測的目的。

決策樹的每個節點代表一個特徵的判斷，每個分支表示特徵的取值，每個葉節點則是最終的預測結果。決策樹是通過尋找資料集當中特徵的分割點，將資料分成不同的群組，從而達到決策的目的。

在分類問題中，決策樹通常使用資訊增益（Information Gain）、基尼指數（Gini Index）等方法來決定如何分割數據；在迴歸問題中，則常用平均平方誤差（Mean Squared Error）等指標。

結構	說明
根節點	決策樹最上層的節點
內部節點	根據某個特徵條件進行分割的節點
分支	特徵條件的不同結果
葉節點	最終預測結果

表：決策樹的組成

決策樹的基本想法

決策樹會不斷選擇一個特徵作為分割條件，使分割後的資料更純。所謂「更純」，是指分割後的節點中，資料越集中在同一個類別越好。例如預測是否有第二屬性時，決策樹可能學到：

- 如果 Defense 大於某個門檻，往左分支。
- 如果 Speed 小於某個門檻，往右分支。
- 最後到達葉節點，輸出 0 或 1。

決策樹的優點

- 解釋性強：決策樹的結構直觀明瞭，可以視覺化為分支結構，易於理解和解釋。
- 不需要大量的數據預處理：決策樹不需要特徵縮放或標準化，對於離散數據和連續數據都能很好地處理。
- 處理非線性數據：決策樹本質上是基於特徵的條件分割，適合處理非線性數據。

- 對缺失值和噪聲有一定的穩健性：決策樹可以處理一些特徵缺失的情況，對噪聲不敏感。
- 適合多種任務：既可以用於分類，也可以用於迴歸問題，應用場景廣泛。

決策樹的缺點

- 容易過度擬合。
- 對資料小變動可能敏感。
- 單棵樹的泛化能力可能不穩定。

資訊增益（Information Gain）

熵（Entropy）

熵用來衡量資料的不純度。若一個節點中資料類別越混亂，熵越高；如果所有資料都屬於同一類，熵為 0。

熵可以看成一種不確定性（Uncertainty），例如你的資料類別組成都是「A,A,A,A,A,…，A」，那麼熵就是 0（代表非常確定）；如果你的資料類別組成是「C,D,A,E,A,…,X」，那麼熵就會較高，不確定性也就愈高。

$$Entropy(S) = -\sum_{c=1}^{K} p_c \log_2(p_c)$$

其中

- S：某個資料集合
- K：類別數量
- p_c：資料集合中第 c 類的比例
- log_2：以 2 為底的對數

二元分類熵

如果只有兩個類別，熵可寫成：

$$Entropy(S) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

其中

- p_1：正類比例
- p_0：負類比例

資訊增益衡量某個特徵分割資料後，使熵下降多少：

$$Information\ Gain = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

其中

- S：分割前的資料集合
- A：用來分割的特徵
- $Values(A)$：特徵 A 的所有可能取值
- S_v：特徵 A 取值為 v 的子集合
- $|S_v|$：子集合 S
- $|S|$：原始資料集合的資料數

資訊增益越大，代表該特徵越能有效降低資料混亂程度，因此越適合作為分割條件。

在經典天氣資料範例中，假設原始資料共有 14 筆，其中 yes 有 9 筆、no 有 5 筆，原始熵為：

$$Entropy([9,5]) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) \approx 0.940$$

若使用 Outlook 分割後，三個分支分別為：

- sunny：2 yes、3 no
- overcast：4 yes、0 no
- rainy：3 yes、2 no

分割後加權熵為：

$$\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \approx 0.694$$

因此資訊增益為：

$$Information\ Gain = 0.940 - 0.694 = 0.246$$

這表示 Outlook 這個特徵可以降低約 0.246 的不純度。因為 Outlook 與其它特徵 Temperature/Humidity/Windy 相較之下，保留的資訊量最高，因此可以作為決策樹的樹根或節點。

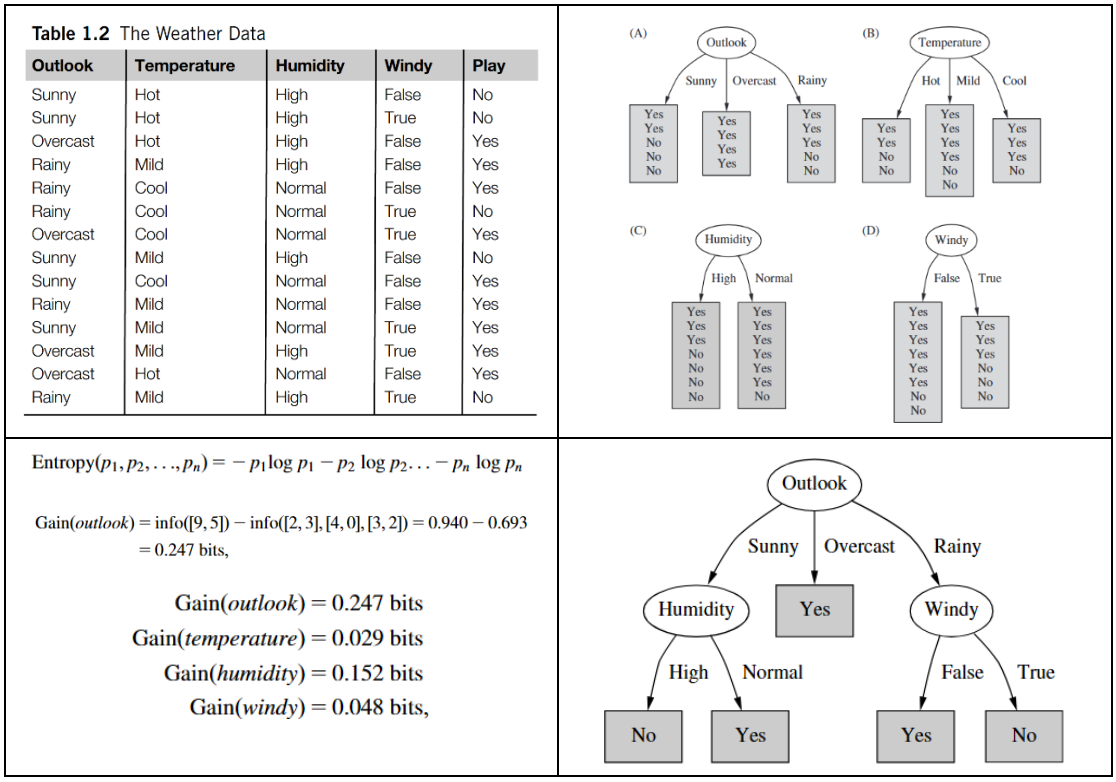


Table 1.2 The Weather Data

Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

$$\text{Entropy}(p_1, p_2, \ldots, p_n) = -p_1 \log p_1 - p_2 \log p_2 \ldots - p_n \log p_n$$

$$\text{Gain}(outlook) = \text{info}([9,5]) - \text{info}([2,3],[4,0],[3,2]) = 0.940 - 0.693 = 0.247 \text{ bits},$$

$$\text{Gain}(outlook) = 0.247 \text{ bits}$$
$$\text{Gain}(temperature) = 0.029 \text{ bits}$$
$$\text{Gain}(humidity) = 0.152 \text{ bits}$$
$$\text{Gain}(windy) = 0.048 \text{ bits},$$

圖：Information Gain 的計算流程。取自 https://www.tenlong.com.tw/products/9780128042915

決策樹分類與迴歸

分類時常見的分割標準包括：

- Entropy
- Information Gain
- Gini Index

Gini Index 也是常見的不純度指標：

$$Gini(S) = 1 - \sum_{c=1}^{K} p_c^2$$

其中

- S：資料集合
- K：類別數量
- p_c：第 c 類的比例

Gini 越小，表示節點越純。

決策樹也可以用於迴歸問題。不同於分類樹輸出類別，迴歸樹的葉節點會輸出數值，通常是該葉節點中訓練資料目標值的平均。迴歸樹常用 MSE 作為分割標準：

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2$$

其中

- n：樣本數
- y_i：第 i 筆資料的真實值
- $\hat{y}$：預測值

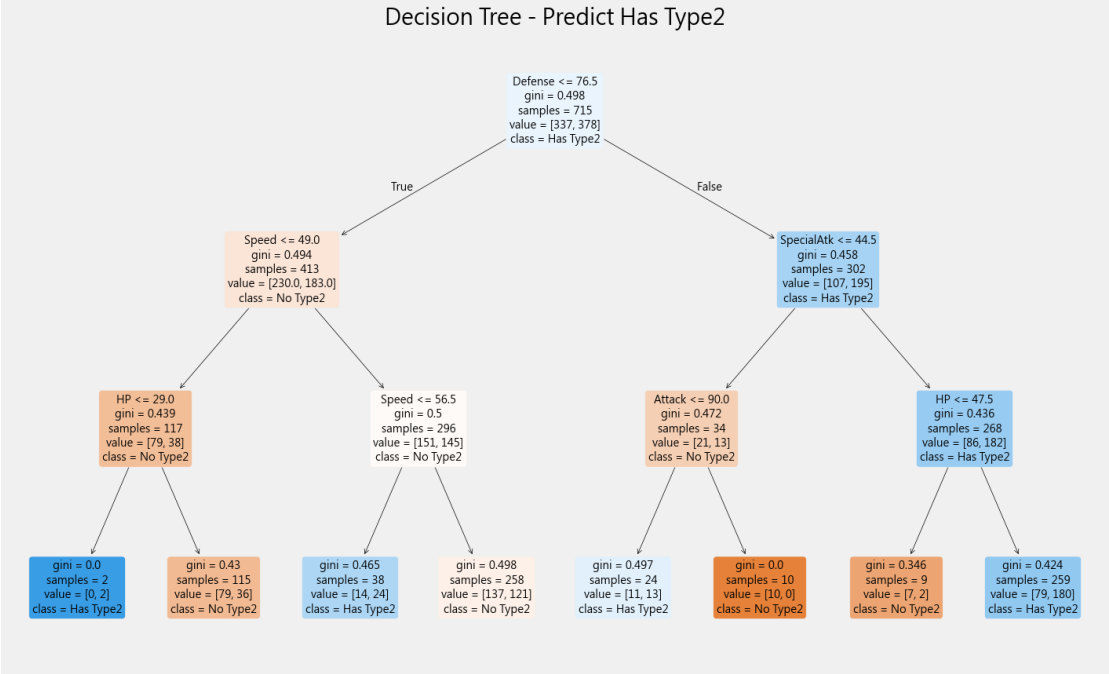
參數	中文說明	功能	參數調大時	參數調小時	建議
max_depth	最大深度	限制決策樹最多可以長到幾層	樹變深，模型較複雜，較容易過度擬合	樹變淺，模型較簡單，較容易欠擬合	可先設 max_depth=3，方便視覺化與解釋
min_samples_split	節點最少分裂樣本數	一個節點至少要有多少筆資料，才允許繼續分裂	分裂條件變嚴格，樹較簡單	分裂條件變寬鬆，樹較容易長很深	可試 min_samples_split=5 或 10
min_samples_leaf	葉節點最少樣本數	每個葉節點至少要保留多少筆資料	葉節點較大，模型較平滑，較不容易過度擬合	葉節點可能很小，容易記住訓練資料細節	官方建議可先嘗試 min_samples_leaf=5 作為起點；分類問題若類別少，1 有時也可行
max_leaf_nodes	最大葉節點數	限制整棵樹最多有幾個葉節點	若數值較大，樹可以更複雜	若數值較小，樹會更簡單	想控制樹的大小時很好用，例如 max_leaf_nodes=10
min_impurity_decrease	最小不純度下降量	只有當分裂能讓 impurity 下降超過指定值	分裂更嚴格，樹較小	分裂更寬鬆，樹較大	可用來避免意義很小的分裂

		時，才允許分裂			
ccp_alpha	成本複雜度修剪參數	用於 post-pruning，訓練後再修剪決策樹	修剪越強，樹越簡單	修剪越弱，樹越接近原本完整的樹	若要示範「後修剪」，可從 ccp_alpha=0.001 或 0.01 試起
max_features	每次分裂可考慮的最大特徵數	限制每次節點分裂時可以使用多少特徵	若使用較多特徵，分裂較精準但可能較容易過度擬合	若使用較少特徵，增加隨機性，可能降低過度擬合	單棵決策樹較少用；隨機森林中更常用
class_weight	類別權重	處理類別不平衡問題	可提高少數類別的重要性	若不設定，模型可能偏向多數類別	類別不平衡時可用 class_weight='balanced'
random_state	隨機種子	固定隨機性，使結果可重現	不影響模型複雜度	不影響模型複雜度	教學與作業建議固定，例如 random_state=0

表：避免決策樹過度擬合的常用參數

類型	中文名稱	做法	常用參數	優點	缺點
Pre-pruning	預先修剪	在決策樹成長過程中，就限制它不要長太深或分太細	max_depth、min_samples_split、min_samples_leaf、max_leaf_nodes	簡單、直觀、容易教學	若限制太嚴格，可能造成欠擬合
Post-pruning	後修剪	先讓樹長出來，再把不重要或過度複雜的部分剪掉	ccp_alpha	可以先看到完整樹，再逐步簡化	對初學者來說概念稍微進階

表：Pre-pruning 與 Post-pruning 比較



圖：決策樹的決策過程

整合多棵決策樹：隨機森林

隨機森林（Random Forest）是一種整合學習方法，由多棵決策樹組成。每棵樹都會做出預測，最後再整合所有樹的結果。分類問題中，隨機森林通常使用多數決：

$$\hat{y} = mode(\hat{y}_1, \hat{y}_2, \cdots, \hat{y}_T)$$

其中

- T：決策樹數量
- $\hat{y}_t$：第 t 棵樹的預測結果
- $mode$：眾數，也就是出現次數最多的類別

迴歸問題中，隨機森林通常使用平均值：

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T}\hat{y}_t$$

Bootstrap Aggregating（Bagging）

隨機森林使用 Bagging，也就是 Bootstrap Aggregating。它會透過

「有放回抽樣」產生多個不同的訓練子集。Bootstrap sample 的概念是：

$$D_t \sim Bootstrap(D)$$

其中

- D：原始訓練資料
- D_t：第 t 棵樹使用的 bootstrap sample

因為每棵樹看到的資料不同，所以每棵樹會學到不同規則。

特徵隨機選擇

隨機森林除了隨機抽樣資料，也會在每個節點分裂時隨機選擇部分特徵。這可以降低樹與樹之間的相似度，提高整體模型穩定性。假設資料有 6 個特徵：HP/Attack/Defense/SpecialAtk/SpecialDef/Speed

如果設定：

設定節點分裂的特徵數量，是以開根號來計算
RandomForestClassifier(n_estimators=100, max_features='sqrt', bootstrap=True, random_state=0)

因為共有 6 個特徵：

$$\sqrt{6} \approx 2.45$$

所以每個節點分裂時，大約會隨機挑一部分特徵來考慮，例如 2 個特徵。某棵樹的某個節點可能只考慮：

節點	隨機拿來比較的特徵
第 1 個節點	HP、Defense
第 2 個節點	Attack、Speed
第 3 個節點	Defense、SpecialAtk
第 4 個節點	HP、Speed

表：某棵樹的某個節點

重點是：每個節點都會重新隨機挑特徵。所以 Defense 可以在第 1 個節點出現，也可以在第 3 個節點再次出現，每個節點都會重新進行一次隨機特徵選擇。

隨機森林的優點

- 比單棵決策樹更穩定。
- 較不容易過度擬合。
- 可以處理非線性關係。
- 可以提供特徵重要性。
- 對資料中的雜訊較有容錯能力。

特徵重要性

隨機森林可以計算每個特徵對模型預測的重要程度。若某個特徵常被用來有效分裂節點，通常會有較高的重要性。特徵重要性可用於：

- 理解模型主要依據哪些特徵做判斷。
- 進行特徵選取。
- 刪除不重要的特徵，簡化模型。

K 最近鄰居

K 最近鄰居（K-Nearest Neighbors, KNN）是一種基於距離的分類方法。它不會像邏輯斯迴歸或決策樹一樣先學出明確的模型規則，而是在預測時尋找距離新資料最近的 K 筆訓練資料，再根據這些鄰居的類別進行預測。因此 KNN 也稱為：

- 非參數模型
- 延遲學習：訓練時只是把資料儲存起來，不做任何計算。所有繁重的計算（計算距離、找鄰居）都推遲到預測（測試）時才進行。

KNN 分類流程

- 給定一筆新資料。
- 計算新資料與所有訓練資料的距離。
- 找出距離最近的 K 個鄰居。
- 使用多數決決定新資料的類別。

歐幾里得距離

KNN 常用歐幾里得距離衡量兩筆資料的距離：

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{n} (x_j - z_j)^2}$$

其中

- X：第一筆資料的特徵向量
- Z：第二筆資料的特徵向量
- x_j：第一筆資料的第 j 個特徵
- z_j：第二筆資料的第 j 個特徵
- n：特徵數量

K 值是 KNN 中最重要的參數

- K 小：模型容易受雜訊影響，分類邊界較不穩定。
- K 大：模型較平滑，但可能忽略局部特徵。

例如：

- K=3：只看最近的 3 個鄰居。
- K=5：看最近的 5 個鄰居。

若多數鄰居屬於類別 1，則新資料預測為類別 1。

特徵標準化的重要性

因為 KNN 是距離導向模型，如果不同特徵尺度差異很大，距離會被數值範圍大的特徵主導。例如：

- HP 範圍可能是 1 到 255。
- Speed 範圍也可能不同。
- 若某特徵數值範圍較大，會對距離計算造成較大影響。

因此使用 KNN 前通常需要標準化：

$$z = \frac{x - \mu}{\sigma}$$

用在分類任務的 KNN

在 KNN 中，給定一個待分類的樣本，KNN 會計算該樣本與訓練集中所有樣本之間的距離（如歐幾里得距離），然後選取 K 個距離最近的鄰居。最後，根據這些鄰居的標籤進行分類，通常使用多數投票的方式來決定最終的分類結果。當票數相同時，scikit-learn 會選擇跟觀察值距離最近的樣本的類別；若有多個最近鄰樣本與觀察值等距，則選擇最先找到的樣本的類別。

KNN 的預測機率可由鄰居投票比例決定

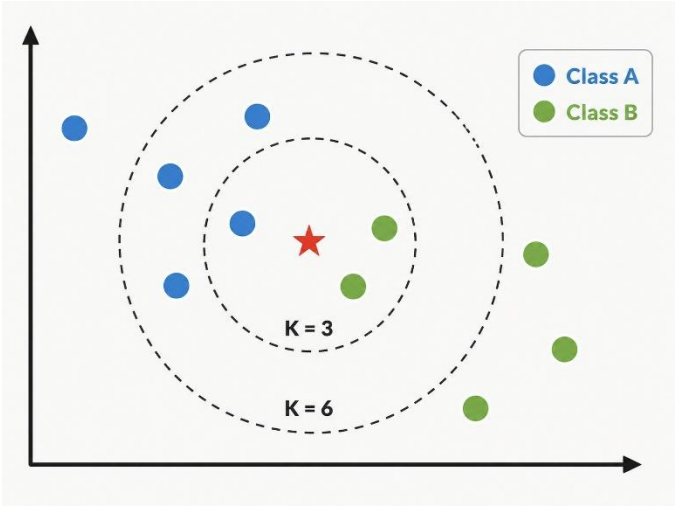
例如 K=5，其中 3 個鄰居是類別 0，2 個鄰居是類別 1，則：

$$P(y=0|\mathbf{x}) = \frac{3}{5} = 0.6$$

$$P(y=1|\mathbf{x}) = \frac{2}{5} = 0.4$$

因此模型會預測：

$$\hat{y} = 0$$



圖：在 K 的鄰居當中，選擇佔某類別數量最多的，來作為自己的類別

Radius Neighbors

除了指定最近的 K 個鄰居，也可以指定半徑 r。Radius Neighbors 會找出距離新資料在半徑 r 以內的所有訓練資料。若半徑內沒有任何鄰居，該資料可能被視為離群值。

Module 4. 模型擬合、評估與超參數調校

工作流程

機器學習工作流程是指從原始資料到模型預測結果的一連串步驟。常見流程如下：

- 讀取資料
- 建立目標欄位
- 選擇特徵與目標
- 切分訓練資料與測試資料
- 特徵選擇
- 特徵轉換
- 建立模型
- 訓練模型
- 評估模型表現

以寶可夢資料為例，如果目標是預測某隻寶可夢是否具有第二屬性，可以建立一個新的欄位 hasType：

- 若 Type2 欄位不是空值，表示該寶可夢具有第二屬性，hasType2 = 1。
- 若 Type2 欄位是空值，表示該寶可夢只有單一屬性，hasType2 = 0。

Name	Type1	Type2	hasType2
妙蛙種子	Grass	Poison	1
小火龍	Fire	空值	0
大王燕	Normal	Flying	1

表：經過特徵工程後的資料集

此時模型任務就是「根據 HP、Attack、Defense、SpecialAtk、SpecialDef、Speed 等數值特徵，預測寶可夢是否具有第二屬性。」

管道化

Pipeline 是將資料處理與模型訓練流程串接起來的工具。例如：

特徵選擇→標準化→模型訓練

可以被整合成一個完整流程。

優點	說明
流程清楚	每個步驟依序執行
避免資料洩漏	前處理會在交叉驗證中只根據訓練折計算
容易維護	可替換不同前處理或模型
方便調參	可與 GridSearchCV、RandomizedSearchCV 結合
提升重現性	減少手動操作錯誤

表：管道化的好處

ColumnTransformer 是用來針對不同欄位套用不同前處理方法的工具，在真實資料中，常常同時存在數值型特徵與類別型特徵。例如寶可夢資料中：

特徵類型	範例欄位	可使用方法
數值型特徵	HP、Attack、Defense	標準化、ANOVA 特徵選擇
類別型特徵	Generation、Type1	獨熱編碼、卡方特徵選擇

表：ColumnTransformer 可以讓不同欄位走不同處理流程，最後再把結果合併起來給模型使用。

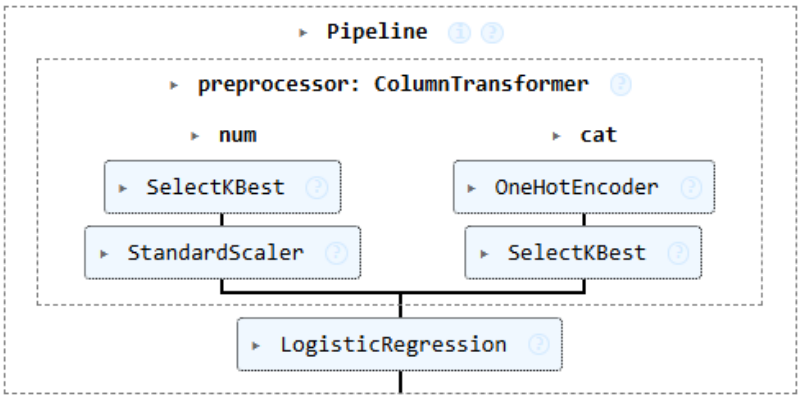
假設要同時使用：

- 數值特徵：HP 到 Speed
- 類別特徵：Generation

可以設計如下流程：

數值特徵→ANOVA 特徵選擇→標準化
類別特徵→獨熱編碼→卡方特徵選擇

最後將兩邊結果合併，再輸入邏輯斯迴歸模型。



圖：完整的 Pipeline 圖形結構

擬合

模型擬合（Fitting）

模型擬合是指模型從訓練資料中學習特徵與目標之間關係的過程：

- 模型擬合得太差，稱為欠擬合。
- 模型擬合得太好，甚至把訓練資料中的雜訊也學進去，稱為過擬合。
- 理想情況是模型能學到資料中的主要規律，並能對新資料做出穩定預測。

欠擬合（Underfitting）

欠擬合是指模型太簡單，無法有效學習資料中的規律。

現象	說明
訓練分數低	模型連訓練資料都學不好
驗證分數低	模型對新資料表現也不好
訓練分數與驗證分數都偏低	通常代表模型能力不足

表：欠擬合的現象

可能原因

- 特徵不足
- 特徵與目標關聯性弱
- 模型太簡單
- 正則化太強
- 資料本身可預測性有限

改善方法	說明
增加特徵	加入更多有用變數
建立組合特徵	例如從身高與體重建立 BMI
提升模型複雜度	例如增加樹深度、使用非線性模型
降低正則化強度	讓模型有更高彈性

表：欠擬合的改善方法

過擬合（Overfitting）

過擬合是指模型對訓練資料表現很好，但對新資料表現不好。

現象	說明
訓練分數高	模型把訓練資料學得很好
驗證分數低	模型無法泛化到新資料
訓練分數與驗證分數差距大	通常代表模型過度記住訓練資料

表：過擬合的現象

可能原因

- 模型太複雜
- 訓練資料太少
- 特徵太多且含有雜訊
- 正則化不足
- 超參數設定不佳

改善方法	說明
增加訓練資料	讓模型看到更多樣本
降低模型複雜度	例如限制決策樹深度
增加正則化強度	降低模型權重過大問題
移除無用特徵	降低雜訊影響
使用交叉驗證	更穩定評估泛化能力

表：過擬合的改善方法

評估模型效能

除了我們先前學到用在迴歸任務的 MSE/RMSE/R^2/殘差分析，還有用在分類任務上的 混淆矩陣/TP/TN/FP/FN/Precision/Recall/F1-score，以及討論模型複雜度高低所造成不同的偏誤（Bias）與變異（Variance）有待權衡（Tradeoff）外，曲線相關的評估指標，同樣可以用在模型效能的評估上。

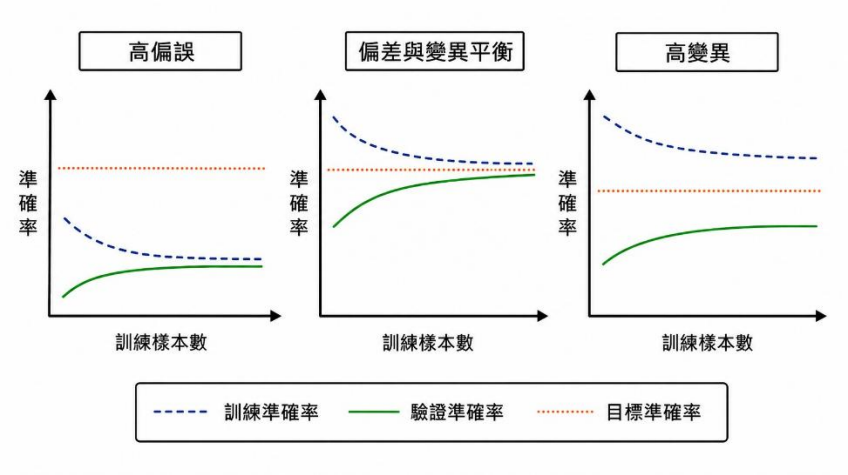
學習曲線與驗證曲線

學習曲線是觀察不同訓練資料量下，模型在訓練資料與驗證資料上的表現變

化。學習曲線的橫軸是訓練樣本數量，縱軸是模型分數，通常會畫出兩條曲線：

曲線	說明
Training score	模型在訓練資料上的分數
Validation score	模型在驗證資料上的分數

表：訓練分數與驗證分數



圖：三種學習曲線的類型

若使用 K 次交叉驗證，在某個訓練資料量 m 下，平均訓練分數為：

$$\bar{s}_{train}(m) = \frac{1}{K}\sum_{k=1}^{K} s_{train}^{(k)}(m)$$

平均驗證分數為：

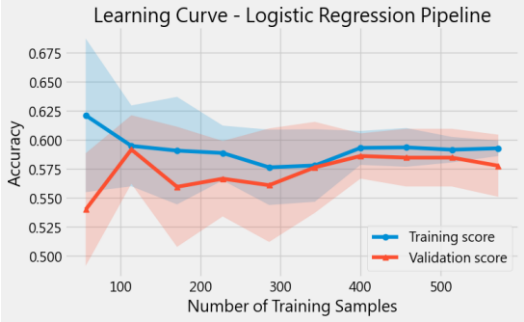
$$\bar{s}_{valid}(m) = \frac{1}{K}\sum_{k=1}^{K} s_{valid}^{(k)}(m)$$

其中

- m：訓練樣本數
- K：交叉驗證次數
- $s_{train}^{(k)}(m)$：第 k 折中，使用 m 筆資料訓練後的訓練分數
- $s_{valid}^{(k)}(m)$：第 k 折中，使用 m 筆資料訓練後的驗證分數
- $\bar{s}_{train}(m)$：平均訓練分數
- $\bar{s}_{valid}(m)$：平均驗證分數

曲線情況	可能問題	解釋
訓練分數高，驗證分數低	過擬合	模型記住訓練資料，但泛化能力差
訓練分數低，驗證分數也低	欠擬合	模型能力不足
兩條曲線逐漸靠近且分數高	理想狀態	模型穩定且泛化能力佳
增加資料後驗證分數持續上升	更多資料可能有幫助	模型仍可從更多資料中學習

表：如何判讀學習曲線



圖：兩個分數後期非常接近，沒有過擬合的問題，但兩者分數都不高，可能有欠擬合的問題。

驗證曲線是觀察不同超參數設定下，模型在訓練資料與驗證資料上的表現變化。學習曲線的橫軸是「訓練資料量」，驗證曲線的橫軸是「超參數值」。例如在邏輯斯迴歸中，可以觀察不同 C 值對模型表現的影響。

在不同超參數值 θ 下，平均驗證分數為：

$$\bar{s}_{valid}(\theta) = \frac{1}{K}\sum_{k=1}^{K} s_{valid}^{(k)}(\theta)$$

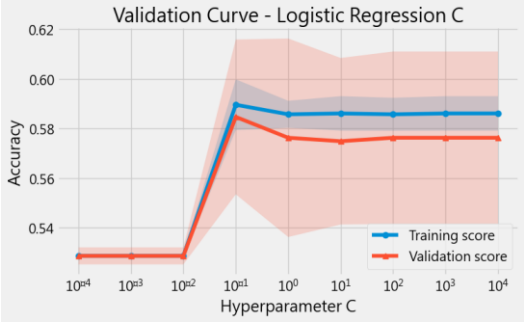
其中

- θ：某個超參數值
- K：交叉驗證次數
- $s_{valid}^{(k)}(\theta)$：第 k 折中，超參數為 θ 時的驗證分數
- $\bar{s}_{valid}(\theta)$：平均驗證分數

例如，若要觀察 LogisticRegression 的 C 值，可設定：

$$C \in \{0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000\}$$

若 C 太小，正則化太強，模型可能欠擬合。
若 C 太大，正則化太弱，模型可能過擬合。



圖：當 C 繼續變大時，驗證分數沒有繼續提升，這代表讓模型更複雜，並沒有帶來更好的泛化能力。

P-R 曲線與 ROC 曲線

P-R 曲線是 Precision-Recall Curve，也就是精確率與召回率曲線。它用來觀察在不同分類門檻下，precision 與 recall 之間的取捨。二元分類模型通常會輸出某筆資料屬於正類的分數或機率。

- 若門檻設定較高，模型會更保守地預測正類，precision 可能提高，但 recall 可能下降。
- 若門檻設定較低，模型會更積極地預測正類，recall 可能提高，但 precision 可能下降。
- P-R 曲線特別適合類別不平衡問題：正類很少、資料不平衡時更有參考價值。

$$Precision = \frac{TP}{TP + FP} \qquad Recall = \frac{TP}{TP + FN}$$

Average Precision：

$$AP = \sum_{n} (R_n - R_{n-1}) P_n$$

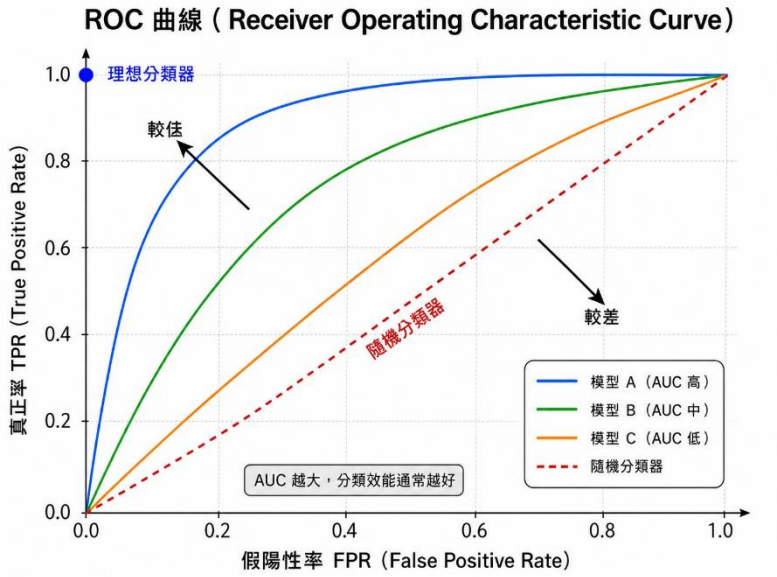
其中

- P_n：第 n 個門檻下的 precision
- R_n：第 n 個門檻下的 recall
- AP：Average Precision，P-R 曲線下的整體表現摘要

任務	原因
疾病偵測	正類很少，但漏判代價高
詐騙偵測	詐騙交易通常是少數
稀有屬性預測	Fire 屬性寶可夢相對較少
異常偵測	異常事件通常遠少於正常事件

表：適合使用 P-R 曲線的情況

ROC 曲線是 Receiver Operating Characteristic Curve，用來觀察不同分類門檻下，TPR 與 FPR 之間的關係；正負類比較平衡時常用。



圖：ROC 曲線是用來觀察不同分類門檻下，TPR 與 FPR 之間的關係。

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

註：TPR 也就是 Recall

ROC 曲線的橫軸是 FPR，縱軸是 TPR。理想模型希望 $TPR \to 1$、$FPR \to 0$，因此 ROC 曲線越靠近左上角，模型表現越好。

符號	意義
TPR	True Positive Rate，真陽性率

FPR	False Positive Rate，偽陽性率
TP	真陽性
FP	偽陽性
FN	偽陰性
TN	真陰性

表：分類任務相關名詞回顧

AUC 是 Area Under Curve，指 ROC 曲線下的面積，AUC 介於 0 到 1 之間。AUC 可以解釋為：隨機抽出一筆正類資料與一筆負類資料時，模型給正類資料的分數高於負類資料的機率。例如：在寶可夢範例中，ROC-AUC 約為 0.65，表示模型有一定區分能力，但並不是非常強，這也呼應前面學習曲線的觀察：模型可能仍有欠擬合或特徵不足的問題。

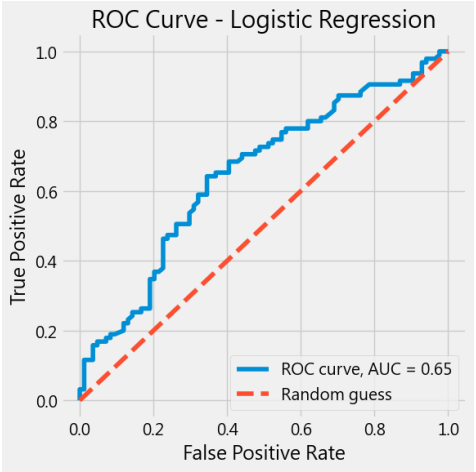
$$AUC = P(s(x^+) > s(x^-))$$

其中

- x^+：正類樣本
- x^-：負類樣本
- $s(x)$：模型給樣本 x 的分數
- $P(s(x^+) > s(x^-))$：正類分數高於負類分數的機率

AUC 值	**解釋**
1.0	完美分類
0.5 到 1.0	優於隨機猜測
0.5	接近隨機猜測
小於 0.5	比隨機猜測還差

表：AUC 值的解釋



圖：ROC Curve

調校超參數

超參數（Hyperparameters）

超參數是在模型訓練前就要設定的參數，不是模型從資料中自動學到的參數。

類型	說明	範例
模型參數	模型從資料中學到	邏輯斯迴歸的權重 (w)、截距 (b)
超參數	訓練前由使用者設定	C、penalty、max_depth、n_estimators

表：模型參數與超參數不同

模型	超參數	說明
LogisticRegression	C	正則化強度的倒數
LogisticRegression	penalty	使用 L1 或 L2 正則化
RandomForestClassifier	n_estimators	決策樹數量
RandomForestClassifier	max_depth	每棵樹最大深度
RandomForestClassifier	max_features	每次切分可使用的特徵數
SVC	C	錯誤懲罰程度
SVC	gamma	RBF 核函數影響範圍
SVC	kernel	核函數類型

表：常見超參數範例

一般手動測試超參數，最直覺的方式是手動用迴圈測試不同超參數組合，

例如：

$$penalty \in \{l1, l2\}$$
$$C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$$

逐一訓練模型並比較分數。

問題	說明
容易過度依賴測試資料	如果一直用測試資料挑參數，測試資料就不再客觀
不容易擴充	參數一多，迴圈會很複雜
不一定搭配交叉驗證	分數可能受單次切分影響
不方便管理結果	難以完整記錄所有組合

表：手動設定超參數的問題

網格搜尋與隨機化搜尋

網格搜尋（Grid Search）

網格搜尋是將所有指定的超參數組合全部測試一遍，並透過交叉驗證找出表現最佳的組合。若有 p 個超參數，每個超參數的候選值數量分別為：

$$|V_1|, |V_2|, \ldots, |V_p|$$

則總組合數為：

$$N_{\text{comb}} = \prod_{j=1}^{p} |V_j|$$

其中

- p：超參數數量
- V_j：第 j 個超參數的候選值集合
- N_{comb}：總共需要測試的組合數

例如，若 LogisticRegression 有：

$$penalty \in \{l1, l2\}$$
$$C \in \{0.0001, 0.001, 0.01, 0.1, 1, 10\}$$

則組合數為 $2 \times 6 = 12$，若使用 5-fold cross-validation，總訓練次數為

$12 \times 5 = 60$。

優點	說明
系統化	所有指定組合都會測試
結果穩定	可搭配交叉驗證
適合小型參數空間	參數組合不多時很好用

表：網格搜尋的優點

缺點	說明
計算成本高	參數組合多時非常耗時
不適合連續範圍很大的參數	只能測試指定值
可能浪費時間	很多組合可能明顯不好

表：網格搜尋的缺點

隨機化搜尋（Randomized Search）

隨機化搜尋是從指定的超參數範圍或分布中，隨機抽取固定數量的組合進行測試。若超參數向量為 θ，隨機化搜尋可表示為：

$$\theta^{(t)} \sim p(\theta), \quad t = 1, 2, \ldots, T$$

其中

- θ：一組超參數
- $p^{(\theta)}$：超參數的抽樣分布
- T：隨機抽樣次數
- $\theta^{(t)}$：第 t 次抽出的超參數組合

優點	說明
計算效率較高	不需要測試所有組合
適合大範圍搜尋	尤其是連續超參數
有機會快速找到好解	隨機探索範圍較廣

表：隨機化搜尋的優點

缺點	說明
結果受隨機性影響	不同 random_state 可能結果不同
不保證測到所有重要組合	可能錯過最佳區域
需要設定抽樣次數	n_iter 太小可能搜尋不足

表：隨機化搜尋的缺點

參數搜尋小技巧

1. 先粗搜，再細搜

超參數調整可以分成兩階段：

- 粗略搜尋大範圍
- 在表現較好的區域進行更細緻搜尋

例如先搜尋：

$$C \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$$

若發現 C=0.001 附近表現較好，再細搜：

$$C \in \{0.0005, 0.001, 0.002, 0.005\}$$

2. 使用對數尺度搜

許多超參數的影響不是線性的，而是跨數量級變化。例如 C、γ、learning rate 等，常用對數尺度搜尋。與其測試：

$$C \in \{1, 2, 3, 4, 5\}$$

更常見的是測試：

$$C \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$$

這樣可以更有效涵蓋不同正則化強度。

3. 根據模型特性調整超參數

不同模型有不同重要超參數，不應該盲目調整。

模型	重要超參數	調整重點
LogisticRegression	C、penalty	控制正則化
RandomForestClassifier	n_estimators、max_depth、max_features	控制樹數量與複雜度
SVC	C、gamma、kernel	控制分類邊界與非線性程度
KNN	n_neighbors	控制鄰居數量
DecisionTreeClassifier	max_depth、min_samples_split	控制樹的複雜度

表：常見模型與調整方向

4. 使用模型內建的 CV 類別

有些模型提供內建交叉驗證的版本，可以自動尋找某些超參數。例如 LogisticRegressionCV 可以自動搜尋 C 值：

$$Cs = 100$$

表示在預設範圍內產生 100 個候選 C 值，並透過交叉驗證選出較佳設定。

優點	說明
使用方便	不需要自己寫完整 GridSearchCV
適合特定模型	對模型本身有較好的整合
程式較簡潔	適合快速實驗

表：使用模型內建 CV 的優點

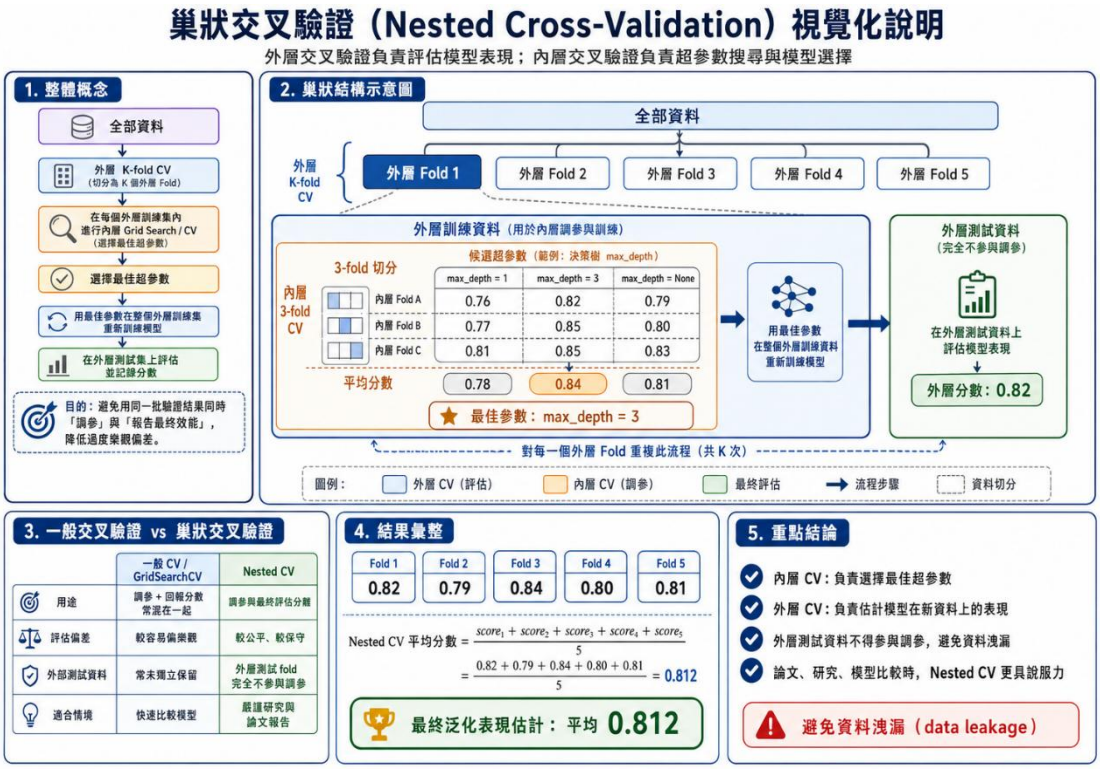
5. 巢狀交叉驗證（Nested Cross-Validation）

巢狀交叉驗證是在交叉驗證內再加入另一層交叉驗證，外層交叉驗證用來評估模型泛化能力，內層交叉驗證用來選擇最佳超參數。一般 GridSearchCV 會在訓練資料中選出最佳參數，但如果又用同一批資料估計模型表現，可能會過度樂觀；巢狀交叉驗證可以更嚴謹地估計模型在未知資料上的表現。

外層CV：評估模型表現

內層CV：選擇超參數

如果從事研究或是撰寫專題報告，需要較嚴謹的模型比較，例如比較 LogisticRegression、RandomForest、SVC 哪一種模型比較好，可以使用巢狀交叉驗證。



圖：巢狀交叉驗證的視覺化說明

6. 以評估指標為依據來調整超參數

超參數搜尋時，不一定要使用 accuracy 作為評估指標，應根據任務目的選擇指標。

任務目的	建議指標
類別平均且重視整體正確	accuracy
正類很少且重視排序能力	roc_auc
正類很少且重視找出正類	recall
正類很少且重視 precision / recall 平衡	f1
重視 P-R 曲線整體表現	average_precision

表：不同任務目的，應選擇不同評估指標

處理類別不平衡

1. 加入 class_weight 參數

class_weight 是在模型訓練時，對不同類別給予不同權重；少數類別可以給較高權重，讓模型更重視少數類別的錯誤。scikit-learn 中

`class_weight='balanced'` 的常見概念為：

$$w_c = \frac{N}{K \times n_c}$$

其中

- w_c：類別 c 的權重
- N：總資料筆數
- K：類別數量
- n_c：類別 c 的資料筆數

若某類別資料越少，n_c 越小，則該類別權重 w_c 越大，這會讓模型在訓練時更重視少數類別。

2. 上採樣（Oversampling）

上採樣是增加少數類別樣本數的方法。常見做法是對少數類別進行放回式重複取樣，使少數類別的資料筆數接近多數類別。

類別	筆數
非 Fire	663
Fire	52 → 上採樣後 → 663

表：對訓練資料類別較少的資料進行上採樣

優點	說明
保留多數類別資料	不會刪除原本資料
簡單直覺	容易實作
可提升少數類別辨識能力	模型會看到更多少數類別樣本

表：上採樣的優點

缺點	說明
可能過擬合少數類別	因為是重複抽樣
訓練資料變多	計算時間可能增加
沒有創造真正新資訊	只是重複既有樣本

表：上採樣的缺點

方法	說明
Random Oversampling	隨機重複抽取少數類別資料
SMOTE	根據少數類別樣本產生新的合成資料

ADASYN	類似 SMOTE，但更重視難分類的少數類別樣本

表：常見 oversampling 方法

3. 下採樣（Undersampling）

下採樣是減少多數類別樣本數的方法，使多數類別與少數類別數量接近。

類別	筆數
非 Fire	663 → 下採樣後 → 52
Fire	52

表：對訓練資料類別較多的資料進行下採樣

優點	說明
資料量變少	訓練速度較快
類別比例平衡	模型不會過度偏向多數類別

表：下採樣的優點

缺點	說明
會丟失資料	多數類別資訊被移除
結果可能不穩定	抽到不同樣本可能影響模型
不適合資料本來就少的情況	可能造成訓練資料不足

表：下採樣的缺點

方法	說明
Random Undersampling	隨機移除多數類別資料
Tomek Links	移除靠近分類邊界、容易混淆的樣本
Edited Nearest Neighbors	移除與鄰近樣本類別不一致的資料
NearMiss	根據與少數類別的距離挑選多數類別資料

表：常見 undersampling 方法

4. 類別不平衡時的模型評估策略

處理類別不平衡時，不只是修改模型，也要改變評估方式。建議的流程如下：

- 先檢查類別分布
- 建立 baseline 模型
- 查看 confusion matrix 與 classification report
- 觀察少數類別的 precision、recall、F1-score

- 嘗試 class_weight='balanced'
- 嘗試上採樣或下採樣
- 使用合適評估指標進行超參數調校
- 最後再用測試資料評估最終模型

例如在 Fire 屬性預測中，若未處理類別不平衡，模型可能完全不預測 Fire 類別，則可以加入 class_weight='balanced' 或使用上採樣後，模型可能會開始預測 Fire 類別。

雖然整體 accuracy 可能下降，但少數類別 recall 可能提高。這代表模型不再只是迎合多數類別，而是開始嘗試辨識少數類別。

Module 5. 非監督式學習：降維與分群

非監督式學習（Unsupervised Learning）是一種不需要預先標記答案的機器學習方法。與監督式學習不同，非監督式學習的資料通常只有特徵，沒有明確的標籤或正確答案。模型的任務是從資料中自動找出潛在結構，例如資料是否可以被分成幾群、哪些樣本彼此相似、哪些特徵可以被濃縮成更少的代表性變數。

主成分分析

主成分分析（Principal Component Analysis, PCA）是一種常見的線性降維方法。它的核心想法是把原始資料中的多個特徵，轉換成一組新的座標軸，這些新座標軸稱為主成分（Principal Components, PCs）。

主成分具有以下特性：

- 主成分彼此正交，也就是彼此垂直。
- 第一個主成分會保留最多的資料變異。
- 第二個主成分會在與第一個主成分垂直的條件下，保留剩下資料中最多的變異。
- 後續主成分依此類推。
- 主成分是原始特徵的線性組合。

換句話說，PCA 並不是直接挑選原始資料中的某幾個欄位，而是把原始欄位重新加權組合，產生新的特徵軸。

假設原始資料有 6 個能力值特徵：HP, Attack, Defense, SpecialAtk, SpecialDef, Speed，如果直接觀察 6 維資料，視覺化與解釋都比較困難。PCA 可以把這 6 個特徵轉換成較少的主成分，例如 2 個主成分：PC1, PC2，如此一來，每一隻寶可夢都可以被投影到二維平面上，方便觀察樣本之間的分布、相似性與群聚現象。

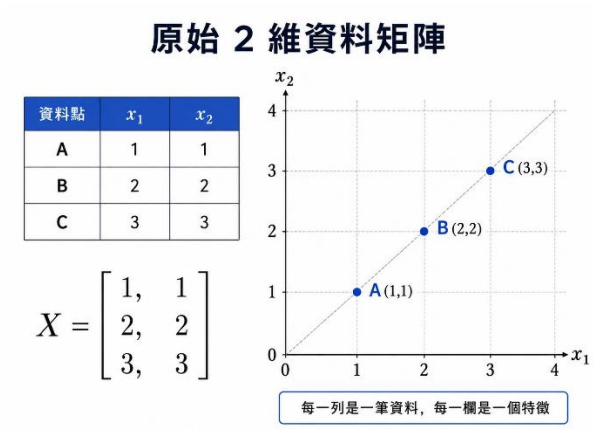
PCA 特別適合用在以下情境：

- 特徵數量很多，希望降低資料維度。
- 特徵之間存在相關性，希望移除冗餘資訊。
- 想把高維資料視覺化到 2 維或 3 維。
- 想在保留主要資訊的同時，減少模型訓練的計算成本。
- 想降低雜訊對後續分析的影響。

PCA 之前需要先標準化

PCA 依賴變異數與距離的概念。如果某些特徵的數值範圍特別大，這些特徵會主導 PCA 的結果。例如在寶可夢資料中，假設某個特徵的數值範圍遠大於其他特徵，PCA 可能會誤以為這個特徵比較重要。

為了避免尺度差異造成偏誤，通常會先做標準化，標準化後，每個特徵的平均值約為 0，標準差約為 1。這樣可以讓不同欄位在 PCA 中有較公平的比較基礎。



圖：原始的 2 維資料矩陣

步驟	計算過程說明	算式
1	建立原始 2 維資料矩陣	$X = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \end{bmatrix}$
2	計算第 1 個特徵 x_1 的平均值	$\bar{x_1} = \frac{1+2+3}{3} = \frac{6}{3} = 2$
3	計算第 2 個特徵 x_2 的平均值	$\bar{x_2} = \frac{1+2+3}{3} = \frac{6}{3} = 2$
4	得到平均向量	$\bar{x} = [2 \quad 2]$
5	將 A 點中心化	$(1,1) - (2,2) = (-1,-1)$
6	將 B 點中心化	$(2,2) - (2,2) = (0,0)$
7	將 C 點中心化	$(3,3) - (2,2) = (1,1)$
8	得到中心化後的資料矩陣	$X_c = \begin{bmatrix} -1 & -1 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$
9	寫出共變異數矩陣公式	$S = \frac{1}{n-1} X_c^T X_c$

10	代入資料筆數 $n=3$	$S=\frac{1}{3-1}X_c^TX_c=\frac{1}{2}X_c^TX_c$
11	計算 X_c 的轉置矩陣	$X_c^T=\begin{bmatrix}-1 & 0 & 1\\ -1 & 0 & 1\end{bmatrix}$
12	計算 $X_c^TX_c$	$X_c^TX_c=\begin{bmatrix}-1 & 0 & 1\\ -1 & 0 & 1\end{bmatrix}\begin{bmatrix}-1 & -1\\ 0 & 0\\ 1 & 1\end{bmatrix}$
13	逐格計算乘法結果	$X_c^TX_c=\begin{bmatrix}2 & 2\\ 2 & 2\end{bmatrix}$
14	得到共變異數矩陣	$S=\frac{1}{2}\begin{bmatrix}2 & 2\\ 2 & 2\end{bmatrix}=\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$
15	寫出特徵值方程式	$\det(\mathrm{S}-\lambda\mathrm{I})=0$
16	寫出單位矩陣	$I=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$
17	計算 λI	$\lambda I=\lambda\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}=\begin{bmatrix}\lambda & 0\\ 0 & \lambda\end{bmatrix}$
18	計算 $S-\lambda I$	$S-\lambda I=\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}-\begin{bmatrix}\lambda & 0\\ 0 & \lambda\end{bmatrix}=\begin{bmatrix}1-\lambda & 1\\ 1 & 1-\lambda\end{bmatrix}$
19	計算行列式	$\det(\mathrm{S}-\lambda\mathrm{I})=(1-\lambda)(1-\lambda)-1\cdot 1=(1-\lambda)^2-1$
20	展開行列式	$(1-\lambda)^2-1=1-2\lambda+\lambda^2-1$
21	整理方程式	$\lambda^2-2\lambda=0$
22	因式分解	$\lambda(\lambda-2)=0$
23	得到兩個特徵值	$\lambda=0$ 或 $\lambda=2$
24	選擇最大特徵值作為第一主成分	$\lambda_1=2$
25	設定特徵向量	$v=\begin{bmatrix}a\\ b\end{bmatrix}$
26	代入特徵向量方程式 （對共變異數矩陣做特徵分解）	$Sv=2v$
27	展開矩陣乘法	$\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\begin{bmatrix}a\\ b\end{bmatrix}=2\begin{bmatrix}a\\ b\end{bmatrix}$
28	計算左右兩邊	$\begin{bmatrix}a+b\\ a+b\end{bmatrix}=2\begin{bmatrix}a\\ b\end{bmatrix}$
29	得到兩個方程式	$a+b=2a,\quad a+b=2b$
30	解出特徵向量的關係	$b=a,\quad a=b\Rightarrow a=b$
31	得到主成分方向的一般形式	$v=\begin{bmatrix}a\\ a\end{bmatrix}=a\begin{bmatrix}1\\ 1\end{bmatrix}$

32	選擇同方向的簡單向量	$v = \begin{bmatrix}1\\1\end{bmatrix}$
33	計算向量長度	$\left\|\begin{bmatrix}1\\1\end{bmatrix}\right\| = \sqrt{1^2+1^2} = \sqrt{2}$
34	正規化成單位向量	單位向量 $= \frac{\text{原向量}}{\text{原向量長度}} \rightarrow v_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$
35	得到第一個主成分方向 （第一個特徵向量）	$v_1 = \begin{bmatrix}\frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{2}}\end{bmatrix} \approx \begin{bmatrix}0.707\\0.707\end{bmatrix}$
36	寫出 PCA 投影公式	$Z = X_c v_1$
37	代入中心化資料 與 第一個主成分方向	$Z = \begin{bmatrix}-1 & -1\\ 0 & 0\\ 1 & 1\end{bmatrix}\begin{bmatrix}\frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{2}}\end{bmatrix}$
38	計算 A 點降維結果 （A 點資料的 PC score）	$z_A = (-1)\frac{1}{\sqrt{2}} + (-1)\frac{1}{\sqrt{2}} = -\frac{2}{\sqrt{2}} = -\sqrt{2} \approx -1.414$
39	計算 B 點降維結果 （B 點資料的 PC score）	$z_B = 0\cdot\frac{1}{\sqrt{2}} + 0\cdot\frac{1}{\sqrt{2}} = 0$
40	計算 C 點降維結果 （C 點資料的 PC score）	$z_C = 1\cdot\frac{1}{\sqrt{2}} + 1\cdot\frac{1}{\sqrt{2}} = \frac{2}{\sqrt{2}} = \frac{2\cdot\sqrt{2}}{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2} \approx 1.414$
41	得到 PCA 降到 1 維後的結果	$Z = \begin{bmatrix}-\sqrt{2}\\0\\\sqrt{2}\end{bmatrix} \approx \begin{bmatrix}-1.414\\0\\1.414\end{bmatrix}$
42	計算第一主成分解釋變異比例	$\frac{\lambda_1}{\lambda_1+\lambda_2} = \frac{2}{2+0} = 1 = 100\%$

表：PCA 的計算過程

提取主成分

是指從原始特徵中找出新的方向，使資料投影到這些方向後能保留最多變異。這些新的方向就是主成分。在 PCA 中，每個主成分可以看成一組權重。這組權重會告訴我們原始特徵對該主成分的貢獻程度。

例如，某個主成分可以表示為：

$$PC1 = w_1 HP + w_2 Attack + w_3 Defense + w_4 SpecialAtk + w_5 SpecialDef + w_6 Speed$$

其中

- $PC1$：第一主成分。
- $w_1, w_2, w_3, \ldots, w_6$：各個原始特徵在第一主成分中的權重。
- 權重的絕對值越大，代表該特徵對該主成分的影響越大。

在寶可夢資料集中，可以使用以下 6 個能力值做 PCA：

HP, Attack, Defense, SpecialAtk, SpecialDef, Speed

經過標準化與 PCA 後，可以得到每個主成分對原始欄位的權重，例如教材範例中，前兩個主成分大致呈現如下概念：

- PC1：多數能力值權重皆為正，代表它比較像「整體能力強弱」的方向。
- PC2：Speed 權重較高，而 Defense 權重較低，代表它可能反映「速度高、防禦相對低」的特性。

因此，一隻寶可夢如果在 PC2 上分數很高，可以解釋為它比較偏向速度型；如果在 PC1 上分數很高，則可能代表整體能力值較強。

解釋變異數

解釋變異數（Explained Variance）用來表示每一個主成分可以保留多少原始資料的變異；解釋變異比例（Explained Variance Ratio）則是指某個主成分解釋的變異量占全部變異量的比例。如果某個主成分的解釋變異比例越高，代表該主成分越能代表原始資料的重要資訊。

第 j 個主成分的解釋變異比例定義為：

$$r_j = \frac{\lambda_j}{\sum_{\ell=1}^{p} \lambda_\ell}$$

其中

- r_j：第 j 個主成分的解釋變異比例。
- λ_j：第 j 個主成分的特徵值。
- p：原始特徵數量。
- $\sum_{\ell=1}^{p} \lambda_\ell$：所有主成分的總變異量。

前 m 個主成分的累積解釋變異比例為：

$$R_m = \sum_{j=1}^{m} r_j$$

其中

- R_m：前 m 個主成分累積保留的變異比例。
- m：選擇保留的主成分數量。

選擇主成分數量，常見做法是觀察累積解釋變異比例，例如：

$$R_m \geq 0.8$$

代表前 m 個主成分已經保留至少 80% 的資料變異。

在實務上，常見選擇標準包括：

- 保留累積解釋變異比例達到 80% 或 90% 的主成分。
- 觀察 scree plot，找出解釋變異下降趨緩的位置。
- 根據視覺化需求，選擇 2 或 3 個主成分。
- 根據後續模型效能決定保留幾個主成分。

例如在寶可夢資料集中，使用 6 個能力值做 PCA 時，前 4 個主成分的解釋變異比例約為：

主成分	解釋變異比例	累積解釋變異比例
PC1	0.4497	0.4497
PC2	0.1872	0.6370
PC3	0.1310	0.7680
PC4	0.1186	0.8866

這表示：

- 只使用 PC1，可保留約 44.97% 的變異。
- 使用 PC1+PC2，可保留約 63.70% 的變異。
- 使用 PC1+PC2+PC3+PC4，可保留約 88.66% 的變異。

如果視覺化目標是畫二維圖，可以選擇前 2 個主成分；如果目標是保留至少 80% 的變異，則可能需要選擇前 4 個主成分。

核主成分分析處理非線性對應

核主成分分析（Kernel PCA, KPCA）是 PCA 的非線性延伸。傳統 PCA 只能處理線性結構，也就是只能找出線性方向上的主要變異。當資料本身呈現非線性分布時，傳統 PCA 的效果可能不好。

Kernel PCA 的核心想法是：先透過一個非線性映射函數，把原始資料映射到更高維的特徵空間，再在該空間中做 PCA。

假設原始資料為 x，Kernel PCA 使用映射函數，把資料映射到高維空間：

$$\phi(x)$$

在高維空間中，兩筆資料的內積為：

$$\phi(x_i)^T \phi(x_j)$$

但實務上通常不會直接計算 $\phi(x)$，而是使用核函數（Kernel Function）：

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

其中

- $k(x_i, x_j)$：第 i 筆與第 j 筆資料的核函數值。
- $\phi(x_i)$：第 i 筆資料映射到高維空間後的表示。
- $\phi(x_j)$：第 j 筆資料映射到高維空間後的表示。

常見的 RBF 核函數為：

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

其中

- γ：控制影響範圍的參數。
- $\|x_i - x_j\|^2$：兩筆資料之間的平方歐氏距離。
- $\exp(\cdot)$：指數函數。

當 γ 較大時，模型對局部差異較敏感；當 γ 較小時，資料點之間的相似性變化較平滑。

Kernel PCA 適合用於資料具有非線性結構的情境。例如：

- 資料呈現圓形、螺旋形、月牙形等非線性分布。
- 線性 PCA 無法有效分離資料。
- 希望保留資料中的非線性關係。
- 在分群或分類前，先做非線性特徵轉換。

需要注意的是，Kernel PCA 的計算成本通常比 PCA 高，因為它需要計算樣本之間的核矩陣。當樣本數很大時，計算時間與記憶體需求會明顯增加。

K-means

基本概念

分群（Clustering）是非監督式學習中的一種重要任務。它的目標是根據資料之間的相似度，把樣本分成不同群組，使得：

- 同一群內的資料彼此相似。
- 不同群之間的資料彼此差異較大。

建立 k-means 分群

K-means 是最常見的分群演算法之一。它會把資料分成 K 個群，並為每一群找出一個中心點，稱為質心（Centroid）。K-means 的目標是最小化每一筆資料到所屬群質心的距離平方和。

$$J = \sum_{i=1}^{n} \sum_{k=1}^{K} r_{ik} \|x_i - \mu_k\|^2$$

其中

- J：K-means 要最小化的目標函數。
- n：樣本數。
- K：分群數量。
- x_i：第 i 筆資料。
- μ_k：第 k 群的質心。
- r_{ik}：指示變數，如果第 i 筆資料屬於第 k 群，則 $r_{ik} = 1$ 否則 $r_{ik} = 0$。
- $\|x_i - \mu_k\|^2$：資料點與質心之間的平方距離。

歐氏距離

K-means 通常使用歐氏距離衡量資料點之間的差異。兩個資料點 x_i 與x_j 的歐氏距離為：

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^{p} (x_{im} - x_{jm})^2}$$

其中

- $d(x_i, x_j)$：兩筆資料之間的距離。
- p：特徵數量。

- x_{im}：第 i 筆資料在第 m 個特徵上的值。
- x_{jm}：第 j 筆資料在第 m 個特徵上的值。

K-means 的基本流程如下：

- 指定分群數量 K。
- 隨機選擇 K 個初始質心。
- 計算每一筆資料到各個質心的距離。
- 將每一筆資料分配給距離最近的質心。
- 根據分群結果重新計算每一群的質心。
- 重複步驟 3 到步驟 5，直到質心不再明顯變動，或達到最大迭代次數。

質心更新公式為：

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

其中

- μ_k：第 k 群的質心。
- C_k：第 k 群中的樣本集合。
- $|C_k|$：第 k 群的樣本數。
- x_i：屬於第 k 群的第 i 筆資料。

K-means 的重要參數

- K：分群數量

  K-means 需要事先指定分成幾群。這是 K-means 的主要限制之一，因為在真實資料中，通常不一定知道最佳群數。

- 初始質心

  初始質心的位置會影響最後的分群結果。如果初始位置不佳，模型可能收斂到較差的局部解。常見初始化方法包括：

  - random：隨機選擇初始中心點。
  - k-means++：用較聰明的方式初始化質心，通常能加速收斂並降低壞結果機率。

- 距離度量

  K-means 通常使用歐氏距離，因此特別適合球狀、大小相近的群集。如果資料群集呈現不規則形狀，K-means 可能表現不佳。

評估分群結果

分群是非監督式學習，因此通常沒有標準答案可以直接比較。為了判斷分群結果是否合理，常見做法包括觀察 Distortion、Elbow Method、Silhouette Score 等指標。

Distortion / SSE

Distortion 通常指每個樣本到所屬群質心的距離平方和，也稱為 SSE（Sum of Squared Errors）或 inertia。

$$SSE = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

其中

- SSE：群內平方誤差總和。
- K：分群數量。
- C_k：第 k 群的樣本集合。
- x_i：第 i 筆資料。
- μ_k：第 k 群的質心。

SSE 越小，代表樣本越接近自己的群中心，群內越緊密。當 K 增加時，SSE 通常會下降。這是因為群數越多，每一群可以更精細地貼近資料，但是，不能單純認為 SSE 越小越好。因為當 K 等於樣本數時，每個樣本自己成為一群，SSE 會接近 0，但這樣的分群沒有實用意義。因此，SSE 通常搭配 Elbow Method 使用。

Elbow Method 又稱肘部法，是用來選擇合適 K 值的方法。它會觀察不同 K 值下的 SSE 變化，找出 SSE 下降幅度開始變小的位置，這個位置通常看起來像手肘，因此稱為 Elbow。

Silhouette Score

又稱輪廓係數，是用來評估分群品質的指標。它同時考慮：

- 樣本與同群內其他樣本的距離。
- 樣本與最近其他群的距離。

Silhouette Score 的範圍為：

$$-1 \le s(i) \le 1$$

其中

- 越接近 1：代表分群效果越好。
- 接近 0：代表樣本位於兩群邊界附近。
- 小於 0：代表樣本可能被分到錯誤的群。

輪廓係數為：

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

其中

- $a(i)$：為第 i 筆資料與同一群內其他資料的平均距離，越小越好。
- $b(i)$：為第 i 筆資料與最近其他群內資料的平均距離，越大越好。
- $s(i)$：第 i 筆資料的輪廓係數。

所有樣本的平均輪廓係數為：

$$S = \frac{1}{n} \sum_{i=1}^{n} s(i)$$

其中

- S：整體分群的平均輪廓係數。
- n：樣本數。

階層式分群

階層式分群（Hierarchical Clustering）是一種建立群集層級結構的分群方法。它不只給出一組固定的分群結果，也可以呈現樣本之間由近到遠的合併過程。階層式分群通常會搭配樹狀圖（Dendrogram）進行視覺化。樹狀圖可以幫助我們觀察哪些樣本最相似，以及在不同切割高度下可以得到幾個群。在這裡以凝聚式階層分群（Agglomerative Hierarchical Clustering）為例，它是一種由下而上的方法，流程如下：

- 一開始把每一筆資料都視為獨立的一群。
- 計算所有群集之間的距離。
- 找出距離最近的兩群，將它們合併。
- 重複合併，直到所有資料合併成一個群，或達到指定群數。

群集之間的距離計算方法

- Single Linkage
  - 使用兩個群集中最近兩個樣本的距離作為群集距離。

$$D(A,B) = \min_{x_i \in A, x_j \in B} d(x_i, x_j)$$

- 其中
  - A, B：兩個群集。
  - x_i, x_j：分別屬於群集 A 與 B 的樣本。
  - $d(x_i, x_j)$：兩個樣本之間的距離。
- 容易形成長條狀或鏈狀群集。

- Complete Linkage
  - 使用兩個群集中最遠兩個樣本的距離作為群集距離。

$$D(A,B) = \max_{x_i \in A, x_j \in B} d(x_i, x_j)$$

  - 傾向形成較緊密的群集。
- Average Linkage
  - 使用兩個群集中所有樣本兩兩距離的平均值作為群集距離。

$$D(A,B) = \frac{1}{|A||B|} \sum_{x_i \in A} \sum_{x_j \in B} d(x_i, x_j)$$

  - 其中
    - $|A|$：群集 A 的樣本數。
    - $|B|$：群集 B 的樣本數。
  - Average linkage 通常比 single linkage 穩定，也比 complete linkage 不那麼嚴格。
- Ward Linkage
  - 會選擇讓群內平方和增加最少的方式合併群集。
  - $$\Delta(A,B) = \frac{|A||B|}{|A|+|B|} \|\mu_A - \mu_B\|^2$$
  - 其中
    - $\Delta(A,B)$：合併群集 A 與 B 所造成的變異增加量。
    - $|A|$、$|B|$：兩群的樣本數。
    - μ_A、μ_B：兩群的質心。
    - $\|\mu_A - \mu_B\|^2$：兩群質心之間的平方距離。
  - 常用於數值型資料，並且傾向產生大小較均衡、群內差異較小的群集。

DBSCAN 分群

全名為 Density-Based Spatial Clustering of Applications with Noise，是一種基於密度的分群方法。與 K-means 不同，DBSCAN 不需要事先指定群數。它會根據資料點附近的密度，把高密度區域視為群集，並把低密度區域的點視為噪音或離群值。

DBSCAN 的重要參數

- ε 或 eps：

$$N_\varepsilon(x_i) = \{x_j \mid d(x_i, x_j) \leq \varepsilon\}$$

  - 其中
    - $N_\varepsilon(x_i)$：以 x_i 為中心、半徑為 ε 的鄰域。
    - x_i：其他樣本點。
    - $d(x_i, x_j)$：兩點之間的距離。
    - ε：半徑參數，用來定義某個樣本點附近的鄰域範圍。
- minPts / min_samples
  - 是判斷核心點所需的最少鄰近樣本數，若某個點的 ε-鄰域中至少有 minPts 個點，則該點為核心點。
  - $$|N_\varepsilon(x_i)| \geq minPts$$
  - 其中
    - $|N_\varepsilon(x_i)|$：第 i 個樣本的 ε-鄰域內樣本數。
    - $minPts$：成為核心點所需的最少樣本數。

DBSCAN 的三種點

- 核心點（Core Point）
  如果一個點的 ε-鄰域內至少包含 minPts 個樣本，則該點為核心點。
- 邊界點（Border Point）
  如果一個點本身不是核心點，但它落在某個核心點的 ε-鄰域內，則該點為邊界點。
- 噪音點（Noise Point）
  如果一個點既不是核心點，也不屬於任何核心點的鄰域，則該點為噪音點或離群值。

DBSCAN 的優點

- 不需要事先指定群數。

DBSCAN 的缺點

- 對 ε 與 minPts 參數敏感。

- 可以發現任意形狀的群集。
- 可以偵測離群值。
- 對非球狀資料比 K-means 更有彈性。
- 若不同群的密度差異很大，效果可能不佳。
- 在高維資料中，距離可能變得不具區辨性。
- 若資料特徵尺度差異大，通常需要先標準化。

在使用 DBSCAN 之前，通常建議對資料進行標準化，但並非絕對必要。是否需要標準化主要取決於以下幾個因素：

- 資料的尺度差異：

  如果資料的不同特徵具有不同的尺度（例如，某些特徵的數值範圍較大，而其他特徵的範圍較小），這會影響到距離的計算。DBSCAN 使用距離來判斷點之間的密度，如果某些特徵的數值範圍大，則這些特徵可能會在計算距離時佔更大比重，影響分群效果。因此，若資料存在明顯的尺度差異，建議先標準化。
- 資料的單位：

  如果特徵的單位不同，例如距離（公里）和時間（小時），也會影響距離的計算，導致某些特徵在分群過程中佔據主導地位。標準化可以消除單位的影響，使得不同單位的特徵在分群中更公平地參與。
- 噪音點的識別：

  DBSCAN 的一個特點是能夠識別出噪音點（outliers），而標準化可以使得這些噪音點更容易被區分出來，減少特徵尺度差異對噪音點判斷的影響。

因此，通常情況下建議進行標準化，特別是在資料特徵範圍相差較大或存在不同單位時。

Module 6. 集成學習

集成學習（Ensemble Learning） 是一種將多個模型結合起來進行預測的方法。這些被結合的模型稱為基礎學習器（Base Learners）或弱學習器（Weak Learners）。集成學習的基本想法可以用一句話說明：「多個模型共同決策，通常比單一模型更穩定。」這與「三個臭皮匠，勝過一個諸葛亮」的概念相似。單一模型可能會因為訓練資料、模型假設或參數設定而產生偏差，但多個模型若能從不同角度學習資料，整合後的結果通常能降低錯誤風險。

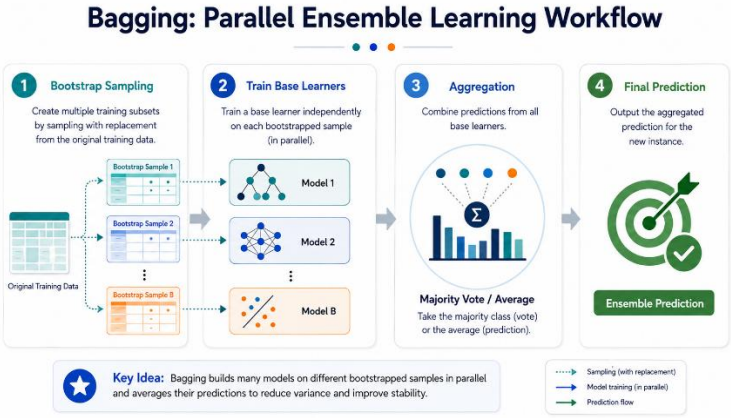
基礎學習器（Base Learner） 指的是被拿來組合集成模型的單一模型。常見的基礎學習器包括：

- 決策樹
- K 最近鄰居
- 邏輯斯迴歸
- 支持向量機
- 單純貝氏分類器
- 決策樹
- 隨機森林
- 其它神經網路模型

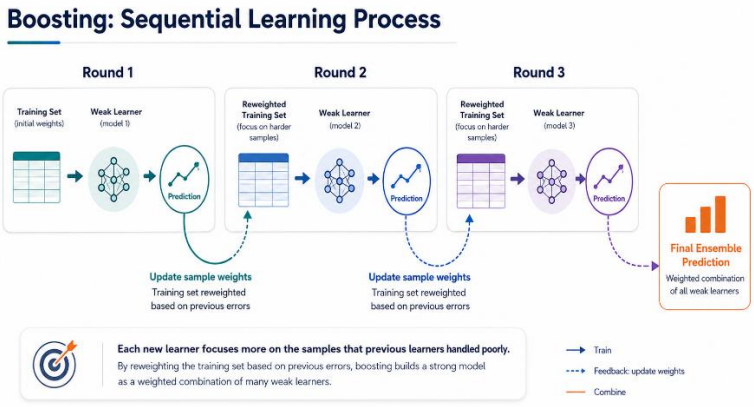
在集成學習中，基礎學習器不一定要是很強的模型。有時候許多簡單模型經過適當組合後，也能得到很好的預測效果。

模型多樣性（Diversity） 是集成學習成功的重要條件。如果多個模型都犯完全一樣的錯誤，那麼即使把它們組合起來，結果也不會明顯改善。相反地，如果不同模型擅長處理不同類型的樣本，錯誤模式彼此不同，集成後就有機會互補。模型多樣性可以來自：

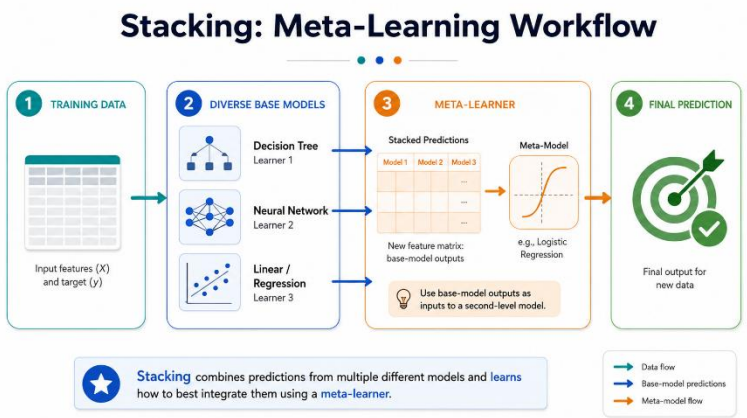
- 使用不同的訓練資料。
- 使用不同的特徵。
- 使用不同的模型演算法。
- 使用相同演算法但不同超參數。
- 使用不同的隨機初始化或抽樣方式。



圖：袋裝法（Bagging）



圖：提升法（Boosting）



圖：堆疊法（Stacking）

類型	中文名稱	代表方法	主要目的
Parallel Ensemble	平行集成	Bagging、Random Forest	降低變異，提升穩定性
Sequential Ensemble	循序集成	AdaBoost、Gradient Boosting	降低偏差，修正前一輪錯誤
Hybrid Ensemble	混合集成	Stacking	結合多種模型優點

表：集成學習大致可以分為三類。參考來源 https://spotintelligence.com/2024/03/18/bagging-boosting-stacking/

特性	Bagging	Boosting	Stacking
Base Model 類型	幾乎可以為同一種	通常為同一種	通常為多種異質模型
Base Model 特性	強模型	弱模型	各具特色的模型
模型間關係	獨立並列	順序修正誤差	層次結構（堆疊）

表：集成學習的模型選擇習慣

可以這樣想：

- Bagging
  - 它主要想解決的是 Variance 過高的問題，也就是模型本身很有能力，但容易因為訓練資料稍微改變，預測結果就大幅改變。
  - 於是它找了很多位能力很強但意見容易受資料影響的專家，最後用投票降低個人判斷的不穩定。
- Boosting
  - 它主要想解決的是 Bias 過高的問題，也就是單一模型能力不夠，容易欠擬合，無法抓到資料中的複雜規律。
  - 於它找了很多位能力普通的學生，第一位先回答，後面的人專門補前面答錯的部分，最後整體答案會越來越好。

袋裝法

袋裝法（Bagging） 是 Bootstrap Aggregating 的縮寫。它的核心概念是：「從原始訓練資料中進行多次隨機抽樣，建立多份不同的訓練子資料集，分別訓練多個模型，最後再將這些模型的預測結果整合起來。」Bagging 是一種平行式集成方法，因為每個基礎模型可以獨立訓練，不需要等待前一個模型的結果。

Bootstrap 抽樣

是指從原始訓練資料中「有放回地」抽樣。假設原始訓練資料為：

$$D = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$$

其中

- D：原始訓練資料集
- x_i：第 i 筆資料的特徵
- y_i：第 i 筆資料的目標值
- n：訓練資料筆數

Bagging 會從資料集 D 中有放回地抽出多個子資料集：

$$D_1, D_2, ..., D_M$$

其中

- D_m：第 m 個 bootstrap 子資料集
- M：基礎模型的數量

因為是有放回抽樣，同一筆資料可能在某個子資料集中出現多次，也可能完全沒有出現。

Aggregating 整合

每個子資料集會訓練出一個基礎模型：

$$h_1(x), h_2(x), ..., h_M(x)$$

其中

- $h_m(x)$：第 m 個基礎模型對輸入 x 的預測
- M：基礎模型總數

Bagging 最後會整合這些模型的預測結果，若多個模型彼此誤差不完全相同，將它們平均後可以降低預測波動（意味著 Bagging 可以降低變異）。

參數	說明
n_estimators	基礎模型數量。數量越多通常越穩定，但訓練時間也會增加
max_samples	每個基礎模型使用多少比例的訓練樣本
max_features	每個基礎模型使用多少比例的特徵

bootstrap	是否使用有放回抽樣
n_jobs	是否平行運算
random_state	控制隨機性，使結果可以重現

表：Bagging 的重要參數

分類問題：使用多數決

在分類問題中，Bagging 通常使用多數決投票。

$$\hat{y} = \arg\max_{c \in C} \sum_{m=1}^{M} I(h_m(x) = c)$$

其中

- $\hat{y}$：最終預測類別
- C：所有可能的類別集合
- c：某一個候選類別
- M：基礎模型數量
- $h_m(x)$：第 m 個模型對 x 的預測類別
- $I(\cdot)$：指示函數，條件成立為 1，不成立為 0

每個模型都對輸入資料 x 投票，最後選擇得票數最多的類別作為最終預測。

迴歸問題：使用平均數

在迴歸問題中，Bagging 通常使用平均值作為最終預測：

$$\hat{y} = \frac{1}{M} \sum_{m=1}^{M} h_m(x)$$

其中

- $\hat{y}$：最終預測值
- M：基礎模型數量
- $h_m(x)$：第 m 個模型對 x 的預測值

多個模型分別給出一個數值預測，最後取平均作為整體模型的預測結果。

隨機森林

隨機森林（Random Forest） 是 Bagging 最具代表性的演算法之一。它以多棵決策樹作為基礎學習器，並透過兩種隨機性建立模型多樣性：

- 對樣本進行 bootstrap 抽樣。
- 在每次節點分裂時，只考慮部分特徵。

單棵決策樹如果深度太大，容易學到訓練資料中的細節與雜訊，因此可能造成過擬合。隨機森林透過建立多棵決策樹，並整合它們的結果，可以降低單棵樹不穩定的問題。

隨機森林的優點包括：

- 通常具有良好的預測表現。
- 能降低單棵決策樹的過擬合風險。
- 可以處理高維度特徵。
- 可以估計特徵重要性。
- 對資料前處理的需求相對較低。

提升法

提升法（Boosting） 是一種循序式集成學習方法。它的核心概念是：「後面的模型會特別關注前面模型預測錯誤的樣本，透過一輪一輪修正錯誤，逐步提升整體模型表現。」Bagging 中的模型通常可以平行訓練；Boosting 則通常是循序訓練，因為後一個模型需要根據前一個模型的錯誤調整學習方向。

弱學習器

弱學習器（Weak Learner） 是指表現只比隨機猜測稍微好一點的模型。在二元分類中，隨機猜測的錯誤率約為 0.5。若一個模型的錯誤率略低於 0.5，就可以視為弱學習器。Boosting 的想法是：即使單一弱學習器不夠強，只要能依序訓練多個弱學習器並適當加權，最後也可能形成強模型。

Boosting 的最終模型可以表示為多個基礎模型的加權組合：

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

其中

- $F(x)$：最終集成模型
- M：基礎模型數量
- $h_m(x)$：第 m 個基礎模型
- α_m：第 m 個模型的權重

若某個模型表現較好，它通常會得到較高的權重；若某個模型表現較差，它的影響力會較低。

AdaBoost

AdaBoost 是 Adaptive Boosting 的縮寫，是 Boosting 的代表性方法之一。它的主要特色是：

- 一開始每筆訓練資料的權重相同。
- 每一輪訓練一個弱學習器。
- 被分類錯誤的樣本，在下一輪會被提高權重。
- 被分類正確的樣本，在下一輪權重相對降低。
- 最後將多個弱學習器依照其表現加權整合。

假設訓練資料共有 n 筆，AdaBoost 一開始會給每筆資料相同權重：

$$w_i^{(1)} = \frac{1}{n}$$

其中

- $w_i^{(1)}$：第 1 輪中第 i 筆資料的權重
- n：訓練資料筆數

接著，在第 m 輪訓練弱學習器 $h_m(x)$，計算其加權錯誤率：

$$\epsilon_m = \frac{\sum_{i=1}^{n} w_i^{(m)} I(y_i \neq h_m(x_i))}{\sum_{i=1}^{n} w_i^{(m)}}$$

其中

- ε_m：第 m 個模型的加權錯誤率

- $w_i^{(m)}$：第 m 輪中第 i 筆資料的權重
- y_i：第 i 筆資料的真實類別
- $h_m(x_i)$：第 m 個模型對第 i 筆資料的預測類別
- $I(\cdot)$：指示函數，條件成立為 1，不成立為 0

模型權重計算方式為：

$$\alpha_m = \frac{1}{2}\ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$$

其中

- a_m：第 m 個模型在最終集成中的權重
- ε_m：第 m 個模型的加權錯誤率

當錯誤率越低，α_m 越大，代表該模型在最終投票中的影響力越大。

接著更新每筆資料的權重：

$$w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$$

其中

- $w_i^{(m+1)}$：下一輪中第 i 筆資料的權重
- $w_i^{(m)}$：目前輪次中第 i 筆資料的權重
- α_m：第 m 個模型的權重
- y_i：真實類別
- $h_m(x_i)$：模型預測類別

如果樣本被分錯，權重會增加；如果樣本被分對，權重會降低。

在二元分類中，AdaBoost 的最終預測可表示為：

$$\hat{y} = \text{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(x)\right)$$

其中

- $\hat{y}$：最終預測類別
- $sign$：判斷正負號的函數
- α_m：第 m 個模型的權重
- $h_m(x)$：第 m 個基礎模型的預測

比較項目	Bagging	Boosting
訓練方式	平行訓練	循序訓練
資料處理	Bootstrap 抽樣	調整樣本權重或學習殘差
主要目的	降低變異	降低偏差
基礎模型	通常可用較複雜模型	通常使用弱學習器
對錯誤樣本的處理	不特別強調	後續模型會特別關注
代表方法	Random Forest	AdaBoost、Gradient Boosting、XGBoost、LightGBM、CatBoost

表：Boosting 與 Bagging 的差異

堆疊法

堆疊法（Stacking, Stacked Generalization） 是一種將多個不同模型的預測結果，再交給另一個模型學習如何整合的方法。它的核心概念是：第一層模型先各自做預測，第二層模型再學習如何根據這些預測產生最後結果。Stacking 通常被視為一種混合集成方法，因為它可以同時結合不同模型、不同特徵處理方式與不同學習策略。

基礎模型

Stacking 的第一層模型稱為 基礎模型（Base Models）。例如：

- KNN
- Random Forest
- Naive Bayes
- Logistic Regression
- SVM
- Gradient Boosting
- Neural Network

這些模型會針對同一批輸入資料產生各自的預測結果。

元學習器

Stacking 的第二層模型稱為元學習器（Meta Learner）或最終模型

（Final Estimator）。元學習器的任務不是直接從原始特徵學習，而是學習：「哪些基礎模型在什麼情況下比較可信。」常見的元學習器包括：

- Logistic Regression
- Linear Regression
- Random Forest
- Gradient Boosting
- XGBoost

基本架構

假設有 M 個基礎模型：

$$h_1(x), h_2(x), ..., h_M(x)$$

每個模型都會對輸入 x 產生預測：

$$z = [h_1(x), h_2(x), ..., h_M(x)]$$

接著，元學習器 g 會使用這些預測結果作為新的輸入：

$$\hat{y} = g(z)$$

也可以寫成：

$$\hat{y} = g(h_1(x), h_2(x), ..., h_M(x))$$

其中

- x：原始輸入特徵
- $h_m(x)$：第 m 個基礎模型的預測
- z：由多個基礎模型預測結果組成的新特徵
- g：元學習器
- $\hat{y}$：最終預測結果

訓練流程

步驟一：訓練基礎模型

使用訓練資料訓練多個基礎模型。這些模型可以是不同演算法，也可以是相同演算法但不同超參數。例如：

- KNN

- Random Forest
- Naive Bayes

步驟二：產生第二層輸入特徵

將每個基礎模型的預測結果整理成新的特徵矩陣。如果有三個基礎模型，第二層資料可能長得像：

樣本	KNN 預測	Random Forest 預測	Naive Bayes 預測
第 1 筆	1	1	0
第 2 筆	0	1	0
第 3 筆	1	0	1

這些預測結果會成為元學習器的輸入。

步驟三：訓練元學習器

元學習器使用第二層輸入特徵與真實標籤進行訓練。它會學習：

- 哪個基礎模型整體表現較好。
- 哪些模型預測一致時較可信。
- 哪些模型可能在特定情況下更有價值。
- 如何組合多個模型的輸出。

步驟四：進行測試與預測

測試階段時，資料會先通過第一層基礎模型，產生多個預測結果。接著，這些預測結果會輸入第二層元學習器，由元學習器產生最終預測。